

Scalable Two-Round n -out-of- n and Multi-Signatures from Lattices in the Quantum Random Oracle Model

Qiqi Lai¹, Feng-Hao Liu², Yang Lu¹, Haiyang Xue³, Yong Yu¹, Yuan Chen⁴

¹ School of Computer Science, Shaanxi Normal University, Xi'an, China.
laiqq@snnu.edu.cn, luyang@snnu.edu.cn, yuyong@snnu.edu.cn

² Washington State University, Pullman, WA, USA. feng-hao.liu@wsu.edu

³ Singapore Management University, Singapore. haiyangxc@gmail.com

⁴ State Key Laboratory of Integrated Services Networks, Xidian University, Xi'an, China. yuanchen@xidian.edu.cn

Abstract. In this paper, we construct the first asymptotically efficient two-round n -out-of- n and multi-signatures from lattices in the quantum random oracle model (QROM), using the Fiat-Shamir with Aborts (FSwA) paradigm. Our protocols can be viewed as the improvements on the two-round protocols by Damgård et al. (JoC 2022).

From a technical perspective, the simulation of QROM and the efficient reduction from breaking underlying assumption to forging signatures are the essential challenges to achieving efficient QROM security for the previously related works. In order to conquer the former one, we adopt the quantum-accessible pseudorandom function (QPRF) to simulate QROM. Particularly, we show that there exists a QPRF that is both invertible and programmable, which can simulate QROM with the same properties. Moreover, we use QPRF as an intermediate tool to successfully simulate QROM whose output is separable, even against a quantum adversary. For the latter challenge, we tweak and apply the online extractability by Unruh (Eurocrypt 2015).

1 Introduction

Due to the applications in the decentralized scenarios, decentralized signatures have recently received wide attentions. *Multi-signature* (MS) and *Threshold signature* (TS) are two similar classes of decentralized signatures.

In a nutshell, MS allows a group of n users, each holding signing key pairs (sk_i, pk_i) , to collectively sign the same message and obtain a succinct signature. On the other hand, TS involves a particular key generation process (regardless of distributed or trusted one), which split the secret key sk into n shares handed out to different participants, such that a message can only be signed if t or more participants agree to do so. The special case of n -out-of- n signature is achieved by setting $t = n$, which is also called as *distributed signature* (DS).

Although these two primitives are quite similar, MS is different from DS in several ways: (i) there is no particular key generation, as each participant generates their own key pairs; (ii) the group of participants is dynamically formed from certain sets; (iii) the verification process does not rely on a fixed public key, but rather on the list or aggregation of the corresponding public keys.

State-of-the-art of Multi/Threshold signature. Numerous recent studies have focused on Schnorr-type schemes, such as the threshold version of ECDSA, threshold Schnorr, and multi-signing of Schnorr. However, Schnorr-type constructions are vulnerable to quantum attacks, it is crucial to investigate the multi and threshold signatures of post-quantum alternatives. Significant attentions have been directed towards lattice-based signatures, including Dilithium [23] and Raccoon [19].

Dilithium is based on the “Fiat-Shamir with Aborts” (FSwA) technique of Lyubashevsky’s signature [37], which employs rejection sampling to guarantee security. Raccoon is also considered as a variant of the Lyubashevsky’s signature, which employs Rényi divergence argument to get rid of rejection sampling. However, unlike Schnorr, thresholding Dilithium or Raccoon presents greater challenges due to the adoption of small distributions. Several schemes with other technique routes have been proposed, but they either rely on costly primitives, such as fully homomorphic encryption(FHE) in [1, 7, 28], or fail to achieve comparable efficiency and full security based on standard assumptions [26].

Damgård et al. [15] address the challenge by using homomorphic trapdoor commitment (HTDC) schemes as a building block. They proposed DS and MS in both two-round and three-round settings, denoted as DS_2/MS_2 and DS_3/MS_3 , respectively. Notably, their DS_2 and MS_2 are the first lattice-based two-round schemes. Subsequently, MulSig-L by Boschini et al. [10] and DualMS by Chen [12] improved upon MS_2 . Compared with MS_2 , MulSig-L does not rely on additional lattice-based HTDC and benefits from preprocessing in the first round. DualMS further enhances the construction by utilizing a trapdoor-free “dual signing simulation” technique, resulting in much better efficiency.

Recently, there are several interesting advancements on TS from lattice assumptions.⁵ del Pino et al. [18] proposed a three-round thresholdized version of Raccoon, named TRaccoon, without relying on any heavy primitives such as FHE or HTDC. In order to further reduce the round complexity, several offline-online two-round TS protocols have been constructed [9, 11, 24]. All these protocols in [9, 11, 24] still require a trusted dealer to generate secret shares. Recently, [16, 41] propose a novel distributed key generation with short share protocols over lattices, which can compile the standard Fiat-Shamir with/without abort signature into the threshold variants.

Common problems in lattice-based setting. All these two-round protocols can only been proven to be secure in the classical random oracle model (ROM), through using the currently known techniques. For post-quantum cryptographic constructions in the ROM, we need to further consider that the quantum adversary might conduct arbitrary superposition queries to ROM, which is further called QROM and first proposed by Boneh et al. in [6]. Currently, only three-round DS and MS in [15, 26] achieve security in the QROM by utilizing the technique of lossy identification [32]. The QROM security for all known existing

⁵ There are also other related works considering adaptive security [31] and robustness [25], but they require four/five rounds. In this paper, we just focus on the construction of two/three-round protocols.

two-round schemes, including DS_2/MS_2 , MulSig-L, DualMS, and recently proposed TS protocols in [11, 24, 24], remains an open problem.

1.1 Our Goals

Therefore, the motivation of this paper is to design asymptotically efficient two-round MS and DS from lattices in the QROM.

(Main Goal 1) Design efficient two-round DS from lattices in the QROM.

(Main Goal 2) Design efficient two-round MS from lattices in the QROM, especially with key aggregation property.

1.2 Our Contributions

This work aims at the above two main goals and makes the following three major contributions.

Contribution 1. We construct the first FSwA-style two-round DS_2 signature from lattices in the QROM, which can be viewed as the significant improvement on two-round protocol by Damgård et al. in [15]. Compared with [15], a notable feature is that our protocol is exactly two-round, i.e., each party performs an independent abort and still outputs a signature in exactly two rounds with polynomial modulus. This makes our schemes significantly more scalable. From the perspective of technique, such exactly two-round is obtained through embedding the protocol challenge in a cut-and-choose way, instead of employing other kinds of Fiat-Shamir without abort techniques, such as noise flooding techniques [1, 24, 28] or relying on Hint-MLWE assumption [9, 11]. We believe our technique for such exactly two-round should be of independently theoretical interest.

For the security proof of our new construction, we leverage the tweaked online extractability technology by Unruh [42] and our newly modified QPRF (which is introduced as the independent technique contribution in the following Contribution 3). Besides, our construction has much lower security loss, as we use online extractability instead of usual rewinding method. Particularly, we improve the adversary's advantage for forging a signature from $\Theta(\varepsilon_{MSIS}^{1/2})$ or $\Theta(\varepsilon_{MSIS}^{1/4})$ to $\Theta(\varepsilon_{MSIS})$, where ε_{MSIS} denotes the hardness of the underlying MSIS assumption.

Contribution 2. We construct the first FSwA-style two-round MS from lattices in the QROM, which has the same properties (i.e., exactly two-round, lower security loss, and polynomial modulus) as the above mentioned n -out-of- n signature. However, compared with n -out-of- n signature, there is an additional obstacle in the security proof. In order to conquer it, we prove the security in the key-verification model, rather than the plain public key model. In the key-verification model, we use as an additional building block a multi-proof straight-line extractable non-interactive zero knowledge proof of knowledge (NIZKPoK). Although such NIZKPoK will induce many more overheads, it gives another nice property, key aggregation.

Contribution 3. As a technical contribution, we make essential modifications on the direct QPRF construction in [44], which was originally proposed by Banerjee, Peikert, and Rosen in [3], such that it becomes an invertible variant. Besides, we show that the invertible QPRF is adaptively programmable even against efficient quantum adversary conducting superposition queries.

With such QPRF, we can prove the security of our two-round protocols in the QROM. Particularly, we can efficiently simulate the adversary’s view even without the real secret key, and then efficiently reduce the security of our constructions to the underlying assumptions. We believe such a new QPRF should be of independent interest.

Overall, we list the comparisons with the most related works in Table 1.

	QROM	Round	Scalable	Reduc.-Appro.	Assumptions
MS ₃ [26]	✓	3	×	Lossy	MLWE, rMLWE
DS ₃ [15]+ [26]	✓	3	×	Lossy	MLWE
DS ₂ [15]	×	2	×	Rewinding	MLWE, MSIS
MS ₂ [15]	×	2	×	Rewinding	MLWE, MSIS
MulSig-L [10]	×	2	×	Rewinding	MLWE, MSIS
DualMS [12]	×	2	×	Rewinding	MLWE, MSIS
TRaccoon [18]	×	3	✓	-	SelfTargetMSIS, PRF, Hint-MLWE
TS ₂ [24]	×	2	✓	Rewinding	AOM-MLWE
TSL [11]	×	2	✓	Rewinding	MSIS
Ringtail [9]	×	2	✓	-	SelfTargetMSIS, PRF, Hint-MLWE, MLWE,
Our QDS ₂	✓	2	✓	Online-Extractability	MLWE, MSIS
Our QMS ₂	✓	2	✓	Online-Extractability	MLWE, MSIS

Table 1. Comparison with previous TS, MS, DS. The column “Reduc. Appro.” indicates the security reduction approach. For two rows TRaccoon and Ringtail, we use “-” to represent the corresponding reduction approach. This is because TRaccoon and Ringtail prove unforgeability directly based on SelfTargetMSIS, rather than much weaker assumption MSIS. However, from the perspective of hardness reduction, we still need to use the rewinding to establish the reduction between SelfTargetMSIS and MSIS.

2 Technical Overview

In this section, we present an overview of our techniques. In fact, our intuition is quite simple: analyze the essential obstacles to proving the existing two-round signatures to be secure in the QROM, then overcome them to achieve the desired security relying on the MLWE, MSIS and DSPR assumptions.

2.1 Recall the existing protocol in [15].

To present our techniques in a natural way, we first recall Damgård et al.’s elegant two-round distributed n -out-of- n protocol, called DS₂, in [15].⁶ The protocols in [15] are based on Dilithium signature scheme [23], and thus work over cyclotomic rings $R = \mathbb{Z}[X]/(f(X))$ and $R_q = \mathbb{Z}_q[X]/(f(X))$, where N is a power of 2, and $f(X) = X^N + 1$ is the $2N$ -th cyclotomic polynomial. *For simplicity, here we just*

⁶ Here, we just focus on the DS₂ construction, as most of the techniques can be easily extended to the setting of multi-signature.

consider the case of 2 parties, which can be naturally generalized to the case of n -party setting. We describe the detailed key and signature generations of DS_2 in [15] in the following Figure 1.

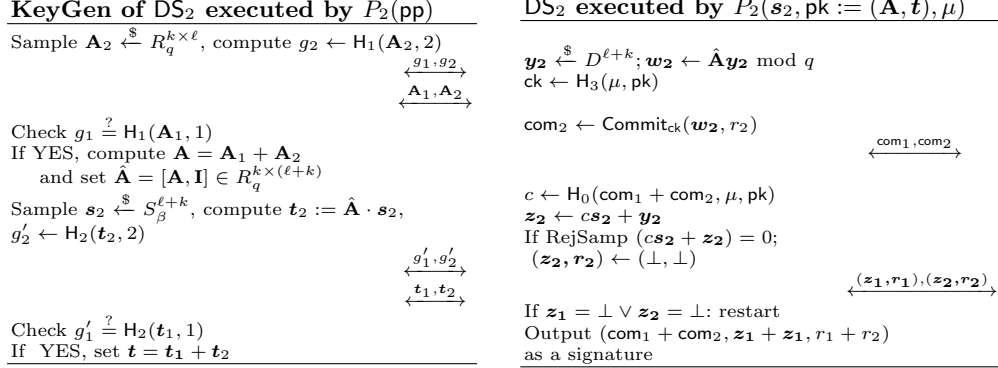


Fig. 1. Simple description of DS_2 Protocol in [15]. Here, we just describe the behaviors of P_2 for saving space, due to the fact that the computations and communications of P_1 are symmetrically equivalent to these of P_2 .

Security analysis of DS_2 . In order to establish the security reduction for DS_2 , there are the following important items needed to be considered:

1. For the key generation, the simulator utilizes the invertible (or called pre-image extractable in certain previous papers) and programmable properties of classical ROM (H_1 and H_2) to recover the adversary's key before programming the honest user's key, and thus embed the MSIS challenge into the public key of DS_2 .
2. For the signature queries, the simulator employs a HTDC scheme to enable the successful simulation of signatures. Moreover, in order to obtain security against concurrent attacks, we need to use the output of H_3 as the commitment key ck , i.e., $\text{ck} \leftarrow H_3(\mu, \text{pk})$.
3. For the reduction from solving hard problems to forging signature, the simulator needs to simulate H_3 such that its output is separable, i.e., for each input pair (μ, pk) , H_3 gets one of two different types of outputs according to certain independent Bernoulli distribution, i.e., gets the normal commitment key ck (i.e., $\text{ck} \leftarrow H_3(\mu, \text{pk})$) for the probability p and the trapdoor commitment key tck (i.e., $\text{tck} \leftarrow H_3(\mu, \text{pk})$) for the probability $(1 - p)$, where $p \in (0, 1)$. Particularly, the uniformity of ck and tck allows the adversary to use ck for the challenge message μ^* , but use tck for all the signature generation queries, with a non-negligible probability. Then, assuming the binding property of the HTDC scheme with normal key ck , the simulator can solve an underlying MSIS problem, through using rewinding technique.

2.2 Enhancing the security of DS_2 into the QROM

We know that there are two main parts for the security proof of DS_2 : the efficient simulations of random oracles and the efficient reduction through rewinding.

Thus, for QROM security, we need to ensure all the proof techniques have the corresponding counterparts for QROM and quantum adversaries.

Simulation of QROM. We need to consider how to simulate QROM efficiently, such that not only it can be both invertible and programmable, but also its output space is separable, following the proof strategy of [15]. Up until now, there are two types of simulation approaches for QROM: stateless simulation (e.g. quantum-secure pseudorandom functions [6, 44] and $2Q$ -wise independent functions [45]), and stateful simulation (e.g. the compressed oracle [21, 34, 46]).

Among them, the compressed oracle is a powerful simulation technique, which supports both inversion and reprogramming [20, 21, 34]. However, it seems that we can not apply the compressed oracle to directly achieve the desired separable property, due to the inconsistency between the commitment key space S_{ck} of tck and the output of compressed oracle.

Particularly, the existing compressed oracle and its variants just support random functions with the following different ranges: uniform distribution over $\{0, 1\}^n$ [34], uniform distribution over abelian groups [13], and non-uniform distribution over $\{0, 1\}^n$ but all outputs are independently sampled [14]. But, tck in the known instantiation of HTDC scheme is the structured matrix, whose distribution is clearly not compatible with the above mentioned compressed oracle techniques in [13, 14, 34].

On the other hand, we can not use stateless simulations, such as QPRF, to directly output tck . This is because for each input pair (μ, pk) , $H_3(\mu, \text{pk})$ should output either tck or ck relying on certain independent randomness, according to the proof strategy of [15]. However, when dealing with superposition queries, these stateless simulations do not have sufficient randomness for all possible input pairs.

Based on the current state of art, we need to improve the existing simulation approaches, such that the resulting QROM satisfies the separable property. Notice that the technical details of the compressed oracle [46] are relatively more complicated. One always needs additional investigations on the related backgrounds. So, as the main technical challenge in this paper, we want to ask

Is there a stateless simulation approach for QROM, which is not only both invertible and programmable, but also its output is separable?

In this paper, our answer is positive. Particularly, our choice is QPRF [44]. We illustrate the high-level idea for using QPRF in the security proof in Figure 2. Of course, we require the concrete instantiation of QPRF is both invertible *and* programmable, even against quantum adversaries, which are significantly non-trivial. As far as we know, it seems that this is the first time to consider how to program and invert QPRF simultaneously in the literature, even the invertible property for PRF has been defined previously in [8, 29]. We believe such QPRF is of independent interest, and can be used on other applications, such as PIR [29].

More precisely, we choose to use the following direct construction of QPRF in [3, 44]: for a key $\mathbf{k} := (\{a_i\}_{i \in [\bar{m}]}, \{s_i\}_{i \in [\bar{\ell}]}) \in \mathcal{K}$ and input $x := (x_1, \dots, x_{\bar{\ell}}) \in \{0, 1\}^{\bar{\ell}}$, let

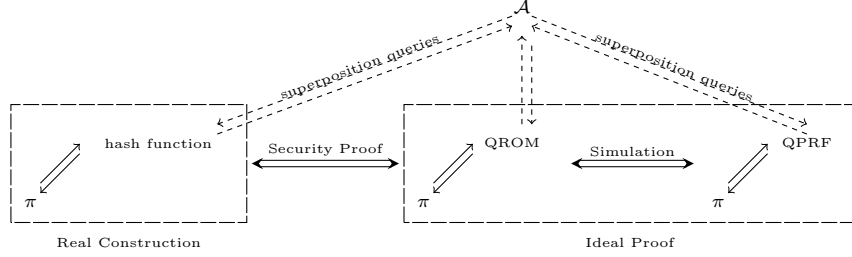


Fig. 2. Illustration of the high-level idea of using QPRF. Clearly, such QPRF is only used in the ideal security proof. In the real constructions, it will be replaced with a good hash function, such as SHA-256. And thus, the instantiation of QPRF will not affect the practical efficiency of our QDS₂ or QMS₂.

$$\text{QPRF}_k(x) = \text{QPRF}_{\{a_i\}, \{s_i\}}(x_1, \dots, x_{\bar{\ell}}) = \left\lfloor \frac{(a_1, \dots, a_{\bar{m}}) \cdot \prod_{i=1}^{\bar{\ell}} s_i^{x_i}}{\bar{p}} \right\rfloor, \quad (1)$$

where $\bar{p}, \bar{q}, \bar{m}, \bar{\ell}$ are integers such that $\bar{q} > \bar{p}$, a_i is chosen from a ring $\bar{R}_{\bar{q}} = \mathbb{Z}_{\bar{q}}[X]/(X^{\bar{N}} + 1)$, s_i is chosen from a small distribution χ over $\bar{R}$, $(a_1, \dots, a_{\bar{m}}) \cdot s = (a_1 \cdot s, \dots, a_{\bar{m}} \cdot s)$, and $\lfloor \cdot \rfloor_{\bar{p}}$ denotes rounding ring elements from $\bar{R}_{\bar{q}}$ to $\bar{R}_{\bar{p}}$.⁷ For certain parameter settings, such QPRF can be proven to be secure, just as in Theorem 4.3. Below, we sketch how to prove the properties of (1).

INVERSION. For the invertible property of QPRF, it is essentially non-trivial. This is because the pseudorandomness of QPRF implicitly implies the one-way property, and it should be impossible to invert the input from certain QPRF output value. But, the situation might be quite different, when the simulator just uses it to simulate QROM. Here the simulator holds the secret key for QPRF, and the adversary can only get outputs through querying the simulator. Even with such new application scenario in our security proof, all existing known QPRFs, including the above (1), still do not satisfy our requirement of inversion.

As one of our significant technical contribution, we tweak the direct QPRF in [44], through (i) embedding a MP trapdoor [39] in the vector $\mathbf{a}^\top = (a_1, \dots, a_{\bar{m}})$; (ii) choosing the specific ring structure such that the small ring element are invertible over $\bar{R}_{\bar{q}}$. Notice that, such a modification will not affect the security of the direct QPRF, as the ring-based learning with errors (RLWE) assumption still holds. Moreover, we can invert such QPRF, i.e., get $\mathbf{x}$ from $\mathbf{y}^\top = \text{QPRF}(\mathbf{x}) \in \bar{R}_{\bar{p}}^{\bar{m}} = (\mathbb{Z}_{\bar{p}}[X]/(X^{\bar{N}} + 1))^{\bar{m}}$, in the following way: (i) with the MP trapdoor in $\mathbf{a}^\top$, we can first get $\hat{s}_0 = \prod_{i=1}^{\bar{\ell}} s_i^{x_i}$ from $\mathbf{y}^\top$, through using the inversion algorithm for the ring-based learning with rounding (RLWR) problem; (ii) in order to determine $x_1 = 0$ or 1 , we directly compute $\hat{s}_1 = s_1^{-1} \cdot \hat{s}_0$. Notice that if $x_1 = 1$, then $\hat{s}_1 = \prod_{i=2}^{\bar{\ell}} s_i^{x_i}$, whose norm should be upper bounded by certain value B with overwhelming probability. Otherwise, $\hat{s}_1 = s_1^{-1} \cdot \prod_{i=2}^{\bar{\ell}} s_i^{x_i}$, whose norm will be

⁷ Here, we use the bar notation to distinguish the notations of QPRF from those of the protocols DS, MS, QDS, QMS. Particularly, the parameter setting of QPRF is independent of QDS₂, QMS₂ protocols in this paper. And thus, the modulus of our protocols is still considered to be polynomial, regardless of the modulus of QPRF.

larger than B with overwhelming probability, according to the decisional small polynomial rate (DSPR) assumption [36]. The detailed inversion algorithm and the related proof are given in Algorithm 1 and Theorem 4.7, respectively.

REPROGRAMMING. In order to prove the reprogrammable property for QPRF, we resort to the existing adaptive programming result and technique for random function in [27, 42]. The high level technique route can be described as follows:

$$\text{QPRF}_k(\cdot) \stackrel{(i)}{\approx} \boxed{\text{RF}(\cdot) \approx \text{RF}'(\cdot)} \stackrel{(ii)}{\approx} \text{QPRF}'_k(\cdot). \quad (2)$$

Here, we use the box to indicate the existing result about the adaptive programming for random functions in [27, 42]. $\text{RF}(\cdot)$ denotes a random function. $\text{RF}'(\cdot)$ and $\text{QPRF}'_k(\cdot)$ denote the programmed functions at certain points. Below, we just need to consider how to prove the steps (i) and (ii) in the above (2).

At the first glance, it seems that the standard security of QPRF, i.e., *oracle indistinguishability* in [44], is sufficient. However, there is still a tiny mismatching! This is because for the box part in the above (2), the adversary not only accesses RF as an oracle, but also takes as input certain pairs $(x, \text{RF}(x))$, where x has sufficient entropy. As a result, it is necessary to introduce a “seemingly” strong definition, *oracle-and-input indistinguishability*, for QPRF as in Definition 3.7. Furthermore, as a justification of such strong security notion, we show that it can be derived from standard oracle indistinguishability in Lemma 3.8.

THE SEPARABLE PROPERTY. Just as analyzed above, we can not make QPRF directly output either ck or tck . In order to conquer this challenge, we employ QPRF as an intermediate tool, i.e., use the output of QPRF as randomness to invoke the corresponding key generation algorithm CGen or TCGen. Particularly, we conduct in the following way: (1) choose a specific key $k_3 \xleftarrow{\$} \mathcal{K}$; (2) divide the output of $\text{QPRF}_{k_3}(\mu, \text{pk})$ into two parts: $ra_1 \in \{0, 1\}^{\ell_{ra_1}}$ and $ra_2 \in \{0, 1\}^{\ell_{ra_2}}$; ⁸ (3) if the number of 1 in ra_1 is larger than certain value num , then we compute $(\text{tck}, \text{td}) \leftarrow \text{Eqv-TCGen}(\text{cpp}_{\text{Eqv}}, ra_2)$, and return tck as the output of H_3 , i.e., define $\text{tck} \leftarrow H_3(\mu, \text{pk})$; otherwise, compute $\text{ck} \leftarrow \text{Eqv-CGen}(\text{cpp}_{\text{Eqv}}, ra_2)$, return ck , i.e., define $\text{ck} \leftarrow H_3(\mu, \text{pk})$. Here, num is set to separate tck and ck with certain probability. cpp_{Eqv} are public parameter, and Eqv-TCGen and Eqv-CGen are two modes of HTDC scheme.

In this case, as the above simulation for H_3 is stateless and a fixed function, we can easily define another fixed function H'_3 , such that it always returns the corresponding td , i.e., define $\text{td} \leftarrow H'_3(\mu, \text{pk})$. According to the pseudorandomness of QPRF, the above simulation for H_3 can not be distinguished by the efficient quantum adversary.

Notice that the above partitioning argument inherently lead to a security loss linear in the number of signing queries. In [40], Pan and Wagner have successfully remove such reduction loss through using pseudorandom matching/path technique. But, there are still other technical obstacles to instantiate their elegant approaches in the lattice-based settings. We leave it as future work.

⁸ Particularly, we define $\text{QPRF}_{k_3}(\cdot) : \{0, 1\}^{l_3^*} \rightarrow (\{0, 1\}^{\ell_{ra_1}} \times \{0, 1\}^{\ell_{ra_2}})$, and compute $(ra_1, ra_2) = \text{QPRF}_{k_3}(\mu, \text{pk})$, where μ denotes the signing message, pk is the generated public key by KeyGen.

FURTHERMORE EFFICIENCY CONSIDERATION. Up until now, we have successfully obtained the desired QPRF in theory, which can simulate QROM satisfying inversion, reprogramming and separation. However, after analyzing the left hand side of Figure 1 deeply, we find the efficiency bottleneck of KeyGen protocol is in its second round, i.e., all participants need to interactively send their own matrix $\mathbf{A}_u \in R_q^{k \times \ell}$. It is preferable to reduce such communication overhead.

Here, our general idea is to introduce another random oracle H'_1 . And each participant's random matrix $\mathbf{A}_u$ is generated as $\mathbf{A}_u \leftarrow H'_1(s_u)$, where $s_u \xleftarrow{\$} \{0, 1\}^{l_1^*}$ is a random seed. In this case, the participants just need to interactively send the $H_1(s_u)$ and s_u , rather than $H_1(\mathbf{A}_u)$ and $\mathbf{A}_u$. Clearly, s_u is significantly shorter than $\mathbf{A}_u$. Particularly, our modified protocol is described in Figure 3.

Modified KeyGen Protocol of QDS₂

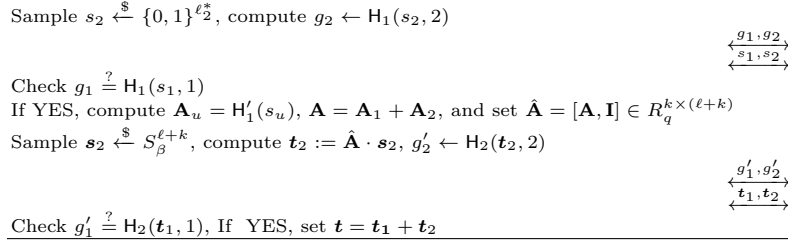


Fig. 3. Simple description of our modified KeyGen protocol for QDS₂ Protocol. Similar to Figure 1, we just describe the behaviors of P_2 for saving space.

For security reduction of DS₂. One of essential obstacles to proving the QROM security of DS₂ is the application of rewinding technique. Although there are several works [33–35] that have solved the dilemma on quantum rewinding for the underlying Dilithium signature scheme [23], these elegant techniques can not be applied to the QROM security of DS₂. This is because all above mentioned techniques require the underlying Σ -protocol to satisfy the collapsing property. But, our Σ -protocol does not satisfy it, due to the usage of equivocal commitment scheme. Up until now, we still do not know how to construct commitment schemes with both equivocal and collapse-binding properties

Another widely used approach of obtaining lattice-based distributed signatures in the QROM is the lossy ID technique [15, 26, 32], which can obtain much tighter security proof. One implicit but crux point in this lossy ID technique is that there should be statistical security in the simulated experiment. However, the HTDC scheme used in the DS₂ protocol just inherently satisfies computational binding, and do not satisfy the essential requirement of lossy technique. Thus, it seems that we need new techniques for proving the QROM security.

2.3 New idea: Online extractability allowing security proof in the QROM.

Online extractability is another reasonable candidate direction to achieve security in the QROM. We notice an existing online extractability technique by Pino and Katsumata in [17].⁹ However, their online extractability technique re-

⁹ In their paper, such a property is named as straight-line extractability.

lies heavily on a primitive called extractable linear homomorphic commitment, which is seemingly inherently not compatible with the equivocation property required for DS_2 in Figure 1. Thus, it is still not clear how to directly apply this online extractability technique to our desired settings.

Fortunately, inspired by Pino and Katsumata’s online extractability technique, we have one crucial observation: the party P_2 might directly generate more than one response $\mathbf{z}_{2,j}$ with respect to the same vector $\mathbf{w}_2 = \mathbf{A} \cdot \mathbf{y}_2$, where each different $\mathbf{z}_{2,j}$ is computed with different challenge $c_{2,j}$. I.e., $\mathbf{z}_{2,j} = \mathbf{y}_2 + c_{2,j} \cdot \mathbf{s}_2$ and $\mathbf{z}_{2,j'} = \mathbf{y}_2 + c_{2,j'} \cdot \mathbf{s}_2$. Based on this, given one forged signature, if we can find two valid responses for two different challenges, we can extract the witness through using the special soundness extractor of the underlying Σ -protocol.

Of course, in order to avoid the trivial extractability from normal valid signature, we need to first hide all different responses by certain hash function, i.e., just send out $h_{2,j} = H(\mathbf{z}_{2,j})$ rather than sending all $\mathbf{z}_{2,j}$ in clear.¹⁰ Then, we can use the idea of cut-and-choose to decide which $\mathbf{z}_{2,j}$ will be disclosed. Notice that if the value j is randomly chosen, we can easily prove its soundness. More importantly, such a new cut-and-choose proof idea provide a chance to allow each participant conducting rejection sampling independently, which is the essential idea to make our protocols to be exactly two-round and scalable.

In fact, the above analysis is matched with the essential idea of Unruh in [42]. Notice that in [20], Don et al. further propose a much better technique to improve the framework of [42]. However, such an improvement can not apply to our distributed signatures.

One more subtlety. Even almost all security targets for two-round protocols have been achieved, there is still one subtlety: the hash function used to hide the responses $\mathbf{z}_j$. Here, as we consider for the case of all parties cooperating to sign message μ , we require it to satisfy the linear homomorphic property. Besides, we also need such a hiding function to have binding and trapdoor-inversion properties, for the reason of security proof. It is a technical challenge to obtain these desired properties for the hiding function in a sufficiently efficient way. Fortunately, we observe that all the responses $\mathbf{z}_j$ follows from the discrete Gaussian distribution, which means all $\mathbf{z}_j$ are small vectors. Based on this significant property, we can utilize the hermite normal form MLWE to instantiate the hiding function with all desired properties.

Particularly, given an uniform matrix $\mathbf{A}_{\text{Inv}} \in R_q^{m' \times (\ell+k)}$ and its trapdoor $\mathbf{T}_{\mathbf{A}_{\text{Inv}}} \in R_q^{m' \times m'}$, we first sample an error vector $\mathbf{r}_j \leftarrow \chi^{m'}$, and then compute $\widetilde{\text{com}}_j = \mathbf{A}_{\text{Inv}} \cdot \mathbf{z}_j + \mathbf{r}_j \in R_q^{m'}$, where the dimension of $\mathbf{z}_j$ is $(\ell+k)$. Clearly, for the adversary without $\mathbf{T}_{\mathbf{A}_{\text{Inv}}}$, $\widetilde{\text{com}}_j$ is computationally distinguishable from uniform, according to the $\text{MLWE}_{q,(\ell+k),m',\chi}$ assumption. But, with $\mathbf{T}_{\mathbf{A}_{\text{Inv}}}$, the simulator can efficiently extract $\mathbf{z}_j$ from $\widetilde{\text{com}}_j$, through using the inversion algorithm for MLWE in [39]. Besides, as $\mathbf{z}_j, \mathbf{r}_j$ are small vectors, the binding of $\widetilde{\text{com}}_j$ follows clearly from the $\text{MSIS}_{q,m',(\ell+k+m'),\beta}$ assumption with respect to $[\mathbf{A}_{\text{Inv}}, \mathbf{I}_{m'}]$, where $\mathbf{I}_{m'}$ is the identity matrix in $R_q^{m' \times m'}$, and $\|(\mathbf{z}_j, \mathbf{r}_j)\| \leq \beta$.

¹⁰ The other one desired property of such a hash function H is collision resistance, which details are deferred to the security proof in Section 5.2.

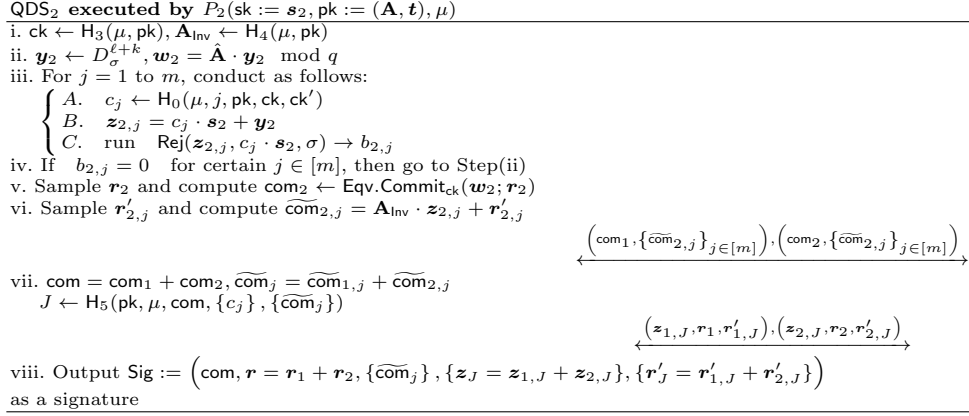


Fig. 4. Our Two-Round n -out-of- n Distributed Signature Protocol

Putting all above ingredients together. We present our main two-round protocol QDS₂ in Figure 4. Below, we slightly analyze QDS₂. Compared with the sign protocol in the right hand side of Figure 1, there are several differences deriving some extra round advantages. We notice that the real challenge for our QDS₂ is J output by the random oracle H_5 . And the challenges $\{c_j\}_{j \in [m]}$ outputted by H_0 are just required to be different from each other, rather than ensuring enough soundness for the underlying Σ -protocol.¹¹ In this case, the parties in QDS₂ first run the rejection sampling algorithm, and then interactively send transcripts, in contrast to the reverse order in the previous DS₂ in [15]. With this particular feature, the outcome of each party's rejection sampling will not affect other parties. And regardless of the number of parties in the system, the whole distributed signature protocol will output the correct signature, after exactly two round interactions. This makes our QDS₂ has the incomparable advantage on the round complexity over other related two-round FSwA-style distributed signature protocols.

In order to ensure the domain of J is large enough, we might need to set the parameter m in Figure 4 to be at least equivalent to 2^λ , where λ is the security parameter. This will clearly cause the construction to be useless. In order to overcome this obstacle, we can first set a relative small value for m (e.g. $m = 2$), and then conduct the parallelization to the current protocol for t times (e.g. $t = \lambda$). The detailed QDS₂ are presented in Figure 8.

2.4 Two-round multi-signature in the QROM.

Similar to [15], we can also convert the above QDS₂ into a two-round multi-signature QMS₂, where our QMS₂ has the nice property of key aggregation. However, after applying all above mentioned techniques, there is still one reduction gap from the fully secure multi-signature. Particularly, through using

¹¹ Here is another difference, that is c_j does not depend on any $\mathbf{w}_j^{(2)}$. But this will not affect our security, due to the following two reasons: (1) the output distribution of rejection sampling algorithms is still simulateable; (2) for the underlying Σ -protocol, the adversary can not forge the valid responses with respect to two different challenges c_{j_1}, c_{j_2} , with $j_1 \neq j_2$.

the above online extractability technique, we can solve the MSIS problem with respect to $[\hat{\mathbf{A}}, \sum_{i \in |L|} a_{j_i} \mathbf{t}_{j_i}]$ from the forged signature Sig^* output by the adversary, where $a_{j_i} = H_{\text{agg}}(L, \mathbf{t}_{j_i})$, $H_{\text{agg}}(\cdot, \cdot)$ is a random oracle, and L is the set of participants in the current running of QMS_2 . Without loss of generality, for multi-signature protocol, we suppose the j_1 -th participant is honest and all others are corrupted together by the adversary. In this case, we should use the reduction algorithm to solve the MSIS problem with respect to $[\hat{\mathbf{A}}, \mathbf{t}_{j_1}]$, rather than $[\hat{\mathbf{A}}, \sum_{i \in |L|} a_{j_i} \mathbf{t}_{j_i}]$. And it seems to be an inherent obstacle for obtaining solutions of MSIS with respect to $[\hat{\mathbf{A}}, \mathbf{t}_{j_1}]$, from that of $[\hat{\mathbf{A}}, \sum_{i \in |L|} a_{j_i} \mathbf{t}_{j_i}]$.

In order to conquer this dilemma, we try to enhance the multi-signature protocol into the key-verification model, where we require each participant to publish a multi-proof straight-line extractable NIZKPoK on his/her secret key sk_j with respect to the corresponding public key pk_j . Then, through using the extractability property of NIZKPoK, we can patch the above mentioned reduction gap, and thus obtain a provably secure multi-signature protocol in the key-verification model. In practice, one participant might want to ensure that the public keys of all his partners are well-formed, before joining into one multi-party protocol. And thus, we believe such a key-verification model is reasonable, even it implicitly implies slightly many more overheads. The formal and detailed protocol of our two-round multi-signature is presented in Section 6.

2.5 Future Directions

In this paper, we only focus on the feasibility of constructing the asymptotically efficient two-round n -out-of- n and multi-signature protocols in the QROM, rather than striving for concrete efficiency. In order to obtain much better concrete efficiency, it is necessary to use more efficient building blocks. Particularly, we can use the more efficient trapdoor-equivocal vector commitment scheme, which can compress many vectors $\mathbf{w}_i$ into one short commitment. One of reasonable candidates of such vector commitment scheme has been proposed in [43]. We treat proposing much more practical commitment schemes and further optimizing the efficiency our QROM secure two-round protocols as interesting directions for future work.

Notice that MulSig-L in [10] and DualMS in [12] can be viewed as significant improvements on MS_2 in [15]. Thus, one might hope to apply our new techniques to [10, 12], which will derive more efficient two-round QMS_2 . But, it seems that the double-forking technique used in [10, 12] is inherently not compatible with our online extractability technique. Of course, if we abandon the key-aggregation property in [10, 12], i.e., just use one-forking in the security proof, it is possible to leverage our techniques to obtain QROM security. But, we still need further analyses. We leave it as an interesting future direction.

3 Preliminaries

Due to space limit, we defer the detailed descriptions on the notations, backgrounds on discrete gaussian distribution, definitions on underlying assumptions such as MSIS, MLWE, DSPR, and rejection sampling together with the signature scheme Dilithium to Sections A.1, A.2, A.3, and A.4, respectively.

3.1 Quantum Computation and Quantum Random Oracle Model

In this Section, we recall several basic results on Quantum Computation and Quantum Random Oracle Model.

Fact 3.1 (Fact 1 in [44]) *For any classical efficiently computable function f , we can efficiently implement it by a quantum computer. Moreover, f can be implemented as an oracle which can be queried on quantum superpositions.*

Definition 3.2 (Quantum Random Oracle, QROM) *Given sets $\mathcal{X}$ and $\mathcal{Y}$, let $\mathcal{Y}^{\mathcal{X}}$ be the set of all functions $H : \mathcal{X} \rightarrow \mathcal{Y}$. The quantum random oracle model (QROM) is a security model, in which any adversary $\mathcal{A}$ gets hash values from the random oracle by querying the oracle on quantum superpositions. Moreover, for a random hash function $H \in \mathcal{Y}^{\mathcal{X}}$, we write $\mathcal{A}^{(H)}$ to denote that $\mathcal{A}$ can query the random oracle H in superpositions.*

There are several ways to simulate QROM. Here, we recall techniques of replacing the random oracle with quantum-secure pseudorandom function (called QPRF, defined in Section 3.2)

Fact 3.3 ([6, 44]) *For any sets $\mathcal{X}$ and $\mathcal{Y}$, we can use quantum-secure pseudorandom function to efficiently simulate quantum random oracle from $\mathcal{X}$ to $\mathcal{Y}$, when considering efficient quantum adversary.*

3.2 Quantum-Secure Pseudorandom Function

Definition 3.4 (PRF [44]) *A pseudorandom function is a function $\text{PRF} : \mathcal{K} \times \mathcal{X} \rightarrow \mathcal{Y}$, where $\mathcal{K}, \mathcal{X}, \mathcal{Y}$ are the key-space, domain and range, respectively. Implicitly, the settings of $\mathcal{K}, \mathcal{X}, \mathcal{Y}$ depend on the security parameter λ . Given any pair $(k, x) \in \mathcal{K} \times \mathcal{X}$, there exists $y \in \mathcal{Y}$, which can be written as $y = \text{PRF}_k(x)$.*

Definition 3.5 (Classical Security) *A pseudorandom function PRF is classical security, if no efficient quantum adversary $\mathcal{A}$ making classical queries can distinguish between a truly random function and the function PRF_k for a random $k \in \mathcal{K}$. More formally, for any efficient quantum adversary $\mathcal{A}$, there exists a negligible function $\varepsilon = \varepsilon(\lambda)$ such that $\left| \Pr_{k \leftarrow \mathcal{K}}[\mathcal{A}^{\text{PRF}_k}(\cdot) = 1] - \Pr_{O \leftarrow \mathcal{Y}^{\mathcal{X}}}[\mathcal{A}^O(\cdot) = 1] \right| < \varepsilon$, where $\mathcal{Y}^{\mathcal{X}}$ denotes the class of all functions from $\mathcal{X}$ to $\mathcal{Y}$.*

Notice that in Definition 3.5, we only allow $\mathcal{A}$ to conduct classical queries. Below, we generalize the definition to allow $\mathcal{A}$ to conduct quantum queries, i.e., directly query one superposition of all $x \in \mathcal{X}$ each time. Such quantum secure pseudorandom functions are called Quantum Pseudorandom Functions, or QPRF.

Definition 3.6 (Quantum Security) *A pseudorandom function PRF is quantum security, if no efficient quantum adversary $\mathcal{A}$ making quantum queries can distinguish between a truly random function and the function PRF_k for a random $k \in \mathcal{K}$. More formally, for any efficient quantum adversary $\mathcal{A}$, there exists a negligible function $\varepsilon = \varepsilon(\lambda)$ such that $\left| \Pr_{k \leftarrow \mathcal{K}}[\mathcal{A}^{|\text{PRF}_k\rangle}(\cdot) = 1] - \Pr_{O \leftarrow \mathcal{Y}^{\mathcal{X}}}[\mathcal{A}^{|\mathcal{O}\rangle}(\cdot) = 1] \right| < \varepsilon$.*

In fact, the above security in Definitions 3.5 and 3.6 are called as Oracle-Indistinguishability, as in [44]. In this paper, we need to use the following “seemingly” strong quantum security: Oracle-and-input indistinguishability.

Definition 3.7 (Strong Quantum Security) *A QPRF is strong quantum security, if no efficient quantum adversary $\mathcal{A}$ making quantum queries and taking several random input-output pairs as input can distinguish between a truly random function and the function PRF_k for a random $k \in \mathcal{K}$. More formally, for any efficient quantum adversary $\mathcal{A}$, there exists a negligible function $\varepsilon = \varepsilon(\lambda)$ such that*

$$\left| \Pr_{(k, x_1, \dots, x_n) \xleftarrow{\$} \mathcal{K} \times \mathcal{X}^n} \left[\mathcal{A}^{|\text{PRF}_k\rangle} \left((x_i, \text{PRF}_k(x_i))_{i \in [n]} \right) = 1 \right] - \Pr_{(O, x_1, \dots, x_n) \xleftarrow{\$} \mathcal{Y}^{\mathcal{X}} \times \mathcal{X}^n} \left[\mathcal{A}^{|O\rangle} \left((x_i, O(x_i))_{i \in [n]} \right) = 1 \right] \right| < \varepsilon.$$

Lemma 3.8 (Oracle-and-input Indistinguishability) *If one PRF satisfies the standard quantum security as in Definition 3.6, then such PRF also satisfies the strong quantum security as in Definition 3.7.*

Due to space limitation, the proof of this lemma is deferred to Section A.5.

3.3 Trapdoor Homomorphic Commitment Scheme

In this section, we recall the notion of trapdoor commitment scheme, called Eqv-Trapdoor Commitment Scheme (Eqv-TCOM). Particularly, for Eqv-TCOM, td is used to equivocate a commitment to an arbitrary message. Of course, the commitment scheme always satisfies the hiding and binding properties. Below, we present the syntaxes for Eqv-trapdoor commitment scheme.

Definition 3.9 (Eqv-Trapdoor Commitment Scheme [15]) *A trapdoor commitment scheme Eqv-TCOM consists of seven algorithms (CSetup, CGen, Commit, Open, TCGen, Eqv-TCommit, Eqv) as follows.*

- $\text{CSetup}(1^\lambda) \rightarrow \text{cpp}$: The setup algorithm takes the security parameter λ as input, and outputs a public parameter cpp defining sets $S_{\text{ck}}, S_{\text{msg}}, S_r, S_{\text{com}}$, and S_{td} and the distribution $\mathcal{D}(S_r)$ from which the randomness is sampled.
- $\text{CGen}(\text{cpp}) \rightarrow \text{ck}$: The key generation algorithm takes cpp as input, and outputs a commitment key from S_{ck} .
- $\text{Commit}_{\text{ck}}(\text{msg}; \text{Rand}) \rightarrow \text{com}$: The commit algorithm takes as input a message $\text{msg} \in S_{\text{msg}}$ and randomness $\text{Rand} \in S_r$, and outputs commitment $\text{com} \in S_{\text{com}}$.
- $\text{Open}_{\text{ck}}(\text{com}, \text{Rand}, \text{msg}) \rightarrow b$: The opening algorithm outputs $b = 1$ if the input tuple is valid, and $b = 0$ otherwise.
- $\text{TCGen}(\text{cpp}) \rightarrow (\text{tck}, \text{td})$: The trapdoor key generation algorithm takes cpp as input, and outputs $\text{tck} \in S_{\text{ck}}$ and the trapdoor $\text{td} \in S_{\text{td}}$.
- $\text{Eqv-TCommit}_{\text{tck}}(\text{td}) \rightarrow \text{com}$: The trapdoor commit algorithm takes tck, td as input, and outputs a commitment $\text{com} \in S_{\text{com}}$.

- $\text{Eqv}_{\text{tck}}(\text{td}, \text{com}, \text{msg}) \rightarrow \text{Rand}$: The equivocation algorithm takes as input $(\text{td}, \text{com}, \text{msg})$, outputs randomness $\text{Rand} \in S_r$, such that $\text{Open}_{\text{tck}}(\text{com}, \text{Rand}, \text{msg}) \rightarrow 1$.

A usual commitment scheme COM is a special case of Eqv-TCOM: it only consists of CSetup, CGen, Commit, and Open. Due to space limitation, we defer the formal presentations of the correctness, hiding, binding, key uniformness, additive homomorphism, and its detailed instantiations to Section A.6.

3.4 n -out-of- n Signature and Multi-Signature

Definition 3.10 (Two-Round DS Protocol) A two-round distributed signature protocol DS_2 consists of the following algorithms.

- $\text{Setup}(1^\lambda) \rightarrow \text{pp}$: The setup algorithm takes a security parameter λ as input, and outputs a public parameter pp .
- $\text{Gen}_j(\text{pp}) \rightarrow (\text{sk}_j, \text{pk})$ for every $j \in [n]$: The interactive key generation algorithm that is run by party P_j . Each P_j runs the protocol on the public parameter pp as input. At the end of the protocol, P_j obtains a secret key share sk_j and public key pk .
- $\text{Sign}_1(\text{pp}, \text{sid}, \text{sk}_j, \text{pk}, \mu) \rightarrow \{\text{msg}_1, \text{st}_1\}$: The first round of interactive signing algorithm is run by party P_j . Each P_j takes as inputs an session ID sid , a pair of signing key and public key (sk_j, pk) , and a message μ . It outputs a protocol message msg_j and a state st_j .
- $\text{Sign}_2(\text{sid}, \text{st}_j, \{\text{msg}_i\}_{i \in [n] \setminus \{j\}}) \rightarrow \text{Sig}_j$: The second round of interactive signing algorithm is run by party P_j . Each P_j takes as inputs an session ID sid , a state information st_j (including a secret key share and all private contributions from the first round) and a set of protocol messages $\{\text{msg}_i\}_{i \in [n] \setminus \{j\}}$. It outputs a partial signature Sig_j .
- $\text{SAgg}(\text{sid}, \{\text{Sig}_1, \dots, \text{Sig}_n\}) \rightarrow \text{Sig}$: The signature aggregation algorithm (which can be run by each of the participants in distributed signature protocol or the third party) takes as inputs an session ID sid , a set of partial signature $\{\text{Sig}_1, \dots, \text{Sig}_n\}$. It outputs a final signature $\widehat{\text{Sig}}$.
- $\text{Ver}(\text{pp}, \text{pk}, \mu, \widehat{\text{Sig}}) \rightarrow b$: The verification algorithm takes as inputs the public parameter pp , the public key pk , a message μ , and a signature $\widehat{\text{Sig}}$. It outputs $b = 1$ if the signature is valid; Otherwise $b = 0$.

Definition 3.11 (Two-Round MS Protocol in the Key-Verification Model)

A two-round multi-signature protocol MS_2 in the key-verification model consists of the following algorithms.

- $\text{Setup}(1^\lambda) \rightarrow \text{pp}$: The set up algorithm that outputs a public parameter pp on a security parameter λ as input.
- $\text{Gen}(\text{pp}) \rightarrow (\text{sk}, \text{pk}, \pi)$: Given a public parameter pp as input, the non-interactive key generation algorithm outputs a key pair (sk, pk) , together with an NIZKPoK proof of validity of the public key pk , denoted as π .
- $\text{KAgg}(L) \rightarrow \text{apk}$: Given a set of public keys $L = \{\text{pk}_1, \dots, \text{pk}_n\}$, the key aggregation algorithm outputs an aggregated public key apk .

- $\text{Sign}_1(\text{pp}, \text{sid}, \text{sk}_j, \text{pk}_j, \mu, L) \rightarrow \{\text{msg}_j, \text{st}_j\}$: The first stage of interactive signing algorithm is run by party P_j . Each P_j takes as inputs an session ID sid , a pair of signing key and public key $(\text{sk}_j, \text{pk}_j)$, a message μ , and a set of co-signers public keys L , and outputs a state st_j as output and a protocol message msg_j .
- $\text{Sign}_2(\text{sid}, \text{st}_j, \{\text{msg}_i\}_{i \in [n] \setminus \{j\}}) \rightarrow \text{Sig}_j$: The second stage of interactive signing algorithm is run by party P_j . Each P_j takes as inputs an session ID sid , a state information st and a set of protocol messages $\{\text{msg}_i\}_{i \in [n] \setminus \{j\}}$, and outputs a partial signature Sig_j .
- $\text{SAgg}(\text{sid}, \{\text{Sig}_1, \dots, \text{Sig}_n\}) \rightarrow \widehat{\text{Sig}}$: The signature aggregation algorithm (which can be run by each of the participants in multi-signature protocol or the third party) takes as inputs an session ID sid , a set of partial signature $\{\text{Sig}_1, \dots, \text{Sig}_n\}$, and outputs a final multi-signature $\widehat{\text{Sig}}$.
- $\text{Ver}(\text{pp}, \widehat{\text{Sig}}, \mu, \text{apk}) \rightarrow b$: The verification algorithm that takes pp , a signature, a message, and a set of public keys and outputs $b = 1$ if the signature is valid. Otherwise $b=0$.
- $\text{KVer}(\text{pp}, \text{pk}, \pi) \rightarrow b$: The key verification algorithm that takes as input pp , pk , and a proof π , and outputs $b = 1$ if pk is a valid public key. Otherwise $b=0$.

Notice that in order to prove the security of our MS_2 against quantum access adversary, we redefine the multi-signature protocol in a more stronger model, i.e., the key-verification model as in [22]. Compared with the plain public key model, this model additionally asks every participant to prove the knowledge of secret key, i.e., publish a NIZKPoK of the used secret key. In this paper, we just focus on how to design the multi-signature protocol itself, since there are existing efficient multi-proof straight-line extractable NIZKPoK protocols for MSIS in the QROM [17, 30], which can be used in a black-box way.

Due to space limitation, we defer the formal security notions for n -out-of- n signature and multi-signature to Section A.7.

4 Simulation of Quantum Random Oracle

In this section, we consider how to simulate QROM through using Quantum secure PRF (QPRF), such that it can be programmable, invertible and separable. Particularly, we notice that the direct QPRF construction in [3, 44], can be used to simulate QROM, according to [6, 44]. Thus, one target of this section is to show that for any efficient quantum adversary conducting superposition queries, the above mentioned direct QPRF construction can be reprogrammable and invertible. The other target of this section is to use QPRF as an intermediate tool to simulate QROM satisfying the separable property.

Below, we first recall a ring-based variant of concrete construction of QPRF in [3, 44]. Then we define a new “injective mode” for such a QPRF, which is computationally close to the original “normal mode”, following from the RLWE assumption. Furthermore, for such “injective mode” QPRF, we present an efficient algorithm, which could invert successfully with certain parameter settings.

Moreover, with the same parameter settings, we show that such QPRF is reprogrammable, i.e., any efficient adversary can not distinguish whether the value $\text{QPRF}_k(x)$ has been redefined or not, when x has sufficient entropy. Finally, we formally define the separable property for random oracle, and show how to simulate QROM with such property, through using QPRF as an intermediate tool.

Besides, we add bar symbol for the variables in this section, in order to indicate that the parameters are locally defined and independent of other parts in this paper.

Construction 4.1 (Direct QPRF in [3, 44]) *Let $\bar{p}, \bar{q}, \bar{d}, \bar{m}, \bar{N}, \bar{\ell}$ be integers with $\bar{q} > \bar{p}$, $\bar{d} = \lceil \log \bar{q} \rceil$, and $\bar{m} = \bar{d} + 2$. Let $\bar{R} = \mathbb{Z}[X]/(X^{\bar{N}} + 1)$ be a $2\bar{N}$ -th cyclotomic ring with $\bar{N}$ being power of 2 and $\bar{R}_{\bar{q}} = \bar{R}/\bar{q}\bar{R}$. Let χ be a small distribution over $\bar{R}$. We define $\text{QPRF} : \mathcal{K} \times \{0, 1\}^{\bar{\ell}} \rightarrow \bar{R}_{\bar{p}}^{1 \times \bar{m}}$ as follows:*

For a key $k := (\{a_i\}_{i \in [\bar{m}]}, \{s_i\}_{i \in [\bar{\ell}]}) \in \mathcal{K}$ and input $x := (x_1, \dots, x_{\bar{\ell}}) \in \{0, 1\}^{\bar{\ell}}$, let $\text{QPRF}_k(x) = \text{QPRF}_{\{a_i\}, \{s_i\}}(x_1, \dots, x_{\bar{\ell}}) = \left[(a_1, \dots, a_{\bar{m}}) \cdot \prod_{i=1}^{\bar{\ell}} s_i^{x_i} \right]_{\bar{p}}$, where $a_i \leftarrow \bar{R}_{\bar{q}}$, $s_i \leftarrow \chi$.

Remark 4.2 *Notice that if $\bar{q}$ is chosen such that $X^{\bar{N}} + 1$ splits into very few irreducible factors modulus $\bar{q}$, and χ is concentrated on ‘small’ elements, then each independent $s_i \leftarrow \chi$ is invertible over $\bar{R}_{\bar{q}}$, according to Corollary 1.2 in [38].*

For the security of the above Construction 3.1, we have the following theorem.

Theorem 4.3 (Generalization of Theorem 6.1 in [44]) *Let $\chi = D_{\bar{R}, \bar{r}}$ be a small distribution over $\bar{R}$, where all coefficients of each polynomial are chosen independently from $D_{\mathbb{Z}, \bar{r}}$. Let $\bar{q} \geq \bar{p} \cdot \bar{\ell} \cdot (\bar{r} \cdot \sqrt{2(\bar{N} + \bar{\ell})} \cdot \omega(\sqrt{\log(\bar{N} + \bar{\ell})}))^{\bar{\ell}} \cdot \bar{N}^{\omega(1)}$. Let QPRF be as in Construction 4.1. If the $\text{RLWE}_{\bar{q}, 1, \bar{m}, \chi}$ holds, then Construction 4.1 is a secure QPRF.*

Generally, the proof idea of this theorem is quite similar to that of Theorem 6.1 in [44], except with the replacement of matrices from $D_{\mathbb{Z}, \bar{r}}^{n \times n}$ with ring elements from $\chi = D_{\bar{R}, \bar{r}}$. In this case, we can still show the security of QPRF through using RLWE. Here, due to space limitation, we defer the detailed proof to Section B.1.

4.1 Inversion of Construction 4.1

In this section, we show that if the vector $\mathbf{a} \in \bar{R}_{\bar{q}}^{\bar{m}}$ is generated together with the trapdoor $\mathbf{T}$ as in [39] and each $s_i \leftarrow \chi$ is invertible over $\bar{R}_{\bar{q}}$, then QPRF in Construction 4.1 can be inverted efficiently. Basically, this is due to the fact that Construction 4.1 is corresponding to the ring learning with rounding (RLWR) problem, which can be inverted efficiently with the related trapdoor.

Particularly, we have the following formal theorems on the RLWR.

Lemma 4.4 (Trapdoors for RLWR [2, 39]) *For any $\bar{N} \geq 1, \bar{q} \geq 2, \bar{d} = \lceil \log \bar{q} \rceil$, $\bar{m} = \bar{d} + 2$, $\bar{p} \geq 3 \cdot \sqrt{\bar{m}\bar{N}} \cdot (\sqrt{2\bar{N}} + \sqrt{\bar{d}\bar{N}})$, there exist the following two efficient algorithms (TrapGen, RLWRInvert).*

$\text{TrapGen}(1^{\bar{N}}, \bar{q}, \bar{m}, \bar{d})$: A PPT algorithm which on input positive integers $\bar{N}, \bar{q}, \bar{m}, \bar{d}$, first samples a vector $(a_1, a_2) \in \bar{R}_{\bar{q}}^2$ and trapdoor $\mathbf{T} \in S_1^{2 \times \bar{d}}$, where $\bar{R}_{\bar{q}} = \mathbb{Z}_{\bar{q}}[X]/(X^{\bar{N}} + 1)$. Furthermore, the algorithm computes $(a_3, \dots, a_{\bar{m}}) = (a_1, a_2)\mathbf{T} + \mathbf{g}^\top$, where $\mathbf{g}^\top = (1, 2, \dots, 2^{\bar{d}-1})$. In this case, $\mathbf{a}^\top = (a_1, \dots, a_{\bar{m}})^\top$ is computationally close to uniform over $R_{\bar{q}}^{\bar{m}}$, according to the $\text{RLWE}_{q,1,1,S_1}$ assumption.

Clearly, it holds $\mathbf{a}^\top \cdot \begin{bmatrix} -\mathbf{T} \\ \mathbf{I}_{\bar{d} \times \bar{d}} \end{bmatrix} = \mathbf{g}^\top$, where $\mathbf{I}_{\bar{d} \times \bar{d}} \in \bar{R}_{\bar{q}}^{\bar{d} \times \bar{d}}$ is an identity matrix.

$\text{RLWRInvert}(\mathbf{T}, \mathbf{a}, \mathbf{b})$: An algorithm taking as input $(\mathbf{a}, \mathbf{T})$ output by $\text{TrapGen}(1^{\bar{N}}, q, \bar{m}, \bar{d})$, and some value $\mathbf{b} \in R_{\bar{p}}^{\bar{m}}$ such that $\mathbf{b}^\top = \lfloor \mathbf{a}^\top \cdot \mathbf{s} \rfloor_{\bar{p}}$ for some $s \in \bar{R}_{\bar{q}}$, outputs s .

Due to space limitation, we defer the detailed proof to Section B.1.

Based on the above result in Lemma 4.4, we can define the following *injective mode* for Construction 4.1, which is almost identical to Construction 4.1 except that $\mathbf{A}$ is generated from the algorithm TrapGen .

Construction 4.5 (Injective mode of Construction 4.1) Let $\bar{p}, \bar{q}, \bar{d}, \bar{m}, \bar{N}, \bar{\ell}$ be integers with $\bar{q} > \bar{p}$, $\bar{d} = \lceil \log \bar{q} \rceil$, and $\bar{m} = \bar{d} + 2$. Let $\bar{R} = \mathbb{Z}[X]/(X^{\bar{N}} + 1)$ be a $2\bar{N}$ -th cyclotomic ring with $\bar{N}$ being power of 2 and $\bar{R}_{\bar{q}} = \bar{R}/\bar{q}\bar{R}$. Let $\chi = D_{\bar{R}, \bar{r}}$ be a small distribution over $\bar{R}$. We define $\text{QPRF} : \mathcal{K} \times \{0, 1\}^{\bar{\ell}} \rightarrow \bar{R}_{\bar{p}}^{1 \times \bar{m}}$ as follows:

For a key $\mathbf{k} := (\{a_i\}_{i \in [\bar{m}]}, \{s_i\}_{i \in [\bar{\ell}]}) \in \mathcal{K}$ and input $x := (x_1, \dots, x_{\bar{\ell}}) \in \{0, 1\}^{\bar{\ell}}$, let $\text{QPRF}_{\mathbf{k}}(x) = \text{QPRF}_{\{a_i\}, \{s_i\}}(x_1, \dots, x_{\bar{\ell}}) = \left\lfloor (a_1, \dots, a_{\bar{m}}) \cdot \prod_{i=1}^{\bar{\ell}} s_i^{x_i} \right\rfloor_{\bar{p}}$, where the vector $\mathbf{a} \in R_{\bar{q}}^{\bar{m}}$ is generated through running the algorithm $\text{TrapGen}(1^{\bar{N}}, \bar{q}, \bar{m}, \bar{d})$, i.e., $(\mathbf{a}, \mathbf{T}) \leftarrow \text{TrapGen}(1^{\bar{N}}, \bar{q}, \bar{m}, \bar{d})$, and $s_i \leftarrow \chi$.

Clearly, for the adversary without the trapdoor matrix $\mathbf{T}$, this injective mode is computationally close to the original *normal mode* in Construction 4.1. Besides, Theorem 4.3 should be still set up in the injective mode, for the adversary without the trapdoor $\mathbf{T}$.

Lemma 4.6 (Indistinguishability of Normal/Injective modes) For the adversary $\mathcal{A}$ without the trapdoor $\mathbf{T}$ of the vector $\mathbf{a}$, if the $\text{RLWE}_{\bar{q},1,1,S_1}$ assumption holds, then Constructions 4.1 and 4.5 are computational indistinguishability, even $\mathcal{A}$ queries the functions in a superposition for any polynomial times.

Due to space limit, we defer the detailed proof to Section B.1.

Below, we describe the concrete invert algorithm for Inj-QPRF .

Theorem 4.7 For some $\mathbf{a} \in R_{\bar{q}}^{\bar{m}}$ and integers $\bar{p}, \bar{q}, \bar{d}, \bar{N}, \bar{m}$ such that $\bar{q} \geq \bar{p} \cdot \bar{\ell} \cdot (\bar{r} \cdot \sqrt{2(\bar{N} + \bar{\ell})} \cdot \omega(\sqrt{\log(\bar{N} + \bar{\ell})}))^{\bar{\ell}} \cdot \bar{N}^{\omega(1)} \geq (\bar{r} \cdot \sqrt{2\bar{N}})^{\bar{\ell}}$, $\bar{d} = \lceil \log \bar{q} \rceil$, and $\bar{m} = \bar{d} + 2$ and $\bar{p} \geq 3 \cdot \sqrt{\bar{m}\bar{N}} \cdot (\sqrt{2\bar{N}} + \sqrt{\bar{d}\bar{N}})$, suppose the oracle $O_{\text{RLWRInvert}}$ in Algorithm 1 correctly invert $\lfloor \mathbf{a}^\top \cdot \mathbf{s} \rfloor_{\bar{p}}$ for any $s \in \bar{R}_{\bar{q}}$. Then, for any invertible $s_i \in \bar{R}_{\bar{q}}$, Algorithm 1 correctly inverts $\text{Inj-QPRF}_{\mathbf{a}, \{s_i\}} = \left\lfloor \mathbf{a}^\top \cdot \prod_{i=1}^{\bar{\ell}} s_i^{x_i} \right\rfloor_{\bar{p}}$, assuming the $\text{DSPR}_{\bar{q}, \bar{R}, \chi}$ assumption.

Due to space limitation, we defer the detailed proof to Section B.1.

Algorithm 1: Efficient algorithm $\text{Invert}^{O_{\text{RLWRInvert}}}(\mathbf{T}, \{a_i\}, \{s_i\}, \{b_i\})$ for inverting the function $\text{Inj-QPRF}_{\{a_i\}, \{s_i\}}(x_1, \dots, x_{\bar{\ell}})$

Input: An oracle $O_{\text{RLWRInvert}}$ for inverting $\lfloor (a_1, \dots, a_{\bar{m}}) \cdot s \rfloor_{\bar{p}}$, when $\bar{p}$ is large enough.

- PRFKey : vector $\mathbf{a} = (a_1, \dots, a_{\bar{m}})^\top \in \bar{R}_{\bar{q}}^{1 \times \bar{m}}$ and $\{s_i\}_{i \in [\bar{\ell}]}$;
- Trapdoor $\mathbf{T} \in \bar{R}^{2 \times \bar{d}}$ for $(a_1, \dots, a_{\bar{m}})$;
- Vector $\mathbf{b} = \left\lfloor \mathbf{a}^\top \cdot \prod_{i=1}^{\bar{\ell}} s_i^{x_i} \right\rfloor_{\bar{p}}$ for any $x_i \leftarrow \{0, 1\}$.

Output: The vector $x = (x_1, \dots, x_{\bar{\ell}}) \in \{0, 1\}^{\bar{\ell}}$.

1. Get $s \leftarrow O_{\text{RLWRInvert}}(\mathbf{T}, \mathbf{a}, \mathbf{b})$, s.t. $\mathbf{b} = \left\lfloor \mathbf{a}^\top \cdot s \right\rfloor_{\bar{p}}$, where $s \in \bar{R}_{\bar{q}}$;
 2. Set $\hat{s} = s$, if $\|\hat{s}\| \geq r^{\bar{\ell}} \cdot (2\bar{N})^{\bar{\ell}/2}$, return $\perp$;
 3. Set $s'_0 = \hat{s}$, for $i = 1, \dots, (\bar{\ell} - 1)$, conduct the following steps:
 - (i) Compute s_i^{-1} , set $s'_i = s_i^{-1} \cdot s'_{i-1}$, where the computation is conducted over $\bar{R}_{\bar{q}}$.
 - (ii) If $\|s'_i\| \leq (\bar{r}\sqrt{2\bar{N}})^{\bar{\ell}-i}$, set $x_i = 1$; Otherwise set $x_i = 0$;
 4. Check if $s'_{\bar{\ell}-1} = s_{\bar{\ell}}$, set $x_{\bar{\ell}} = 1$; Otherwise set $x_{\bar{\ell}} = 0$;
- return** $\mathbf{x} = (x_1, \dots, x_{\bar{\ell}})$.
-

4.2 Adaptive Programming for QPRF in Construction 4.1

In this section, we need to prove that when using the QPRF to simulate QROM, we can conduct adaptive programming similar to the results in [27, 42], which is needed for the security proof of our two-round protocols in the QROM. Particularly, we show that even when conducting quantum queries, an efficient quantum adversary can not distinguish whether the value $\text{QPRF}_k(x)$ in Construction 4.1 has been redefined or not, where x has sufficient entropy. Overall, the result of this section can be viewed as a generalization of the existing results in [27, 42].

Theorem 4.8 (QPRF programming, adaptive) *Let $\text{QPRF} : \mathcal{K} \times (\mathcal{X}_1, \mathcal{X}_2) \rightarrow \mathcal{Y}$ be a quantum secure pseudorandom function for certain sets $\mathcal{K}, \mathcal{X}_1, \mathcal{X}_2, \mathcal{Y}$. For a random key $k \xleftarrow{\$} \mathcal{K}$, consider the following adaptive reprogramming games:*

GAME Repro_b	Reprogram(x_2)
1: $\mathcal{O}_0 := \text{QPRF}_k(\cdot, \cdot)$	
2: $\mathcal{O}_1 := \mathcal{O}_0$	1: $(x_1, y) \xleftarrow{\$} X_1 \times Y$
3: $b' \leftarrow A^{ \mathcal{O}_b , \text{Reprogram}}$	2: $\mathcal{O}_1 := \mathcal{O}_1^{(x_1 \ x_2) \mapsto y}$
4: return b'	3: return x_1

Fig. 5. Adaptive reprogramming games Repro_b for QPRF, with $b \in \{0, 1\}$
 Let $\mathcal{A}$ be any efficient adversary conducting q_1 times quantum queries to QPRF and q_2 times reprogramming queries as defined in the above Figure 5. Then the distinguishing advantage of $\mathcal{A}$ is bounded by

$$\left| \Pr(\text{Repro}_0^{\mathcal{A}} = 1) - \Pr(\text{Repro}_1^{\mathcal{A}} = 1) \right| \leq \frac{3q_2}{2} \sqrt{q_1/|\mathcal{X}_1|} + 2\varepsilon_{\text{QPRF}},$$

where $\varepsilon_{\text{QPRF}}$ is the probability for the efficient quantum adversary to distinguish QPRF and random function.

Due to space limit, we defer the detailed proof to Section B.2.

4.3 Separation of QROM

In this section, we first formally define the separable property for random oracle, and then analyze the essential challenge for such property when against adversary conducting superposition queries. Finally, we show that how to overcome such obstacle through using QPRF in Construction 4.1.

Definition 4.9 (Separation of ROM) *Given a random oracle $H : \mathcal{X} \rightarrow \mathcal{Y}$, we say it is separable, if for each query on $x \in \mathcal{X}$, there exists an independent Bernoulli random variable X satisfying $\Pr(X = 1) = p$, such that $H(x)$ returns the random sample from one of two different but public sampling distributions Y_1, Y_2 supported over $\mathcal{Y}$, according to the value of X , i.e.,*

$$H(x) := \begin{cases} y' & \text{if } X = 1 \\ y'' & \text{if } X = 0 \end{cases}$$

where $y' \leftarrow Y_1, y'' \leftarrow Y_2$.

For classical ROM. In the case of classical random oracle, such separable property can be obtained through simulating H in a lazy sampling way. Particularly, the simulator first initialize an empty table HT . Then, when receiving the adversary's query x , if $HT[x] = \perp$, then the simulator sets $HT[x]$ in the following way: (i) generates an independent Bernoulli $X \in \{0, 1\}$; (ii) if $X = 1$, samples $y_1 \leftarrow Y_1$ with ra_2 as running randomness. Otherwise, samples $y_2 \leftarrow Y_2$ with ra_2 as running randomness; (iii) stores y_1 or y_2 in tables $HT[x]$. If $HT[x] \neq \perp$, then the simulator directly returns $HT[x]$ as the value of $H(x)$.

For QROM. In the case of QROM, it seems that there is an essential challenge for the above lazy sampling simulation approach. Particularly, for compressed oracle and its variants, we need to ensure the compatibility between the distributions Y_1, Y_2 and the output distribution of compressed oracle. Unfortunately, the existing instantiations on lattice-based homomorphic HTDC are not compatible. On the other hand, we can not directly use the stateless simulations neither, as they do not have sufficient randomness for all possible inputs.

In order to overcome the above obstacles, we propose the following simulation algorithm through using QPRF.

1. For any input $x \in \mathcal{X}$, invoke $\text{QPRF}_k(x) : \mathcal{X} \rightarrow (\{0, 1\}^{l_{ra_1}} \times \{0, 1\}^{l_{ra_2}})$, where $k \xleftarrow{\$} \mathcal{K}$, and l_{ra_1}, l_{ra_2} are the bit-lengths of two parts of outputs.
2. Compute $(ra_1, ra_2) = \text{QPRF}_k(x)$.
3. If the number of 1 in ra_1 is more than num , then sample $y_1 \leftarrow Y_1$ through using ra_2 as running randomness, i.e., running the sample algorithm $y_1 \leftarrow \text{Sample}_1(ra_2)$, return y_1 as $H(x)$. Here, num is set to make $\Pr[\|ra_1\|_1 > num] = p$.
4. Otherwise, compute $y_2 \leftarrow Y_2$ through using ra_2 as the running randomness, i.e., running the sample algorithm $y_2 \leftarrow \text{Sample}_2(ra_2)$, return y_2 as $H(x)$.

Fig. 6. Simulation of QROM $H : \mathcal{X} \rightarrow \mathcal{Y}$ with the separable property through using QPRF

Theorem 4.10 *Assume QPRF exists, the above simulation for QROM in Figure 6 satisfies the separable property.*

Due to space limitation, we defer the detailed proof to Section B.3.

Parameter	Description
n	Number of parties
N	A power of two defining the degree of $f(X)$
$f(X) = X^N + 1$	The $2N$ -th cyclotomic polynomial
q	Prime modulus
$R = \mathbb{Z}[X]/(f(X))$	Cyclotomic ring
$R_q = \mathbb{Z}_q[X]/(f(X))$	Ring
k, m'	The height of random matrices $\mathbf{A}$ and $\mathbf{A}_{\text{Inv}}$, respectively
ℓ	The width of random matrices $\mathbf{A}$
$B = \sigma\sqrt{2N(\ell+k)}$	The upper bound of $\ \mathbf{z}_{i,J_i}^{(u)}\ $
$B_n = \sqrt{n}B$	The upper bound of $\ \mathbf{z}_{i,J_i}\ $, with $\mathbf{z}_{i,J_i} = \sum_{u=1}^n \mathbf{z}_{i,J_i}^{(u)}$
$C = \{c \in R : \ c\ _\infty = 1 \wedge \ c\ _1 = \kappa\}$	Challenge space where $ C = \binom{N}{n} 2^\kappa$
$\mathcal{M}$	Message space
κ	The ℓ_1 -norm of challenge $c \in C$
$S_\eta = \{x \in R : \ x\ _\infty \leq \eta\}$	Set of small secrets
m, t	Iteration parameters for Sign protocol
$T = \kappa\eta\sqrt{m \cdot N(\ell+k)}$	The upper bound of $\ (c_{i,j} \mathbf{s}_n)_{j \in [m]}\ $
α	Parameter defining σ and M , according to Lemma A.8
$\sigma = \alpha T$	Standard deviation of the Gaussian distribution of $\mathbf{y}_i^{(n)}$
$M = \exp\left(\sqrt{\frac{2(\lambda+1)}{\log e}} \cdot \frac{1}{\alpha} + \frac{1}{2\alpha^2}\right)$	The expected number of restarts until Rej output 1.
cpp_{Eqv}	Public parameters for commitment scheme, honestly generated by Eqv-CSetup
$l_0, l_1, l'_1 = k \cdot \ell \cdot N \cdot \log q, l_2, l_5 = t \log m$	Output bit lengths of random oracles
$l_0^* = \log(m \cdot t \cdot \mathcal{M}) + k \cdot N \cdot \log q \cdot (\ell + 1)$	Input bit lengths of random oracles H_0 , where $\text{Eqv-}S_{\text{ck}}$
$+ \log \text{Eqv-}S_{\text{ck}} + \log(q \cdot m' \cdot (\ell + k))$	is specified by cpp_{Eqv}
l_1^*	Input bit lengths of random oracles H_1, H'_1
$l_2^* = k \cdot N \cdot \log q + \log n$	Input bit lengths of random oracles H_2
$l_3^* = l_4^* = \log \mathcal{M} + k \cdot N \cdot \log q \cdot (\ell + 1)$	Input bit lengths of random oracles H_3, H_4
$l_5^* = k \cdot N \cdot \log q \cdot (\ell + 1) + \log \mathcal{M} $	Input bit lengths of random oracles H_5 , where $\text{Eqv-}S_{\text{com}}$
$+ t \log \text{Eqv-}S_{\text{com}} + mt \log(2N\kappa(q \cdot m' \cdot (\ell + k)))$	is specified by cpp_{Eqv} .

Table 2. Parameters of Our Two Round n -out-of- n Threshold Signature

5 Two Round n -out-of- n Threshold Signature from lattices in the QROM

In this section, we present one of our main constructions: two-round n -out-of- n threshold signature, which is provably secure based on MSIS, MLWE and DSPR in the QROM. Below, we first describe our protocol in Section 5.1, and then prove the correctness and the security of unforgeability in Section 5.2.

5.1 Construction

Generally, our protocol can be viewed as enhancing the security of the existing protocol by Damgård et al. in [15] from classical ROM into the QROM, through leveraging the tweaked online extractability framework by Unruh in [42] and our modified QPRF in Construction 4.5. Similar to [15], we need to use as a building block an additively HTDC scheme Eqv-TCOM with uniform keys.

Particularly, our construction is formally specified in Figures 7, 8, 9. Here, as all players have the same role, we just describe the n -th player's behavior. In order to help the readers to understand our construction more easily, we go over the high-level ideas for each step as follows.

Parameter setup. According to Definition 3.10, the algorithm $\text{QDS}_2.\text{Setup}$ should be invoked by a trusted party, and outputs a set of public parameters as in Table 2. Notice that most of our parameters follow from those of [15], except with the following case:

Protocol QDS₂.Gen_n(pp):
The protocol is parameterized by public parameters described in Table 2 and relies on the random oracles: $H_1 : \{0, 1\}^{l_1^*} \rightarrow \{0, 1\}^{l_1}$, $H'_1 : \{0, 1\}^{l_1^*} \rightarrow \{0, 1\}^{l'_1}$, $H_2 : \{0, 1\}^{l_2^*} \rightarrow \{0, 1\}^{l_2}$.

Matrix Generation

1. Sample a random seed $s_n \in \{0, 1\}^{l_1^* - \log n}$, and generate a random oracle commitment $g_n \leftarrow H_1(s_n, n)$. Send out g_n .
2. Upon receiving g_u for all $u \in [n-1]$, send out the seed s_n .
3. Upon receiving s_u for all $u \in [n-1]$:
 - (a) If $H_1(s_u, u) \neq g_u$ for some u , then send out $\perp$.
 - (b) Otherwise compute $\mathbf{A}_u = H'_1(s_u, u)$ for all $u \in [n]$. And set public random matrix $\bar{\mathbf{A}} := [\mathbf{A} | \mathbf{I}] \in R_q^{k \times (\ell + k)}$, where $\mathbf{A} := \sum_{u \in [n]} \mathbf{A}_u$.

Key Pair Generation

1. Sample a secret key shares $\mathbf{s}_n \xleftarrow{\$} S_{\eta}^{\ell+k}$ and compute a public key share $\mathbf{t}_n := \bar{\mathbf{A}} \mathbf{s}_n$, respectively, and generate a random oracle commitment $g'_n \leftarrow H_2(\mathbf{t}_n, n)$. Send out g'_n .
2. Upon receiving g'_u for all $u \in [n-1]$, send out $\mathbf{t}_n$.
3. Upon receiving $\mathbf{t}_u$ for all $u \in [n-1]$:
 - (a) If $H_2(\mathbf{t}_u, u) \neq g'_u$ for some u then send out $\perp$.
 - (b) Otherwise set a combined public key $\mathbf{t} := \sum_{u \in [n]} \mathbf{t}_u$.

If the protocol does not abort, P_n obtain $(\text{sk}_n, \text{pk}) = (\mathbf{s}_n, (\mathbf{A}, \mathbf{t}))$ as local output.

Fig. 7. Gen Protocol of Our Two-Round n -out-of- n Threshold Signature Scheme

- As we want to generalize the framework of Unruh in [42] into the threshold setting, it is necessary to replace the random oracle for hashing the vector $\mathbf{z}_j$ as an additively homomorphic trapdoor-inversion function indexed by the matrix $\mathbf{A}_{\text{Inv}} \in R_q^{m' \times (\ell + k)}$. Particularly, such function is defined to be $\text{Inv-H}(\mathbf{z}_j) := \mathbf{A}_{\text{Inv}} \cdot \mathbf{z}_j + \mathbf{r}_j$, where $\mathbf{r}_j$ is randomly sampled from the same small distribution as that of $\mathbf{z}_j$. Thus, we need to set the dimensions of $\mathbf{A}_{\text{Inv}}$ accordingly, ensuring the function Inv-H to satisfy both binding and hiding properties. Moreover, with the corresponding trapdoor $\mathbf{T}_{\mathbf{A}_{\text{Inv}}} \in R^{m' \times m'}$, it should be efficient to get $\mathbf{z}_j$ from $\text{Inv-H}(\mathbf{z})$.

Key generation. The key generation algorithm QDS₂.Gen almost follows that of [15], except that we introduce another random oracle H'_1 as the randomness generator. Particularly, in order to interactively generate a random matrix $\mathbf{A} \in R_q^{k \times \ell}$ in a secure way, the n -th participant employs the following random oracle commitments: first choose his random seed $s_n \xleftarrow{\$} \{0, 1\}^{l_1^*}$, then compute and send out $g_n \leftarrow H_1(s_n, n)$. Then with s_u for all $u \in [n]$, any one can generate the random matrix $\mathbf{A}_u \xleftarrow{\$} H'_1(s_u, u)$. Due to the uniform distribution of s_n , the input of H'_1 has sufficient entropy. And thus we can reprogram H'_1 in the security proof. Notice that in this case, the participants just need to send out the seed $s_u \in \{0, 1\}^{l_2^*}$, rather than $\mathbf{A}_u \in R_q^{k \times \ell}$, in the public channel. Clearly, this will significantly reduce the communication overhead of our construction.

Signature generation. One important point for the QDS₂.Sign_n algorithm in Figure 8 is the iterations at (1.a) of **Signature generation**. With these steps, we can realize online extractability, according to [42]. And thus, we can circumvent the essential obstacle, rewinding, for the security proof of signature in the QROM. The other crucial point is the computation of $c_{i,j}$, i.e., $c_{i,j} \leftarrow H_0(i, j, \mu, \text{pk}, \text{ck}, \text{ck}')$. In fact, this step has at least three significance:

- For fixed i and different j and j' , $c_{i,j} \neq c_{i,j'}$. This is necessary for successful extractability through using the extractor Ext presented in Figure 20, according to [42].



Fig. 8. Sign Protocol of Our Two-Round n -out-of- n Threshold Signature Scheme

- The computation of $c_{i,j}$ does not rely on $\text{com}_i^{(u)}$ or $w_i^{(u)}$. And thus, for each $(i, j) \in [t] \times [m]$, all participants will use the same challenge $c_{i,j}$ for the related individual running of underlying Dilithium signature scheme. Clearly, only with such condition, $\{z_{i,J_i}\}_{i \in [t]}$ in the final signature can be verified successfully with respect to public key (A, t) , according to the step (2.c) of the algorithm QDS₂.Ver in Figure 9.
- Although each challenge $c_{i,j} \leftarrow H_0(i, j, \mu, pk, ck, ck')$ remains to be unchanged across all different participants, it does not affect the simulation of the transcript $(w_i^{(n)}, c_{i,J_i}, z_{i,J_i})$. Particularly, we first sample z_{i,J_i} from the desired distribution, and then compute $w_i^{(n)} = \overline{A}z_{i,J_i}^{(n)} - c_{i,J_i}t_n$. Finally, with the help of the trapdoor-equivocation commitment scheme Eqv-TCOM, we can derive the related randomness $r_i^{(n)} \leftarrow \text{Eqv}_{\text{tck}}(\text{td}, \text{com}_i^{(n)}, w_i^{(n)})$. This simulation is crucial to the Game 6 in the proof of Theorem 5.2.

Verification. Thanks to the linearity of the underlying signature scheme, and additive homomorphism of Eqv-TCOM and Inv-H(z) with respect to both message and randomness, the verifier just need to verify the sum of signature shares, i.e., $\sum_{u=1}^n \text{Sig}_u$, where Sig_u is the signature share from each participant.

Algorithm QDS₂.Ver(($\{\text{com}_i\}_{i \in [t]}$, $\{\widehat{\text{com}}_{i,j}\}_{i \in [t], j \in [m]}$, $\{\mathbf{z}_i\}_{i \in [t]}$, $\{\mathbf{r}_i\}_{i \in [t]}$, $\{\mathbf{r}'_i\}_{i \in [t]}$, μ , pk)
 Upon receiving a message μ , signature $(\{\text{com}_i\}_{i \in [t]}, \{\widehat{\text{com}}_{i,j}\}_{i \in [t], j \in [m]}, \{\mathbf{z}_i\}_{i \in [t]}, \{\mathbf{r}_i\}_{i \in [t]}, \{\mathbf{r}'_i\}_{i \in [t]})$, and combined public key $\text{pk} := (\mathbf{A}, \mathbf{t})$ works as follows:
 1. Generate commitment keys $\text{ck} \leftarrow \text{H}_3(\mu, \text{pk})$, $\mathbf{A}_{\text{Inv}} \leftarrow \text{H}_4(\mu, \text{pk})$, derive $c_{i,j} \leftarrow \text{H}_0(i, j, \mu, \text{pk}, \text{ck}, \text{ck}')$ for all $i \in [t], j \in [m]$ and compute $J_1 || \dots || J_t \leftarrow \text{H}_5(\text{pk}, \mu, \{\text{com}_i\}_{i \in [t]}, \{c_{i,j}\}_{i \in [t], j \in [m]}, \{\widehat{\text{com}}_{i,j}\}_{i \in [t], j \in [m]})$.
 2. Perform the checks as follows:
 (a) for $i = 1$ to t do:
 Check that $c_{i,1}, \dots, c_{i,m}$ pairwise distinct.
 (b) for $i = 1$ to t do:
 Check that $\|\mathbf{z}_i\| \leq B_n$.
 (c) for $i = 1$ to t do:
 Reconstruct $\mathbf{w}_i := \overline{\mathbf{A}}\mathbf{z}_i - c_{i,J_i}\mathbf{t}$, and check $\text{Eqv-Open}_{\text{ck}}(\text{com}_i, \mathbf{r}_i, \mathbf{w}_i) = 1$.
 (d) for $i = 1$ to t do:
 Check $\widehat{\text{com}}_{i,J_i} = \mathbf{A}_{\text{Inv}} \cdot \mathbf{z}_i + \mathbf{r}'_i$.
 If all checks succeed then return 1, otherwise, return 0.

Fig. 9. Ver Algorithm of Our Two-Round n -out-of- n Threshold Signature Scheme

5.2 Correctness and Security

Theorem 5.1 (Correctness) *For public parameters as in Table 2, two-round threshold n -out-of- n signature $\text{QDS}_2 = (\text{Setup}, (\text{Gen}_u)_{u \in [n]}, (\text{Sign}_u)_{u \in [n]}, \text{Ver})$ in Figures 7, 8, 9 satisfies the correctness. In other words, suppose the underlying Dilithium scheme is correct, and the trapdoor commitment scheme Eqv-TCOM and the trapdoor-inversion function Inv-H are correct and additively homomorphic, then a valid generated signatures will be accepted by the verification algorithm, except with a negligible probability.*

Due to space limitation, we defer the detailed proof of this theorem to Section C. Below, we focus on the security of our QDS_2 construction.

Theorem 5.2 *Suppose that the trapdoor commitment scheme Eqv-TCOM is hiding, binding, additively homomorphic, and has a secure equivocation trapdoor, uniform keys and uniform commitment. Particularly, the output of $\text{Eqv-TCommit}_{\text{tck}}(\text{td})$ has sufficient bit-length ϑ . And suppose there exists QPRF that can be programmable and invertible simultaneously under $\text{DSPR}_{\bar{q}, \bar{R}, \chi}$ assumption. For any quantum polynomial-time adversary $\mathcal{A}$ that initiates a single key generation protocol by querying $\mathcal{O}_{\text{Gen}}(\cdot, \cdot)$, initiates Q_s signature generation protocols by querying $\mathcal{O}_{\text{Sign}_1}(\cdot, \cdot), \mathcal{O}_{\text{Sign}_2}(\cdot, \cdot)$ with sid , and makes Q_h quantum superpositions queries to random oracle $\text{H}_0, \text{H}_1, \text{H}'_1, \text{H}_2, \text{H}_3, \text{H}_4, \text{H}_5$, the protocol QDS_2 of Figures 7, 8, 9 is QDS-UF-CMA secure under $\text{MSIS}_{q,k,k+\ell+1,\beta}$ and $\text{MLWE}_{q,\ell,k,\eta}$ assumptions in the QROM, where $\beta = 2\sqrt{B_n^2 + \kappa}$. Concretely, using other parameters specified in Table 2, the advantage of $\mathcal{A}$ is bounded as follows.*

$$\begin{aligned} \text{Adv}_{\text{QDS}_2}^{\text{UF-CMA}}(\mathcal{A}) &\leq 2\varepsilon_{\text{Inj-QPRF}} + 5\varepsilon_{\text{QPRF}} + e(Q_h + Q_s + 1) \left[(Q_h + Q_s) \cdot t \cdot \varepsilon_{\text{td}} \right. \\ &\quad + 2(Q_h + Q_s) \cdot \varepsilon_{\text{QPRF}} + \frac{3Q_s}{2} \sqrt{Q_s + Q_h} 2^{-\frac{t \cdot \vartheta}{2}} + 2\varepsilon_{\text{QPRF}} + t \cdot Q_s \cdot (m - 1) \\ &\quad \cdot \text{Adv}_{\text{MLWE}_{q,(\ell+k),m',\chi}} + t \cdot Q_s \cdot \varepsilon_{\text{Rej}} + \frac{3}{2} \sqrt{Q_h} (2^{-\frac{l_1^*}{2}} + 2^{-\frac{kN \log q}{2}}) \\ &\quad + 4(\varepsilon_{\text{QPRF}} + \varepsilon_{\text{Inj-QPRF}}) + \text{Adv}_{\text{MLWE}_{q,\ell,k,\eta}} + 2(Q_h + 1) 2^{-(t \log m)/2} + \varepsilon_{\text{bind}} \\ &\quad \left. + \text{Adv}_{\text{MSIS}_{q,k,k+\ell+1,\beta}} \right] \end{aligned}$$

Here, $\varepsilon_{\text{QPRF}}$ denotes the advantage for an efficient quantum adversary distinguishing QROM and QPRF in Construction 4.1, where $\varepsilon_{\text{QPRF}} = \text{Adv}_{\text{RLWE}_{\bar{q},1,\bar{m},\chi}}$

Algorithm QMS₂.Gen(pp)
 Given as inputs the public matrix $\bar{\mathbf{A}} := [\mathbf{A}|\mathbf{I}] \in R_q^{k \times (\ell+k)}$ together with the common reference string CRS of the used multi-proof NIZKPoK system Π , the Algorithm conducts the following steps:

1. Sample a secret key shares $\mathbf{s}_n \leftarrow S_{\eta}^{\ell+k}$ and compute a public key share $\mathbf{t}_n := \bar{\mathbf{A}}\mathbf{s}_n$;
2. Run $\Pi.\text{Prove}(\text{CRS}, \bar{\mathbf{A}}, \mathbf{t}_n, \mathbf{s}_n)$ to output a NIZKPoK proof π_n as an appendix of public key.
3. Output $(\text{sk}_n, \text{pk}_n) = (\mathbf{s}_n, (\mathbf{t}_n, \pi_n))$ as local output.

Algorithm QMS₂.KVer(pp, pk_n)
 Given pp, pk_n as input, the algorithm conducts the following steps:

1. Parse $\text{pk}_n := (\mathbf{t}_n, \pi_n)$, where $\mathbf{t}_n \in R_q^k$ and π_n is a proof of NIZKPoK system Π ;
2. Runs $\Pi.\text{Verify}(\text{CRS}, \bar{\mathbf{A}}, \mathbf{t}_n, \pi_n)$ to output 1 if π_n is a valid proof; Otherwise output 0.

Algorithm QMS₂.KAgg(pp, L)
 Given pp and the public key set $L := \{\text{pk}_1, \dots, \text{pk}_{(n'-1)}, \text{pk}_n\}$ as input, the algorithm conducts the following steps:

1. Parse each public key $\text{pk}_j \in L$ as $\text{pk}_j := (\mathbf{t}_j, \pi_j)$, and run $\text{QMS}_2.\text{KVer}(\text{pp}, \text{pk}_j)$. If all public keys pass the key verification algorithm $\text{QMS}_2.\text{KVer}(\cdot, \cdot)$, then conduct the following steps; Otherwise, abort.
2. For $j \in [n' - 1]$, compute $a_j := H_{\text{agg}}(L, \mathbf{t}_j)$, and compute $a_n := H_{\text{agg}}(L, \mathbf{t}_n)$;
3. Compute and return $\tilde{\mathbf{t}} = \sum_{j=1}^{n'-1} a_j \cdot \mathbf{t}_j + a_n \cdot \mathbf{t}_n$.

Fig. 10. Gen, KVer, KAgg algorithms of Our Two-Round Multi-Signature Scheme
 with $\chi = D_{\bar{R}, \bar{r}} \cdot \varepsilon_{\text{Inj-QPRF}}$ denotes the advantage distinguishing injective QPRF in Construction 4.5 from the direct Construction 4.1, where $\varepsilon_{\text{Inj-QPRF}} = \text{Adv}_{\text{RLWE}_{q,1,1,S_1}} + \text{DSPR}_{\bar{q}, \bar{R}, \chi} \cdot \varepsilon_{\text{td}}$ is the statistical distances of true commitment key for Eqv-TCOM and the uniform. ε_{Rej} is the statistical distances of the output distribution of rejection sampling algorithm and the ideal distribution. $\varepsilon_{\text{bind}}$ are the advantages of breaking Eqv-TCOM, for any efficient quantum adversary. Moreover, all these values are negligible according to the related instantiations in this paper.
 Due to space limitation, we defer the proof of this theorem to Section C.

6 Two Round Multi-Signature from lattices in the QROM

In this section we describe our two-round multi-signature QMS₂ in the key-verification model. We remark that with the help of the multi-proof straight-line extractable NIZKPoK in the key generation stage, our QMS₂ can be proven secure relying on essentially the same idea as QDS₂. And the main difference from n -out-of- n signature is that, the protocol requires no interactive key generation at all, and instead for each signing execution a party receives a set of public keys L together with a message to be signed. Particularly, our construction of two-round multi-signature $\text{QMS}_2 = (\text{Setup}, \text{Gen}, \text{KVer}, \text{KAgg}, \text{Sign}, \text{Ver})$ is formally specified in Figures 10, 11, 12. As the number of participants may change for each signing attempt, in this section we define n to be the maximum number of signers allowed in a single execution of signing protocol. Without loss of generality, we assume that each signer assign the index n to itself, and consider other signers' indices as $1, \dots, (n' - 1)$, where $n' = |L| \leq n$. Due to space limitation, we defer the formal definition of the multi-proof straight-line extractable NIZKPoK systems to Section D, which can be instantiated in the QROM as in [17, 30].

6.1 Construction

As the construction and proof of QMS₂ is almost same to these of QDS₂, below we highlight the differences in red color.

Given a multi-proof straight-line extractable NIZKPoK system $\Pi = (\Pi.\text{Setup}, \Pi.\text{Prove}, \Pi.\text{Verify})$ for $\mathcal{L}_{B,q}$ just as defined in Definition D.1, we make a brief overview of our QMS₂ scheme as follows.

- The **Setup** works just like the one for QDS_2 , but it additionally outputs a matrix $\bar{\mathbf{A}} = [\mathbf{A}|\mathbf{I}] \in R_q^{k \times (\ell+k)}$ as part of public parameters, so we assume that $\bar{\mathbf{A}}$ is generated by a trusted third party. And the input lengths of QROM is changed and we show these as follows.

- $l_0^* = \log(m \cdot t \cdot |\mathcal{M}|) + k \cdot N \cdot \log q \cdot (n+1) + \log |\text{Eqv-}S_{\text{ck}}| + \log |\text{Inv-}S_{\text{ck}}|$
- $l_3^* = l_4^* = \log |\mathcal{M}| + n \cdot k \cdot N \cdot \log q$
- $l_5^* = nk \cdot N \log q + \log |\mathcal{M}| + t \log |\text{Eqv-}S_{\text{com}}| + mt \log(2N\kappa |\text{Inv-}S_{\text{com}}|)$

Besides, the **Setup** algorithm runs $\Pi.\text{Setup}$ to output the common string reference CRS.

- The **Gen**, **KVer**, **KAgg** algorithms are formally specified in Figure 10.
- The signing protocol **Sign** and verification **Ver** are described in Figures 12 and 11. The main differences from $\text{QDS}_2.\text{Sign}$ and $\text{QDS}_2.\text{Ver}$ is that at the beginning stages of $\text{QDS}_2.\text{Sign}$ and $\text{QDS}_2.\text{Ver}$, each participant need to first verify the well-formedness of other participant's public keys.

Protocol $\text{QMS}_2.\text{Ver}(\{\text{com}_i\}_{i \in [t]}, \{\widehat{\text{com}}_{i,j}\}_{i \in [t], j \in [m]}, \{\mathbf{z}_i\}_{i \in [t]}, \{\mathbf{r}_i\}_{i \in [t]}, \{\mathbf{r}'_i\}_{i \in [t]}, \mu, \mathbf{L})$
Upon receive a message μ , signature $\text{Sig} = (\{\text{com}_i\}_{i \in [t]}, \{\widehat{\text{com}}_{i,j}\}_{i \in [t], j \in [m]}, \{\mathbf{z}_i\}_{i \in [t]}, \{\mathbf{r}_i\}_{i \in [t]}, \{\mathbf{r}'_i\}_{i \in [t]})$, and a set of public keys $\mathbf{L}$, if $|\mathbf{L}| > n$ then reject the signature. Otherwise work as follows:

1. Check public key in $\mathbf{L}$, i.e., run $\Pi.\text{Verify}$ algorithm. If there exist certain $(\mathbf{t}_j, \pi_j) \in \mathbf{L}$ such that $\Pi.\text{Verify}(\text{CRS}, \bar{\mathbf{A}}, \mathbf{t}_j, \pi_j) = 0$, then send out $\perp$. Otherwise, conduct the following steps.
2. Run the algorithm to $\text{QMS}_2.\text{KAgg}(\text{pp}, \mathbf{L})$ to generate the aggregated key $\text{apk} := \tilde{\mathbf{t}}$.
3. Generate commitment keys $\text{ck} \leftarrow \text{H}_3(\mu, \mathbf{L})$ and $\mathbf{A}_{\text{inv}} \leftarrow \text{H}_4(\mu, \mathbf{L})$, Compute challenges $c_{i,j} \leftarrow \text{H}_0(i, j, \mu, \text{ck}, \mathbf{A}_{\text{inv}}, \mathbf{L})$. Then Compute $J_1 || \dots || J_t \leftarrow \text{H}_5(\mathbf{L}, \mu, \{\text{com}_i\}_{i \in [t]}, \{c_{i,j,u}\}_{i \in [t], j \in [m], u \in [n']}, \{\widehat{\text{com}}_{i,j}\}_{i \in [t], j \in [m]})$, where $n' = |\mathbf{L}|$.
4. Perform the checks as follows:
 - (a) for $i = 1$ to t do:
Check that $c_{i,1}, \dots, c_{i,m}$ pairwise distinct.
 - (b) for $i = 1$ to t do:
check that $\|\mathbf{z}_i\| \leq B_n$.
 - (c) for $i = 1$ to t do:
Reconstruct $\mathbf{w}_i := \bar{\mathbf{A}}\mathbf{z}_i - c_{i,J_i}\tilde{\mathbf{t}}$, check $\text{Eqv-Open}_{\text{ck}}(\text{com}_i, \mathbf{r}_i, \mathbf{w}_i) = 1$.
 - (d) for $i = 1$ to t do:
Check $\widehat{\text{com}}_{i,J_i} = \mathbf{A}_{\text{inv}} \cdot \mathbf{z}_i + \mathbf{r}'_i$.

If all checks succeed then return 1, otherwise, return 0.

Fig. 11. Ver Algorithm of Our Two-Round Multi-Signature Scheme

6.2 Correctness and Security

As the correctness of QMS_2 is quite similar to that of Theorem 5.1, here we omit it for simplicity. Below, we just focus on the security. Similar to QDS_2 in Section 5, we show that our QMS_2 is UF-CMA secure.

Theorem 6.1 *Suppose that the trapdoor commitment scheme Eqv-TCOM is hiding, binding, additively homomorphic, and has a secure equivocation trapdoor, uniform keys and uniform commitment. Particularly, the output of $\text{Eqv-TCommit}_{\text{tck}}(\text{td})$ has sufficient bit-length ϑ . Suppose $\Pi = (\Pi.\text{Setup}, \Pi.\text{Prove}, \Pi.\text{Verify})$ is a multi-proof straight-line extractable NIZKPoK system for $\mathcal{L}_{B,q}$, just as defined in Definition D.1. And suppose there exists QPRF that can be programmable and invertible simultaneously. For any quantum polynomial-time adversary $\mathcal{A}$ that initiates a single key generation protocol by querying $\mathcal{O}_n^{\text{QMS}_2}$ with $\text{sid} = 0$, initiates $\mathcal{Q}_s$ signature generation protocols by querying $\mathcal{O}_n^{\text{QMS}_2}$ with $\text{sid} \neq 0$, and makes*



Fig. 12. Sign Protocol of Our Two-Round Multi-Signature Scheme

Q_h quantum superpositions queries to random oracle H_0, H_3, H_4, H_5 , the protocol QMS₂ of Figures 10, 12, 11 is QMS-SUF-CMA secure under MSIS_{q,k,k+\ell+1,\beta} and MLWE_{q,\ell,k,\eta} assumptions in the QROM, where $\beta = \sqrt{(2B_n)^2 + 4\kappa^2 + \eta^2(4\kappa^2 \cdot (\ell + k))}$. Concretely, using other parameters specified in Table 2, the advantage of $\mathcal{A}$ is

bounded as follows.

$$\begin{aligned}
\mathbf{Adv}_{\text{QMS}_2}^{\text{UF-CMA}}(\mathcal{A}) &\leq 2\varepsilon_{\text{Inj-QPRF}} + 5\varepsilon_{\text{QPRF}} + e(Q_h + Q_s + 1) \left[(Q_h + Q_s) \cdot t \cdot \varepsilon_{\text{td}} \right. \\
&\quad + 2(Q_h + Q_s) \cdot \varepsilon_{\text{QPRF}} + \frac{3Q_s}{2} \sqrt{Q_s + Q_h} 2^{-\frac{t \cdot \vartheta}{2}} + 2\varepsilon_{\text{QPRF}} + t \cdot Q_s \cdot (m - 1) \\
&\quad \cdot \mathbf{Adv}_{\text{MLWE}_{q,(\ell+k),m',\chi}} + t \cdot Q_s \cdot (\varepsilon_{\text{Rej}} + \varepsilon_{\text{td}}) + \frac{3}{2} \sqrt{Q_h} (2^{-\frac{t_1^*}{2}} + 2^{-\frac{kN \log q}{2}}) \\
&\quad + 4(\varepsilon_{\text{QPRF}} + \varepsilon_{\text{Inj-QPRF}}) + \mathbf{Adv}_{\text{MLWE}_{q,\ell,k,\eta}} + 2(Q_h + 1) 2^{-(t \log m)/2} + \varepsilon_{\text{bind}} \\
&\quad \left. + \mathbf{Adv}_{\text{MSIS}_{q,k,k+\ell+1,\beta}} \right]
\end{aligned}$$

Due to space limit, we defer the analysis to Section D.2.

References

1. S. Agrawal, D. Stehlé, and A. Yadav. Round-optimal lattice-based threshold signatures, revisited. In M. Bojanczyk, E. Merelli, and D. P. Woodruff, editors, *ICALP 2022*, volume 229 of *LIPICs*, pages 8:1–8:20. Schloss Dagstuhl, July 2022.
2. J. Alwen, S. Krenn, K. Pietrzak, and D. Wichs. Learning with rounding, revisited - new reduction, properties and applications. In R. Canetti and J. A. Garay, editors, *CRYPTO 2013, Part I*, volume 8042 of *LNCS*, pages 57–74. Springer, Berlin, Heidelberg, Aug. 2013.
3. A. Banerjee, C. Peikert, and A. Rosen. Pseudorandom functions and lattices. In D. Pointcheval and T. Johansson, editors, *EUROCRYPT 2012*, volume 7237 of *LNCS*, pages 719–737. Springer, Berlin, Heidelberg, Apr. 2012.
4. C. Baum, I. Damgård, V. Lyubashevsky, S. Oechsner, and C. Peikert. More efficient commitments from structured lattice assumptions. In D. Catalano and R. De Prisco, editors, *SCN 18*, volume 11035 of *LNCS*, pages 368–385. Springer, Cham, Sept. 2018.
5. M. Bellare, E. C. Crites, C. Komlo, M. Maller, S. Tessaro, and C. Zhu. Better than advertised security for non-interactive threshold signatures. In Y. Dodis and T. Shrimpton, editors, *CRYPTO 2022, Part IV*, volume 13510 of *LNCS*, pages 517–550. Springer, Cham, Aug. 2022.
6. D. Boneh, Ö. Dagdelen, M. Fischlin, A. Lehmann, C. Schaffner, and M. Zhandry. Random oracles in a quantum world. In D. H. Lee and X. Wang, editors, *ASIACRYPT 2011*, volume 7073 of *LNCS*, pages 41–69. Springer, Berlin, Heidelberg, Dec. 2011.
7. D. Boneh, R. Gennaro, S. Goldfeder, A. Jain, S. Kim, P. M. R. Rasmussen, and A. Sahai. Threshold cryptosystems from threshold fully homomorphic encryption. In H. Shacham and A. Boldyreva, editors, *CRYPTO 2018, Part I*, volume 10991 of *LNCS*, pages 565–596. Springer, Cham, Aug. 2018.
8. D. Boneh, S. Kim, and D. J. Wu. Constrained keys for invertible pseudorandom functions. In Y. Kalai and L. Reyzin, editors, *TCC 2017, Part I*, volume 10677 of *LNCS*, pages 237–263. Springer, Cham, Nov. 2017.
9. C. Boschini, D. Kaviani, R. W. F. Lai, G. Malavolta, A. Takahashi, and M. Tibouchi. Ringtail: Practical two-round threshold signatures from learning with errors. Cryptology ePrint Archive, Report 2024/1113, 2024.

10. C. Boschini, A. Takahashi, and M. Tibouchi. MuSig-L: Lattice-based multi-signature with single-round online phase. In Y. Dodis and T. Shrimpton, editors, *CRYPTO 2022, Part II*, volume 13508 of *LNCS*, pages 276–305. Springer, Cham, Aug. 2022.
11. R. Chairattana-Apirom, S. Tessaro, and C. Zhu. Partially non-interactive two-round lattice-based threshold signatures. Cryptology ePrint Archive, Report 2024/467, 2024.
12. Y. Chen. DualMS: Efficient lattice-based two-round multi-signature with trapdoor-free simulation. In H. Handschuh and A. Lysyanskaya, editors, *CRYPTO 2023, Part V*, volume 14085 of *LNCS*, pages 716–747. Springer, Cham, Aug. 2023.
13. K.-M. Chung, S. Fehr, Y.-H. Huang, and T.-N. Liao. On the compressed-oracle technique, and post-quantum security of proofs of sequential work. In A. Canteaut and F.-X. Standaert, editors, *EUROCRYPT 2021, Part II*, volume 12697 of *LNCS*, pages 598–629. Springer, Cham, Oct. 2021.
14. J. Czajkowski, C. Majenz, C. Schaffner, and S. Zur. Quantum lazy sampling and game-playing proofs for quantum indistinguishability. Cryptology ePrint Archive, Report 2019/428, 2019.
15. I. Damgård, C. Orlandi, A. Takahashi, and M. Tibouchi. Two-round n-out-of-n and multi-signatures and trapdoor commitment from lattices. *Journal of Cryptology*, 35(2):14, Apr. 2022.
16. R. del Pino, T. Espitau, G. Niot, and T. Prest. Simple and efficient lattice threshold signatures with identifiable aborts. Cryptology ePrint Archive, Report 2025/871, 2025.
17. R. del Pino and S. Katsumata. A new framework for more efficient round-optimal lattice-based (partially) blind signature via trapdoor sampling. In Y. Dodis and T. Shrimpton, editors, *CRYPTO 2022, Part II*, volume 13508 of *LNCS*, pages 306–336. Springer, Cham, Aug. 2022.
18. R. Del Pino, S. Katsumata, M. Maller, F. Mouhartem, T. Prest, and M.-J. O. Saarinen. Threshold raccoon: Practical threshold signatures from standard lattice assumptions. In M. Joye and G. Leander, editors, *EUROCRYPT 2024, Part II*, volume 14652 of *LNCS*, pages 219–248. Springer, Cham, May 2024.
19. R. del Pino, T. Prest, M. Rossi, and M. O. Saarinen. High-order masking of lattice signatures in quasilinear time. In *44th IEEE Symposium on Security and Privacy, SP 2023, San Francisco, CA, USA, May 21-25, 2023*, pages 1168–1185. IEEE, 2023.
20. J. Don, S. Fehr, C. Majenz, and C. Schaffner. Efficient NIZKs and signatures from commit-and-open protocols in the QROM. In Y. Dodis and T. Shrimpton, editors, *CRYPTO 2022, Part II*, volume 13508 of *LNCS*, pages 729–757. Springer, Cham, Aug. 2022.
21. J. Don, S. Fehr, C. Majenz, and C. Schaffner. Online-extractability in the quantum random-oracle model. In O. Dunkelman and S. Dziembowski, editors, *EUROCRYPT 2022, Part III*, volume 13277 of *LNCS*, pages 677–706. Springer, Cham, May / June 2022.
22. M. Drijvers, K. Edalatnejad, B. Ford, E. Kiltz, J. Loss, G. Neven, and I. Stepanovs. On the security of two-round multi-signatures. In *2019 IEEE Symposium on Security and Privacy*, pages 1084–1101. IEEE Computer Society Press, May 2019.
23. L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, P. Schwabe, G. Seiler, and D. Stehlé. CRYSTALS-Dilithium: A lattice-based digital signature scheme. *IACR TCHES*, 2018(1):238–268, 2018.

24. T. Espitau, S. Katsumata, and K. Takemure. Two-round threshold signature from algebraic one-more learning with errors. In L. Reyzin and D. Stebila, editors, *CRYPTO 2024, Part VII*, volume 14926 of *LNCS*, pages 387–424. Springer, Cham, Aug. 2024.
25. T. Espitau, G. Niot, and T. Prest. Flood and submerge: Distributed key generation and robust threshold signature from lattices. In L. Reyzin and D. Stebila, editors, *CRYPTO 2024, Part VII*, volume 14926 of *LNCS*, pages 425–458. Springer, Cham, Aug. 2024.
26. M. Fukumitsu and S. Hasegawa. A lattice-based provably secure multisignature scheme in quantum random oracle model. In K. Nguyen, W. Wu, K.-Y. Lam, and H. Wang, editors, *ProvSec 2020*, volume 12505 of *LNCS*, pages 45–64. Springer, Cham, Nov. / Dec. 2020.
27. A. B. Grilo, K. Hövelmanns, A. Hülsing, and C. Majenz. Tight adaptive reprogramming in the QROM. In M. Tibouchi and H. Wang, editors, *ASIACRYPT 2021, Part I*, volume 13090 of *LNCS*, pages 637–667. Springer, Cham, Dec. 2021.
28. K. D. Gür, J. Katz, and T. Silde. Two-round threshold lattice-based signatures from threshold homomorphic encryption. In M.-J. Saarinen and D. Smith-Tone, editors, *Post-Quantum Cryptography - 15th International Workshop, PQCrypto 2024, Part II*, pages 266–300. Springer, Cham, June 2024.
29. A. Hoover, S. Patel, G. Persiano, and K. Yeo. Plinko: Single-server PIR with efficient updates via invertible PRFs. Cryptology ePrint Archive, Report 2024/318, 2024.
30. S. Katsumata. A new simple technique to bootstrap various lattice zero-knowledge proofs to QROM secure NIZKs. In T. Malkin and C. Peikert, editors, *CRYPTO 2021, Part II*, volume 12826 of *LNCS*, pages 580–610, Virtual Event, Aug. 2021. Springer, Cham.
31. S. Katsumata, M. Reichle, and K. Takemure. Adaptively secure 5 round threshold signatures from MLWE/MSIS and DL with rewinding. In L. Reyzin and D. Stebila, editors, *CRYPTO 2024, Part VII*, volume 14926 of *LNCS*, pages 459–491. Springer, Cham, Aug. 2024.
32. E. Kiltz, V. Lyubashevsky, and C. Schaffner. A concrete treatment of Fiat-Shamir signatures in the quantum random-oracle model. In J. B. Nielsen and V. Rijmen, editors, *EUROCRYPT 2018, Part III*, volume 10822 of *LNCS*, pages 552–586. Springer, Cham, Apr. / May 2018.
33. R. W. F. Lai, G. Malavolta, and N. Spooner. Quantum rewinding for many-round protocols. In E. Kiltz and V. Vaikuntanathan, editors, *TCC 2022, Part I*, volume 13747 of *LNCS*, pages 80–109. Springer, Cham, Nov. 2022.
34. Q. Liu and M. Zhandry. Revisiting post-quantum Fiat-Shamir. In A. Boldyreva and D. Micciancio, editors, *CRYPTO 2019, Part II*, volume 11693 of *LNCS*, pages 326–355. Springer, Cham, Aug. 2019.
35. A. Lombardi, F. Ma, and N. Spooner. Post-quantum zero knowledge, revisited or: How to do quantum rewinding undetectably. In *63rd FOCS*, pages 851–859. IEEE Computer Society Press, Oct. / Nov. 2022.
36. A. López-Alt, E. Tromer, and V. Vaikuntanathan. On-the-fly multiparty computation on the cloud via multikey fully homomorphic encryption. In H. J. Karloff and T. Pitassi, editors, *44th ACM STOC*, pages 1219–1234. ACM Press, May 2012.
37. V. Lyubashevsky. Lattice signatures without trapdoors. In D. Pointcheval and T. Johansson, editors, *EUROCRYPT 2012*, volume 7237 of *LNCS*, pages 738–755. Springer, Berlin, Heidelberg, Apr. 2012.

38. V. Lyubashevsky and G. Seiler. Short, invertible elements in partially splitting cyclotomic rings and applications to lattice-based zero-knowledge proofs. In J. B. Nielsen and V. Rijmen, editors, *EUROCRYPT 2018, Part I*, volume 10820 of *LNCS*, pages 204–224. Springer, Cham, Apr. / May 2018.
39. D. Micciancio and C. Peikert. Trapdoors for lattices: Simpler, tighter, faster, smaller. In D. Pointcheval and T. Johansson, editors, *EUROCRYPT 2012*, volume 7237 of *LNCS*, pages 700–718. Springer, Berlin, Heidelberg, Apr. 2012.
40. J. Pan and B. Wagner. Chopsticks: Fork-free two-round multi-signatures from non-interactive assumptions. In C. Hazay and M. Stam, editors, *EUROCRYPT 2023, Part V*, volume 14008 of *LNCS*, pages 597–627. Springer, Cham, Apr. 2023.
41. R. D. Pino and G. Niot. Finally! A compact lattice-based threshold signature. In T. Jager and J. Pan, editors, *PKC 2025, Part III*, volume 15676 of *LNCS*, pages 169–199. Springer, Cham, May 2025.
42. D. Unruh. Non-interactive zero-knowledge proofs in the quantum random oracle model. In E. Oswald and M. Fischlin, editors, *EUROCRYPT 2015, Part II*, volume 9057 of *LNCS*, pages 755–784. Springer, Berlin, Heidelberg, Apr. 2015.
43. H. Wee and D. J. Wu. Succinct vector, polynomial, and functional commitments from lattices. In C. Hazay and M. Stam, editors, *EUROCRYPT 2023, Part III*, volume 14006 of *LNCS*, pages 385–416. Springer, Cham, Apr. 2023.
44. M. Zhandry. How to construct quantum random functions. In *53rd FOCS*, pages 679–687. IEEE Computer Society Press, Oct. 2012.
45. M. Zhandry. Secure identity-based encryption in the quantum random oracle model. In R. Safavi-Naini and R. Canetti, editors, *CRYPTO 2012*, volume 7417 of *LNCS*, pages 758–775. Springer, Berlin, Heidelberg, Aug. 2012.
46. M. Zhandry. How to record quantum queries, and applications to quantum indistinguishability. In A. Boldyreva and D. Micciancio, editors, *CRYPTO 2019, Part II*, volume 11693 of *LNCS*, pages 239–268. Springer, Cham, Aug. 2019.

A Supplementary for Preliminaries

Due to the space limitation in the main body, we present many more supplementary materials for Preliminaries in Section 3

A.1 Notations

In this paper, $\mathbb{Z}$ and $\mathbb{R}$ denote the sets of integers and real numbers. For positive integers n, q , let $[n]$ denotes the set $\{1, \dots, n\}$ and $\mathbb{Z}_q$ denotes the ring of integers modulo q . We use λ to denote the security parameter, which is the implicit input for all algorithms presented in this paper. A function $f(\lambda) > 0$ is negligible and denoted by $\text{negl}(\lambda)$ if for any $c > 0$ and sufficiently large λ , $f(\lambda) < 1/\lambda^c$. A probability is called to be overwhelming if it is $1 - \text{negl}(\lambda)$. A column vector is denoted by a bold lower case letter (e.g., $\mathbf{x}$). A matrix is denoted by a bold upper case letter (e.g., $\mathbf{A}$), and its transposition is denoted by $\mathbf{A}^\top$. Let $R = \mathbb{Z}[x]/(x^N + 1)$ be a cyclotomic ring, with N be a power of 2. The norm of an element in $R_q = \mathbb{Z}_q[x]/(x^N + 1)$ will be the norm of its unique representative with coefficients in $[-(q-1)/2, (q-1)/2]$. For positive $\beta \in \mathbb{R}$, we use S_β to denote the set of all polynomials of infinity norm less than β , i.e., $S_\beta = \{a \in R \mid \|a\|_\infty \leq \beta\}$.

We define a rounding function $[\cdot]_p : \mathbb{Z}_q \rightarrow \mathbb{Z}_p$ for $q \geq p \geq 2$ as $[x]_{q \rightarrow p} = \lfloor (p/q)\bar{x} \rfloor_{q \rightarrow p}$, where $\bar{x} \in \mathbb{Z}$ is any integer congruent to $x \bmod q$. Furthermore, $[\cdot]_{q \rightarrow p}$ can be extended component-wise to vectors and matrices over $\mathbb{Z}_q$. Especially, for a ring element $a \in R$ represented as coefficient embedding, we first view it as the vector consisting of all its coefficients, and then conduct rounding function to such vector. In places where the context is clear about the modulus q , we would omit q in the notation as $[\cdot]_p$ for simplicity of presentation.

For a distribution or a set D , we write $x \stackrel{\$}{\leftarrow} D$ to denote the operation of sampling an uniformly random x according to D . We denote as $\text{Supp}(D)$ the support of a distribution D . For two distributions D_1, D_2 , we let $\text{SD}(D_1, D_2)$ denote their statistical distance. We write $D_1 \stackrel{s}{\approx} D_2$ to mean that they are statistically close, and $D_1 \stackrel{c}{\approx} D_2$ to say that they are computationally indistinguishable. The collision entropy of a random variable X is $-\log \Pr[X = X']$ where X' is independent of X and has the same distribution. The min-entropy of X is $\min_x (-\log \Pr[X = x])$.

Matrix norms. For a vector $\mathbf{x}$, its Euclidean norm (also known as the ℓ_2 norm) is defined as $\|\mathbf{x}\| = (\sum_i x_i^2)^{1/2}$. For a matrix $\mathbf{R}$, we denote its i -th column vector as $\mathbf{r}_i$, and use $\tilde{\mathbf{R}}$ to denote its Gram-Schmidt orthogonalization. In addition,

- $\|\mathbf{R}\|$ denotes the Euclidean norm of $\mathbf{R}$, i.e., $\|\mathbf{R}\| = \max_i \|\mathbf{r}_i\|$.
- $s_1(\mathbf{R})$ denotes the spectral norm of $\mathbf{R}$, i.e., $s_1(\mathbf{R}) = \sup_{\|\mathbf{x}\|=1} \|\mathbf{R}\mathbf{x}\|$, with $\mathbf{x} \in \mathbb{Z}^m$.

Besides, we have the following lemma for the bounding spectral norm.

Lemma A.1 *Let $\mathbf{X} \in \mathbb{R}^{n \times m}$ be a subgaussian random matrix with parameter s . There exists a universal constant $c \approx 1/\sqrt{2\pi}$ such that for any $t > 0$, we have $s_1(\mathbf{X}) \leq c \cdot s \cdot (\sqrt{m} + \sqrt{n} + t)$ except with probability at most $\frac{2}{e^{\pi t^2}}$.*

A.2 Discrete Gaussian Distribution

For a ring R of degree N , we can define the discrete Gaussian distribution over it in the following way.

Definition A.2 (Definition 4.2 in [37]) For any positive integer ℓ , the discrete Gaussian distribution over R^ℓ centered around $\mathbf{v} \in R^\ell$ with standard deviation $\sigma > 0$ is given by $D_{\mathbf{v},\sigma}^{\ell,N}(\mathbf{z}) = \frac{e^{-\|\mathbf{z}-\mathbf{v}\|^2/2\sigma^2}}{\sum_{\mathbf{z}' \in R^\ell} e^{-\|\mathbf{z}'\|^2/2\sigma^2}}$. When $\mathbf{v} = 0$, we just write $D_\sigma^{\ell,N}$ for simplicity. Particularly, we write $D_{\mathbb{Z},\sigma}$ to denote the discrete Gaussian distribution over $\mathbb{Z}$ with standard deviation σ .

We also need to use the following facts about the discrete Gaussian distribution.

Lemma A.3 (Lemma 4.4 in [37]) For any positive integer ℓ and any real $\sigma > 0$, and a sample sampled from $D_\sigma^{\ell,N}$ defined as above. Then

1. for $x \leftarrow D_\sigma$, it holds $\Pr[|x| > t \cdot \sigma] \leq 2e^{-\frac{t^2}{2}}$;
2. for $\mathbf{x} \leftarrow D_\sigma^{\ell,N}$, it holds $\Pr[\|\mathbf{x}\| > t \cdot \sigma \sqrt{\ell N}] \leq \left(te^{-\frac{t^2}{2}}\right)^{\ell N}$,

where t is any constant value.

Lemma A.4 (Sum of Discrete Gaussian Samples) Let $\mathbf{x}_i$ for $i \in [n]$ be vectors sampled independently from D_σ^m . Suppose $\sigma \cdot \sqrt{2\pi} \geq \sqrt{2} \cdot \omega(\log m)$, then the distribution of $\sum_i \mathbf{x}_i$ is statistically close to $D_{\sigma\sqrt{n}}^m$.

A.3 Lattices Problems and Underlying Assumptions

Definition A.5 (MSIS) The $\text{MSIS}_{q,\ell,m,\beta}$ problem (over an implicit ring R) is defined as follows. Given an uniformly random matrix $\mathbf{A} \in R_q^{\ell \times m}$, output vector $\mathbf{z} \in R^m$ such that $\mathbf{A}\mathbf{z} = 0$ and $0 < \|\mathbf{z}\| \leq \beta$.

Definition A.6 (MLWE) For an error distribution χ over R , the decision $\text{MLWE}_{q,\ell,m,\chi}$ problem (over an implicit ring R) is defined as follows. For $\mathbf{s} \xleftarrow{\$} \chi^\ell$, use $A_{q,\mathbf{s}}$ to denote the distribution of $(\mathbf{a}, \langle \mathbf{a}, \mathbf{s} \rangle + e) \in R_q^\ell \times R_q$, where $\mathbf{a} \xleftarrow{\$} R_q^\ell$ and $e \xleftarrow{\$} \chi$. The goal is to distinguish m samples from either $A_{q,\mathbf{s}}$ or $\mathcal{U}(R_q^\ell, R_q)$, i.e., distinguish $(\mathbf{A}, \mathbf{A} \cdot \mathbf{s} + \mathbf{e})$ from $(\mathbf{A}, \mathbf{u})$, where $\mathbf{A} \xleftarrow{\$} R_q^{m \times \ell}$, $\mathbf{u} \xleftarrow{\$} R_q^m$, $\mathbf{s} \leftarrow \chi^\ell$, and $\mathbf{e} \leftarrow \chi^m$.

Moreover, the $\text{MLWE}_{q,\ell,m,\chi}$ problem defined above are the so-called ‘‘Hermite Normal Form’’ version, as its secret key and error are chosen from the identical ‘‘small’’ distribution χ . And such an ‘‘Hermite Normal Form’’ can be easily reduced to the standard MLWE. For standard MLWE and the above defined MSIS, it is known to be at least as hard as certain standard lattice problems over ideal lattice in the worst case. It should be pointed out that the ring learning with errors problem (RLWE) is the special case of MLWE for $\ell = 1$. Particularly, we denote the corresponding problem as $\text{RLWE}_{q,1,m,\chi}$. More generally, for a small

set S_β , we use $\text{RLWE}_{q,1,m,S_\beta}$ to denote that both secret key and error are sampled uniformly at random from S_β .

Notice that even for $\beta = 1$, we can still ensure the hardness of $\text{RLWE}_{q,1,m,1}$ through setting the sufficiently large ring dimension N . And the parameters in this case are still practical enough, just as pointed out by Baum et al. in [4].

Definition A.7 (DSPR [36]) *For an error distribution χ over R , the decisional small polynomial ratio (DSPR) assumption $\text{DSPR}_{q,R,\chi}$ says that the following two distributions are indistinguishable:*

- a polynomial $h = g \cdot f^{-1} \in R_q$, where $g, f \leftarrow \chi$.
- a polynomial $u \xleftarrow{\$} R_q$.

A.4 Rejection Sampling and Dilithium

In this paper, we use the well-known Dilithium signature scheme the basis for our distributed signature protocols. Thus, for completeness, we present the non-optimized version of Dilithium signature scheme in Algorithms 2 to 4.

Algorithm 2: Key generation

Input: $\text{pp} = (R_q, k, \ell, \eta, B, s, M)$

Output: (sk, pk)

1. $\mathbf{A} \xleftarrow{\$} R^{k \times \ell}$
 2. $\overline{\mathbf{A}} := [\mathbf{A} | \mathbf{I}] \in R^{k \times (\ell + k)}$
 3. $(\mathbf{s}_1, \mathbf{s}_2) \xleftarrow{\$} S_\eta^\ell \times S_\eta^k; \mathbf{s} := \begin{pmatrix} \mathbf{s}_1 \\ \mathbf{s}_2 \end{pmatrix}$.
 4. $\mathbf{t} := \overline{\mathbf{A}}\mathbf{s}$
 5. $\text{sk} := \mathbf{s}$
 6. $\text{pk} := (\overline{\mathbf{A}}, \mathbf{t})$
- return** (sk, pk)
-

Besides, we recall the rejection sampling algorithm as in Lemma A.8, which is important for the security of the FSwA-style signature such as Dilithium-G.

Lemma A.8 (Rejection Sampling [37]) *Let V be a subset of $\mathbb{R}^m$ in which all elements have norms less than T , and $\rho : V \rightarrow [0, 1]$ be a probability distribution. Let $\sigma = \alpha T$ for $\alpha = O(\sqrt{\lambda})$ and*

$$M = \exp \left(\sqrt{\frac{2(\lambda + 1)}{\log e}} \cdot \frac{1}{\alpha} + \frac{1}{2\alpha^2} \right) = O(1).$$

Now, sample $\mathbf{v} \xleftarrow{\$} \rho$ and $\mathbf{y} \xleftarrow{\$} D_\sigma^m$, set $\mathbf{z} = \mathbf{y} + \mathbf{v}$, and run $b \leftarrow \text{Rej}(\mathbf{z}, \mathbf{v}, \sigma)$ defined in Table 3. Then, the probability that $b = 1$ is at least $\frac{1-2^{-\lambda}}{M}$. And conditioned on $b = 1$, the distribution of $(\mathbf{v}, \mathbf{z})$ is within statistical distance of $\varepsilon_{\text{Rej}} = \frac{2^{-\lambda}}{M}$ of the product distribution $\rho \times D_\sigma^m$.

Algorithm 3: Signature generation

Input: $\text{sk}, \text{pk}, \mu, \text{pp} = (R_q, k, \ell, \eta, B, s, M)$

Output: valid signature pair (z, c)

1. $(y_1, y_2) \xleftarrow{\$} D_s^\ell \times D_s^k; \mathbf{y} := \begin{pmatrix} y_1 \\ y_2 \end{pmatrix}$
 2. $\mathbf{w} = \overline{\mathbf{A}}\mathbf{y}$
 3. $c \leftarrow H_0(\mathbf{w}, \mu, \text{pk})$
 4. $\mathbf{z} := c\mathbf{s} + \mathbf{y}$
 5. With prob. $\min(1, D_s^{\ell+k}(\mathbf{z}) / (M \cdot D_{cs,s}^{\ell+k}(\mathbf{z})))$:
 6. return $(\mathbf{z}, c)$
 7. Restart otherwise
-

Algorithm 4: Signature verification

Input: $\text{pk}, (z, c), \mu, \text{pp} = (R_q, k, \ell, \eta, B, s, M)$

1. If $\|\mathbf{z}\| \leq B$ and $c = H_0(\overline{\mathbf{A}}\mathbf{z} - c\mathbf{t}, \mu, \text{pk})$:
 2. return 1
 3. Otherwise:
 4. return 0
-

Theorem A.9 (Gadget Trapdoor [39]) *Let n, m, q be integers, and let $m' = n(\lceil \log q \rceil + 1)$. Then there are efficient probabilistic polynomial-time algorithms (TrapGen, MLWEInvert) as follows.*

- $\text{TrapGen}(1^n, q, m) \rightarrow (\mathbf{A}, \mathbf{T}_\mathbf{A})$: On input the lattice dimension n , the modulus q , and the number of samples m , the trapdoor-generation algorithm outputs a matrix $\mathbf{A} \in R_q^{n \times m}$ together with a trapdoor $\mathbf{T}_\mathbf{A} \in R^{m \times m}$.
- $\text{MLWEInvert}(\mathbf{T}_\mathbf{A}, \mathbf{A}, \mathbf{t}) \rightarrow \mathbf{e}$: On input a matrix $\mathbf{A} \in R_q^{n \times m}$, a trapdoor $\mathbf{T}_\mathbf{A} \in R^{m \times m}$, a MLWE instance vector $\mathbf{t} := \mathbf{A}^\top \cdot \mathbf{s} + \mathbf{e} \in R_q^m$ with $\mathbf{s} \leftarrow \chi^n$, $\mathbf{e} \leftarrow \chi^m$ and χ is a small distribution over R , the invert algorithm MLWEInvert outputs the correct error vector $\mathbf{e} \in R_q^m$ with overwhelming probability.

A.5 Supplementary for Quantum-Secure Pseudorandom Function in Section 3.2

Lemma A.10 (Restatement of Lemma 3.8) *If one PRF satisfies the standard quantum security as in Definition 3.6, then such PRF also satisfies the strong quantum security as in Definition 3.7.*

Proof. In order to prove such lemma in a more clear way, we first notice the following facts: Definitions 3.5, 3.6, and 3.7 can be depicted equivalently as the corresponding interactive experiments between the adversary $\mathcal{A}$ and the challenger $\mathcal{C}$. Particularly, the challenger first chooses a random bit $b \in \{0, 1\}$ to indicate running PRF or truly random function $\mathcal{O}$. Then, $\mathcal{A}$ makes queries for any polynomial times, and ends up with a decision $b' \in \{0, 1\}$. If $b = b'$, we say $\mathcal{A}$

$\text{Rej}(\mathbf{z}, \mathbf{v}, \sigma)$
01 $u \xleftarrow{\$} [0, 1)$
02 If $u > \frac{1}{M} \cdot \exp(\frac{-2\langle \mathbf{z}, \mathbf{v} \rangle + \ \mathbf{v}\ ^2}{2\sigma^2})$
03 return 0 (i.e. abort)
04 Else
05 return 1 (i.e. non-abort)

Table 3. Standard rejection sampling algorithm in [37].

wins the experiment, and the experiment outputs 1; Otherwise, $\mathcal{A}$ fails, and the experiment outputs 0. Taking Definition 3.7 as example, we denote its interactive experiment as $\mathbf{Exp}_{\text{QPRF}}^{\text{S-IND}}(\mathcal{A})$, and a secure QPRF implies that for any efficient $\mathcal{A}$, the probability $\mathbf{Adv}_{\text{QPRF}}^{\text{S-IND}}(\mathcal{A}) := \Pr[\mathbf{Exp}_{\text{QPRF}}^{\text{S-IND}}(\mathcal{A}) \rightarrow 1] \leq \frac{1}{2} + \text{negl}(\lambda)$.

Suppose $\mathbf{Exp}_{\text{QPRF}}^{\text{Ind}}(\mathcal{A})$ and $\mathbf{Exp}_{\text{QPRF}}^{\text{S-IND}}(\mathcal{A})$ are the corresponding interactive experiments for Definitions 3.6 and 3.7, respectively. It suffices to show that if there is an efficient adversary $\mathcal{A}$ such that $\mathbf{Exp}_{\text{QPRF}}^{\text{S-IND}}(\mathcal{A}) \rightarrow 1$, then there is another efficient adversary $\hat{\mathcal{A}}$ such that $\mathbf{Exp}_{\text{QPRF}}^{\text{Ind}}(\hat{\mathcal{A}}) \rightarrow 1$. In this case, it holds

$$\Pr[\mathbf{Exp}_{\text{QPRF}}^{\text{S-IND}}(\mathcal{A}) \rightarrow 1] \leq \Pr[\mathbf{Exp}_{\text{QPRF}}^{\text{Ind}}(\hat{\mathcal{A}}) \rightarrow 1].$$

And thus, the standard quantum security implies the strong quantum security, for any polynomial n .

Finally, suppose $\mathcal{A}$ is the adversary making q times quantum queries and taking n additional inputs $(x_i^*, O(x_i^*))_{i \in [n]}$ or $(x_i^*, \text{QPRF}_k(x_i^*))_{i \in [n]}$, where each x_i^* is randomly chosen from the domain $\mathcal{X}$, and q, n are polynomial in λ . Let $\hat{\mathcal{A}}$ be the adversary directly making $(q + n)$ times quantum queries. During all these additional n times superposition queries, $\hat{\mathcal{A}}$ can adaptively make and query the particular superpositions $\sum_{x_j \in \mathcal{X}} \alpha_i \cdot |x_j\rangle$ with $\sum_{i=1}^n \alpha_i^2 = 1$, such that the function values of $(O(x_i^*))_{i \in [n]}$ or $(\text{QPRF}_k(x_i^*))_{i \in [n]}$ can be measured from the returned superpositions $\sum_{x_j \in \mathcal{X}} \beta_i \cdot |O(x_j)\rangle$ or $\sum_{x_j \in \mathcal{X}} \beta_i \cdot |\text{QPRF}_k(x_j)\rangle$ with overwhelming probability, where $\sum_{i=1}^n \beta_i^2 = 1$. For example, given a randomly chosen $x_i^* \in \mathcal{X}$, $\hat{\mathcal{A}}$ can directly generate the pure state of x_i^* or a superposition with most of wight over x_i^* , rather than an uniform position, such that the value of x_i^* can be successfully measured at least with overwhelming probability. $\square$

A.6 Supplementary for Trapdoor Homomorphic Commitment Scheme in Section 3.3

In this section, we first present the properties of trapdoor homomorphic commitment scheme, and then present its concrete instantiation for completeness.

Correctness. Eqv-TCOM (resp. COM) is correct if for any $\text{msg} \in S_{\text{msg}}$

$$\Pr \left[\begin{array}{l} \text{cpp} \leftarrow \text{CSetup}(1^\lambda); \text{ck} \leftarrow \text{CGen}(\text{cpp}) \\ \text{Open}_{\text{ck}}(\text{com}, \text{Rand}, \text{msg}) \rightarrow 1 : \text{Rand} \xleftarrow{\$} D(S_r); \\ \text{com} \leftarrow \text{Commit}_{\text{ck}}(\text{msg}; \text{Rand}) \end{array} \right] = 1.$$

Hiding. Eqv-TCOM (resp. COM) is unconditionally (resp. computationally) hiding if the following probability is negligible in λ for any probabilistic adversary (resp. probabilistic polynomial-time adversary) $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$.

$$\epsilon_{\text{hide}} := \left| \Pr \left[\begin{array}{l} \text{cpp} \leftarrow \text{CSetup}(1^\lambda); \text{ck} \leftarrow \text{CGen}(\text{cpp}) \\ (\text{msg}_0, \text{msg}_1) \leftarrow \mathcal{A}_1(\text{ck}, \text{cpp}) \\ b \xleftarrow{\$} \{0, 1\}; \text{com} \leftarrow \text{Commit}_{\text{ck}}(\text{msg}_b) \\ b' \leftarrow \mathcal{A}_2(\text{com}) \end{array} \right] - \frac{1}{2} \right|$$

Binding. Eqv-TCOM (resp. COM) is unconditionally (resp. computationally) binding if the following probability is negligible in λ for any probabilistic adversary (resp. probabilistic polynomial-time adversary) $\mathcal{A}$.

$$\epsilon_{\text{bind}} := \Pr \left[\begin{array}{l} \text{msg} \neq \text{msg}' \\ \wedge \text{Open}_{\text{ck}}(\text{com}, \text{Rand}, \text{msg}) \rightarrow 1 : \text{cpp} \leftarrow \text{CSetup}(1^\lambda) \\ \wedge \text{Open}_{\text{ck}}(\text{com}, \text{Rand}', \text{msg}') \rightarrow 1 \quad (\text{com}, \text{msg}, \text{Rand}, \text{msg}', \text{Rand}') \leftarrow \mathcal{A}(\text{ck}) \end{array} \right]$$

In particular, unconditionally binding implies that the following probability is also negligible in λ , since otherwise unbounded adversaries can simply check all possible values in S_{com} , S_{msg} and S_r to find a tuple that breaks binding.

$$\epsilon_{\text{ubind}} := \Pr \left[\begin{array}{l} \exists (\text{com}, \text{Rand}, \text{msg}, \text{Rand}', \text{msg}') : \\ \text{msg} \neq \text{msg}' : \text{cpp} \leftarrow \text{CSetup}(1^\lambda) \\ \text{Open}_{\text{ck}}(\text{com}, \text{Rand}, \text{msg}) \rightarrow 1 \quad \text{ck} \leftarrow \text{CGen}(\text{cpp}) \\ \wedge \text{Open}_{\text{ck}}(\text{com}, \text{Rand}', \text{msg}') \rightarrow 1 \end{array} \right]$$

Secure Trapdoor. Eqv-TCOM has the secure trapdoors if Eqv-TCOM has a secure trapdoor. Particularly, Eqv-TCOM has a secure trapdoor if for any $\text{msg} \in S_{\text{msg}}$, the statistical distance ϵ_{td} between $(\text{ck}, \text{msg}, \text{com}, \text{Rand})$ and $(\text{tck}, \text{msg}, \text{com}^*, \text{Rand}^*)$ is negligible in λ , where $\text{cpp}_{\text{Eqv}} \leftarrow \text{CSetup}(1^\lambda); \text{ck} \leftarrow \text{CGen}(\text{cpp}_{\text{Eqv}}); \text{Rand} \xleftarrow{\$} D(S_r); \text{com} \leftarrow \text{Commit}_{\text{ck}}(\text{msg}; \text{Rand})$ and $(\text{tck}, \text{td}) \leftarrow \text{TCGen}(\text{cpp}_{\text{Eqv}}); \text{com}^* \leftarrow \text{Eqv-TCommit}_{\text{tck}}(\text{td}); \text{Rand}^* \leftarrow \text{Eqv-tck}(\text{td}, \text{com}^*, \text{msg}), \text{com} \leftarrow \text{Commit}_{\text{ck}}(\text{msg}; \text{Rand})$.

Definition A.11 (Uniform Key) A commitment key is said to be uniform if the output of $\text{CGen}(\text{cpp})$ follows the uniform distribution over the key space S_{ck} .

Definition A.12 (Additive Homomorphism) A commitment scheme is said to be additively homomorphic if for any $\text{msg}, \text{msg}' \in S_{\text{msg}}$

$$\Pr \left[\begin{array}{l} \text{Open}_{\text{ck}}(\text{com} + \text{com}', \text{Rand} + \text{Rand}', : \\ \text{msg} + \text{msg}') \rightarrow 1 \end{array} \begin{array}{l} \text{cpp} \leftarrow \text{CSetup}(1^\lambda) \\ \text{ck} \leftarrow \text{CGen}(\text{cpp}) \\ \text{Rand} \xleftarrow{\$} D(S_r); \text{Rand}' \xleftarrow{\$} D(S_r) \\ \text{com} \leftarrow \text{Commit}_{\text{ck}}(\text{msg}; \text{Rand}) \\ \text{com}' \leftarrow \text{Commit}_{\text{ck}}(\text{msg}'; \text{Rand}') \end{array} \right] = 1$$

Moreover, in the following detailed security proof for our constructions, we additionally need the commitment of **Eqv-TCOM** satisfies statistic/computational uniform property, which is much more stronger than the previously defined hiding property. In other words, we require that $\text{com}^* \leftarrow \text{Eqv-TCommit}_{\text{tck}}(\text{td})$ has sufficient min-entropy, say, follows the uniform distribution.

Definition A.13 (Uniform Commitment) *For a Eqv-TCOM scheme, it is said to be uniform commitment if $\text{com}^* \leftarrow \text{Eqv-TCommit}_{\text{tck}}(\text{td})$ follows the uniform distribution over the commitment space S_{com} .*

For completeness, we present the known instantiation of Eqv-COM scheme in [15].

Construction A.14 (Eqv-COM Scheme) *The scheme consists of the following algorithms.*

- $\text{CSetup}(1^\lambda)$ takes a security parameter as input, and outputs $\text{cpp} = (N, q, \bar{s}, s, B, \ell, w)$.
- $\text{CGen}(\text{cpp})$ takes a commitment parameter as input, and samples $\hat{a}_{1,1} \xleftarrow{\$} R_q^\times$ (a uniform invertible element of R_q) and $\hat{a}_{1,j} \xleftarrow{\$} R_q$ for $j = 2, \dots, \ell + 2w$, $\hat{a}_{2,j} \xleftarrow{\$} R_q$ for $j = 3, \dots, \ell + 2w$. It then outputs:

$$\hat{\mathbf{A}} = \begin{bmatrix} \hat{a}_{1,1} & \hat{a}_{1,2} & \hat{a}_{1,3} & \dots & \hat{a}_{1,\ell+2w} \\ 0 & 1 & \hat{a}_{2,3} & \dots & \hat{a}_{2,\ell+2w} \end{bmatrix}$$

as ck .

- $\text{Commit}_{\text{ck}}(x; \mathbf{r})$ takes $x \in R_q$ and $\mathbf{r} \xleftarrow{\$} D_s^{\ell+2w}$ as input, and outputs

$$\mathbf{f} = \hat{\mathbf{A}} \cdot \mathbf{r} + \begin{bmatrix} 0 \\ x \end{bmatrix} \in R_q^2.$$

To ensure perfect correctness, retry unless $\|\mathbf{r}\| \leq B$.

- $\text{Open}_{\text{ck}}(\mathbf{f}, \mathbf{r}, x)$ takes commitments, randomness and message as input, and checks that

$$\mathbf{f} = \hat{\mathbf{A}} \cdot \mathbf{r} + \begin{bmatrix} 0 \\ x \end{bmatrix} \text{ and } \|\mathbf{r}\| \leq B.$$

- $\text{TCGen}(\text{cpp})$ takes a commitment parameter as input, and samples $\overline{\mathbf{A}}$ of the form:

$$\overline{\mathbf{A}} = \begin{bmatrix} \bar{a}_{1,1} & \bar{a}_{1,2} & \bar{a}_{1,3} & \dots & \bar{a}_{1,\ell} \\ 0 & 1 & \bar{a}_{2,3} & \dots & \bar{a}_{2,\ell} \end{bmatrix}$$

where all the $\bar{a}_{i,j}$ are uniform in R_q , except $\bar{a}_{1,1}$ which is uniform in $R_q^\times$. It also samples $\mathbf{R} \xleftarrow{\$} D_{\bar{s}}^{\ell \times 2w}$ with discrete Gaussian entries. It then outputs $\mathbf{A}$ as the trapdoor td and $\hat{\mathbf{A}} = [\bar{\mathbf{A}}|\mathbf{G} - \bar{\mathbf{A}}\mathbf{R}]$ as the commitment key tck , where $\mathbf{G}$ is given by:

$$\mathbf{G} = \begin{bmatrix} 1 & 2 & \dots & 2^{w-1} & 0 & 0 & \dots & 0 \\ 0 & 0 & \dots & 0 & 1 & 2 & \dots & 2^{w-1} \end{bmatrix} \in R^{2 \times 2w}.$$

- $\text{Eqv-TCommit}_{\text{tck}}(\text{td})$ simply returns a uniformly random commitment $\mathbf{f} \xleftarrow{\$} R_q^{2 \times 1}$. There is no need to keep a state.
- $\text{Eqv}_{\text{tck}}(\mathbf{R}, \mathbf{f}, x)$ uses the trapdoor discrete Gaussian sampling algorithm of Micciancio-Peikert [39, Algorithm 3] to sample $\mathbf{r} \xleftarrow{\$} D_{\Lambda_{\mathbf{u}}^\perp(\hat{\mathbf{A}}, s)}$ according to the discrete Gaussian of parameter s supported on the lattice coset: $\Lambda_{\mathbf{u}}^\perp(\hat{\mathbf{A}}) = \{\mathbf{z} \in R^{\ell+2w} : \hat{\mathbf{A}} \cdot \mathbf{z} \equiv \mathbf{u} \pmod{q}\}$ where $\mathbf{u} = \mathbf{f} - [0 \ x]$

Theorem A.15 (Theorem 3 of [15]) *The trapdoor commitment scheme of above, with the following choice of parameters:*

$$\begin{array}{lll} \bar{s} = \Theta(N) & s = \Theta(N^{3/2} \log^2 N) & B = \Theta(N^2 \log^3 N) \\ \ell = w = \lceil \log_2 q \rceil & q = N^{2+\varepsilon} & (\varepsilon > 0, q \text{ prime}) \end{array}$$

is a secure trapdoor commitment scheme assuming that the $\text{MSIS}_{q,1,\ell+2w-1,2B}$ problem is hard.

A.7 Supplementary for n -out-of- n Signature and Multi-Signature in Section 3.4

Algorithm 5: $\text{Exp}_{\text{QDS}}^{\text{UF-CMA}}(\mathcal{A})$

```

1 : Mset  $\leftarrow \emptyset$  //set of signed messages
2 :  $\mathcal{S} \leftarrow \emptyset$  //open signing sessions
3 :  $\mathcal{S}' \leftarrow \emptyset$  //close signing sessions
4 :  $\mathcal{K} \leftarrow \emptyset$  //open key generation sessions
5 :  $\text{pp} \leftarrow \text{Setup}(1^\lambda, n)$ 
6 :  $\mathcal{C} \leftarrow \mathcal{A}^{|\mathcal{H}|}(\text{pp})$ 
7 : if  $(\mathcal{C} = [n])$  then
8 :   return  $\perp$ 
9 :  $(\mathcal{H} = [n] \setminus \mathcal{C})$ 
10 :  $(\text{sk}_1, \dots, \text{sk}_n, \text{pk}) \leftarrow \text{Gen}(\text{pp})$ 
11 :  $(\mu^*, \widehat{\text{Sig}}^*) \leftarrow \mathcal{A}^{\mathcal{O}_{\text{Gen}}, \mathcal{O}_{\text{Sign}_1}, \mathcal{O}_{\text{Sign}_2}, |\mathcal{H}|}(\text{pp}, \text{pk})$ 
12 :  $b \leftarrow \text{Ver}(\mu^*, \widehat{\text{Sig}}^*, \text{pk})$ 
13 : return  $(b = 1) \wedge (\mu^* \notin \text{Mset})$ 

```

Fig. 13. QDS-UF-CMA experiment. The corresponding oracles are described in Figure 14.

```

Oracle  $\mathcal{O}_{\text{Gen}}(\text{sid}, i)$ :
1 : if  $(\text{sid} \in \mathcal{K}) \vee (i \notin \mathcal{H})$  then
2 :   return  $\perp$ 
3 :  $(\text{sk}_i, \text{pk}) \leftarrow \text{Gen}(\text{pp})$  //running the distributed key generation protocol with other participants
4 :  $\mathcal{K} \leftarrow \mathcal{K} \cup \{\text{sid}\}$ 
5 : return  $(\text{sk}_i, \text{pk})$ 

Oracle  $\mathcal{O}_{\text{Sign}_1}(\text{sid}, i, \mu)$ :
1 : if  $(\text{sid} \notin \mathcal{K}) \vee ((\text{sid}, i, \cdot, \cdot) \in \mathcal{S}) \vee (i \notin \mathcal{H})$  then
2 :   return  $\perp$ 
3 :  $\{\text{msg}_i, \text{st}_i\} \leftarrow \text{Sign}_1(\text{pp}, \text{sid}, \text{sk}_i, \text{pk}, \mu)$ 
4 :  $\mathcal{S} \leftarrow \mathcal{S} \cup \{(\text{sid}, i, \text{msg}_i, \text{st}_i)\}$ 
5 : return  $\text{msg}_i$ 

Oracle  $\mathcal{O}_{\text{Sign}_2}(\text{sid}, i, \{\text{msg}_j\}_{j \in [n] \setminus i}, \mu)$ :
1 : if  $((\text{sid}, i, \cdot, \cdot) \notin \mathcal{S}) \vee ((\text{sid}, i) \in \mathcal{S}') \vee (\exists j \in \mathcal{H} \setminus \{i\} : (\text{sid}, j, \cdot, \cdot) \notin \mathcal{S})$  then
2 :   return  $\perp$ 
3 :  $\text{Sig}_i \leftarrow \text{Sign}_2(\text{sid}, \text{st}_i, \{\text{msg}_j\}_{j \in \mathcal{H} \setminus \{i\}})$ 
4 :  $\mathcal{S}' \leftarrow \mathcal{S}' \cup \{(\text{sid}, i)\}$ 
5 :  $\text{Mset} \leftarrow \text{Mset} \cup \{\mu\}$ 
6 : return  $\text{Sig}_i$ 

Oracle  $|H\rangle$ :
1 : return  $|H(x)\rangle$ 

```

Fig. 14. Honest party oracle for the distributed signing protocol

Algorithm 6: $\text{Exp}_{\text{QMS}}^{\text{UF-CMA}}(\mathcal{A})$

```

1 :  $\text{Mset} \leftarrow \emptyset$  //set of signed messages
2 :  $\mathcal{S} \leftarrow \emptyset$  //open signing sessions
3 :  $\mathcal{S}' \leftarrow \emptyset$  //close signing sessions
4 :  $\text{pp} \leftarrow \text{Setup}(1^\lambda, n)$ 
5 :  $(\text{sk}_1, \text{pk}_1) \leftarrow \text{Gen}(\text{pp})$ 
6 :  $(\mathbb{L}^*, \mu^*, \widehat{\text{Sig}}^*) \leftarrow \mathcal{A}^{\mathcal{O}_{\text{Sign}_1}, \mathcal{O}_{\text{Sign}_2}, |H\rangle}(\text{pp})$ 
7 :  $b \leftarrow \text{Ver}(\mu^*, \widehat{\text{Sig}}^*, \text{KAgg}(\mathbb{L}^*))$ 
8 : return  $(\text{pk}_1 \notin \mathbb{L}^*) \wedge (b = 1) \wedge (\mathbb{L}^*, \mu^*) \notin \text{Mset}$ 

```

Fig. 15. QMS-UF-CMA experiment. The corresponding oracles are described in Figure 16.

Definition A.16 (QDS-UF-CMA Security) A distributed signature protocol QDS is said to be QDS-UF-CMA, i.e., distributed signature unforgeability against chosen message attacks, secure, if for any quantum polynomial time adversary $\mathcal{A}$, its advantage

$$\text{Adv}_{\text{QDS}}^{\text{UF-CMA}}(\mathcal{A}) := \Pr \left[\text{Exp}_{\text{QDS}}^{\text{UF-CMA}}(\mathcal{A}) \rightarrow 1 \right]$$

is negligible in λ , where $\text{Exp}_{\text{QDS}}^{\text{UF-CMA}}(\mathcal{A})$ is described in Figure 13.

Definition A.17 (QMS-UF-CMA Security) A multisignature protocol QMS is said to be QMS-UF-CMA, i.e., multisignature unforgeability against chosen message attacks, secure, if for any quantum polynomial time adversary $\mathcal{A}$, its ad-


```

Oracle  $\mathcal{O}_{\text{Sign}_1}(\text{sid}, \mu, \mathbb{L})$ :
1 : if  $((\text{sid}, \cdot, \cdot) \in \mathcal{S}) \vee (\text{pk}_1 \notin \mathbb{L})$  then
2 :   return  $\perp$ 
3 :  $\{\text{msg}_1, \text{st}_{\text{sid}}\} \leftarrow \text{Sign}_1(\text{pp}, \text{sid}, \text{sk}_1, \mu, \mathbb{L})$ 
4 :  $\mathcal{S} \leftarrow \mathcal{S} \cup \{(\text{sid}, \text{msg}_1, \text{st}_{\text{sid}})\}$ 
5 : return  $\text{msg}_1$ 

Oracle  $\mathcal{O}_{\text{Sign}_2}(\text{sid}, \{\text{msg}_j\}_{j \in [n] \setminus \{1\}}, \mu)$ :
1 : if  $((\text{sid}, \cdot, \cdot) \notin \mathcal{S}) \vee (\text{sid} \in \mathcal{S}')$  then
2 :   return  $\perp$ 
3 :  $\text{Sig}_1 \leftarrow \text{Sign}_2(\text{sid}, \text{st}_1, \{\text{msg}_j\}_{j \in [n] \setminus \{1\}}, \mu)$ 
4 :  $\mathcal{S}' \leftarrow \mathcal{S}' \cup \{\text{sid}\}$ 
5 :  $\text{Mset} \leftarrow \text{Mset} \cup \{(\mathbb{L}, \mu)\}$ 
6 : return  $\text{Sig}_1$ 

Oracle  $|H\rangle$ :
1 : return  $|H(x)\rangle$ 

```

Fig. 16. Honest party oracle for the multi-signature protocol

vantage

$$\text{Adv}_{\text{QMS}}^{\text{UF-CMA}}(\mathcal{A}) := \Pr \left[\text{Exp}_{\text{QMS}}^{\text{UF-CMA}}(\mathcal{A}) \rightarrow 1 \right]$$

is negligible in λ , where $\text{Exp}_{\text{QMS}}^{\text{UF-CMA}}(\mathcal{A})$ is described in Figure 15.

Remark A.18 Our unforgeability for QDS is equivalent to the TS-UF-0 security defined in [5]. As we just focus on the n -out-of- n case, we implicitly assume $|\mathcal{C}| = (n - 1)$ in the security proof of Theorem 5.2.

B Supplementary for QPRF in Section 4

Due to the space limitation in the main body, we present many more supplementary materials for QPRF in Section 4.

B.1 Detailed Proof for theorems in Section 4.1

Theorem B.1 (Restatement of Theorem 4.3) Let $\chi = D_{\bar{R}, \bar{r}}$ be a small distribution over $\bar{R}$, where all coefficients of each polynomial are chosen independently from $D_{\mathbb{Z}, \bar{r}}$. Let $\bar{q} \geq \bar{p} \cdot \bar{\ell} \cdot (\bar{r} \cdot \sqrt{2(N + \bar{\ell})} \cdot \omega(\sqrt{\log(N + \bar{\ell})}))^{\bar{\ell}} \cdot \bar{N}^{\omega(1)}$. Let QPRF be as in Construction 4.1. If the $\text{RLWE}_{\bar{q}, 1, \bar{m}, \chi}$ holds, then Construction 4.1 is a secure QPRF.

Proof. Similar to the proof of Theorem 6.1 by Zhandry in [44], we first define a class of functions $G : \mathcal{K} \times [2]^{\bar{\ell}} \rightarrow \bar{R}_{\bar{q}}^{1 \times \bar{d}}$ as

$$G_{\mathbf{k}}(x) = (a_1, \dots, a_{\bar{m}}) \cdot \prod_{i=1}^{\bar{\ell}} s_i^{x_i} \mod \bar{q},$$

where $x := (x_1, \dots, x_{\bar{\ell}}) \in \{0, 1\}^{\bar{\ell}}$. Then, we define a related class of functions $\tilde{G}^{(\bar{\ell})}$ in the following recursive way.

- $\tilde{G}^{(0)}$ is a function from $[2]^0$ to $\bar{R}_{\bar{q}}^{1 \times \bar{m}}$ defined as follows: sample $\mathbf{a}^\top = (a_1, \dots, a_{\bar{m}}) \leftarrow \bar{R}_{\bar{q}}^{1 \times \bar{m}}$, and set $\tilde{G}^{(0)}(\epsilon) = \mathbf{a}^\top$.
- $\tilde{G}^{(i)}$ is a function from $[2]^i$ to $\bar{R}_{\bar{q}}^{1 \times \bar{m}}$ defined as follows: choose a random $\tilde{G}^{(i-1)}$, sample $s_i \leftarrow \chi$, and for each $x' := (x_1, \dots, x_{\bar{\ell}-1}) \in [2]^{i-1}$, sample $\mathbf{e}_{x'} \leftarrow \chi^{1 \times \bar{m}}$. Then,

$$\tilde{G}^{(i)}(x = (x'|x_i)) = \tilde{G}^{(i-1)}(x') \cdot s_i^{x_i} + x_i \cdot \mathbf{e}_{x'} \pmod{\bar{q}}.$$

Furthermore, we define two truly random function $U : [2]^{\bar{\ell}} \rightarrow \bar{R}_{\bar{p}}^{1 \times \bar{m}}$ and $U' : [2]^{\bar{\ell}} \rightarrow \bar{R}_{\bar{q}}^{1 \times \bar{m}}$.

With above definitions, the high-level proof route is that for any adversary choosing query $x \in [2]^{\bar{\ell}}$, it holds

$$\text{QPRF}_k(x) := \lfloor G_k(x) \rfloor_{\bar{p}} \stackrel{(i)}{\approx}_c \lfloor \tilde{G}^{(\bar{\ell})}(x) \rfloor_{\bar{p}} \stackrel{(ii)}{\approx}_c \lfloor U'(x) \rfloor_{\bar{p}} \stackrel{(iii)}{\approx}_c U(x), \quad (3)$$

with overwhelming probability.

According to the above definition on $G^{(\bar{\ell})}(x)$, we know that

$$\begin{aligned} \tilde{G}(x_1 \cdots x_{\bar{\ell}}) &= (\cdots ((\mathbf{a}^\top \cdot s_1^{x_1} + x_1 \cdot \mathbf{e}_\epsilon) \cdot s_2^{x_2} + x_2 \cdot \mathbf{e}_1) \cdots) \cdot s_{\bar{\ell}}^{x_{\bar{\ell}}} + x_{\bar{\ell}} \cdot \mathbf{e}_{x_1 \cdots x_{\bar{\ell}-1}} \\ &= \mathbf{a}^\top \cdot \prod_{i=1}^{\bar{\ell}} s_i^{x_i} + x_1 \cdot \mathbf{e}_\epsilon \cdot \prod_{i=2}^{\bar{\ell}} s_i^{x_i} + x_2 \cdot \mathbf{e}_{x_1} \cdot \prod_{i=3}^{\bar{\ell}} s_i^{x_i} + \cdots x_{\bar{\ell}} \cdot \mathbf{e}_{x_1 \cdots x_{\bar{\ell}-1}} \\ &= G_k(x) + x_1 \cdot \mathbf{e}_\epsilon \cdot \prod_{i=2}^{\bar{\ell}} s_i^{x_i} + x_2 \cdot \mathbf{e}_{x_1} \cdot \prod_{i=3}^{\bar{\ell}} s_i^{x_i} + \cdots x_{\bar{\ell}} \cdot \mathbf{e}_{x_1 \cdots x_{\bar{\ell}-1}}, \end{aligned}$$

where the above computations are conducted over $\bar{R}_{\bar{q}}$. Notice that according to Lemma 2.3 in [3], for $s_i \leftarrow \chi$, and each error vector $\mathbf{e}_{x_1 \cdots x_{i-1}} \leftarrow D_{\bar{R}, \bar{r}}^{1 \times \bar{m}}$, it holds the difference between the coefficient of each entry of $G_k(x)$ and the corresponding coefficient of $\tilde{G}(x)$ is bounded by $\bar{B} = \bar{\ell} \cdot (\bar{r} \cdot \sqrt{2\bar{N}} \cdot \omega(\sqrt{\log \bar{N}}))^{\bar{\ell}} / \sqrt{\bar{N}}$.

Then, in order to ensure the indistinguishability even with all QPRF queries in $[2]^{\bar{\ell}}$ by the quantum adversary, just as Zhandry's argument in [44], we reset $\bar{B} = \bar{\ell} \cdot (\bar{r} \cdot \sqrt{2(\bar{N} + \bar{\ell})} \cdot \omega(\sqrt{\log(\bar{N} + \bar{\ell})}))^{\bar{\ell}} / \sqrt{\bar{N}}$. With this value $\bar{B}$, for each $y \in \mathbb{Z}_{\bar{q}}$, we can define $\text{BAD}(y)$ to be the event that $\lfloor y + [-\bar{B}, \bar{B}] \rfloor_{\bar{p}} \neq \lfloor y \rfloor_{\bar{p}}$. Suppose, we can set $\bar{q} \geq \bar{p} \cdot \bar{\ell} \cdot (\bar{r} \cdot \sqrt{2(\bar{N} + \bar{\ell})} \cdot \omega(\sqrt{\log(\bar{N} + \bar{\ell})}))^{\bar{\ell}} \cdot \bar{N}^{\omega(1)}$ such that $\frac{(2\bar{B}+1)\bar{p}}{\bar{q}} \cdot \bar{m} \cdot \bar{N} = \text{negl}(\lambda)$. Then, for all coefficients in the output of $\tilde{G}^{(\bar{\ell})}(x)$, the BAD happens with negligible probability. And thus, the step (i) in (3) will hold.

And the computational indistinguishability of $\tilde{G}^{(\bar{\ell})}(x)$ follows from the oracle-LWE indistinguishability defined by Zhandry in [44], which further follows from the underlying RLWE $_{\bar{q}, 1, \bar{m}, \chi}$ assumption, defined in Definition A.7. This also implies that the step (ii) in (3) holds.

Finally, for the step (iii) in (3), it holds due to the fact that the event BAD happens with negligible probability. Overall, (3) is set up, and thus the statement of this theorem holds. $\square$

Lemma B.2 (Restatement of Lemma 4.4) For any $\bar{N} \geq 1, \bar{q} \geq 2, \bar{d} = \lceil \log \bar{q} \rceil, \bar{m} = \bar{d} + 2, \bar{p} \geq 3 \cdot \sqrt{\bar{m}\bar{N}} \cdot (\sqrt{2\bar{N}} + \sqrt{\bar{d}\bar{N}})$, there exist the following two efficient algorithms (TrapGen, RLWRInvert).

TrapGen($1^{\bar{N}}, \bar{q}, \bar{m}, \bar{d}$): A PPT algorithm which on input positive integers $\bar{N}, \bar{q}, \bar{m}, \bar{d}$, first samples a vector $(a_1, a_2) \in \bar{R}_{\bar{q}}^2$ and trapdoor $\mathbf{T} \in S_1^{2 \times \bar{d}}$, where $\bar{R}_{\bar{q}} = \mathbb{Z}_{\bar{q}}[X]/(X^{\bar{N}} + 1)$. Furthermore, the algorithm computes $(a_3, \dots, a_{\bar{m}}) = (a_1, a_2)\mathbf{T} + \mathbf{g}^\top$, where $\mathbf{g}^\top = (1, 2, \dots, 2^{\bar{d}-1})$. In this case, $\mathbf{a}^\top = (a_1, \dots, a_{\bar{m}})^\top$ is computationally close to uniform over $\bar{R}_{\bar{q}}^{\bar{m}}$, according to the $\text{RLWE}_{\bar{q}, 1, 1, S_1}$ assumption.

Clearly, it holds $\mathbf{a}^\top \cdot \begin{bmatrix} -\mathbf{T} \\ \mathbf{I}_{\bar{d} \times \bar{d}} \end{bmatrix} = \mathbf{g}^\top$, where $\mathbf{I}_{\bar{d} \times \bar{d}} \in \bar{R}_{\bar{q}}^{\bar{d} \times \bar{d}}$ is an identity matrix.

RLWRInvert($\mathbf{T}, \mathbf{a}, \mathbf{b}$): An algorithm taking as input $(\mathbf{a}, \mathbf{T})$ output by **TrapGen**($1^{\bar{n}}, \bar{q}$), and some value $\mathbf{b} \in \bar{R}_{\bar{p}}^{\bar{m}}$ such that $\mathbf{b}^\top = \lfloor \mathbf{a}^\top \cdot s \rfloor_{\bar{p}}$ for some $s \in \bar{R}_{\bar{q}}$, outputs s .

Proof. Given RLWR samples $(\mathbf{a}^\top, \mathbf{b}^\top = \lfloor \mathbf{a}^\top \cdot s \rfloor_{\bar{p}})$, we first transform it into RLWE samples $(\mathbf{a}^\top, \mathbf{a}^\top \cdot s + \mathbf{e}^\top)$, and then invert such RLWE problem through using the trapdoor for $\mathbf{a}$. Thus, we will get the secret s for the original RLWR samples.

Particularly, given $\mathbf{b} \in \bar{R}_{\bar{p}}^{\bar{m}}$, we compute $\lfloor \frac{\bar{q}}{\bar{p}} \cdot \mathbf{b} \rfloor \in \bar{R}_{\bar{q}}^{\bar{m}}$. More precisely, it holds

$$\mathbf{c} = \lfloor \frac{\bar{q}}{\bar{p}} \cdot \mathbf{b} \rfloor = \lfloor \frac{\bar{q}}{\bar{p}} \cdot \lfloor \frac{\bar{p}}{\bar{q}} \cdot \mathbf{a} \cdot s \rfloor \rfloor = \lfloor \frac{\bar{q}}{\bar{p}} \cdot (\frac{\bar{p}}{\bar{q}} \cdot \mathbf{a} \cdot s + \mathbf{e}') \rfloor = \mathbf{a} \cdot s + \mathbf{e},$$

where $\mathbf{e}' \in (-\frac{1}{2}, \frac{1}{2}]^{\bar{N} \cdot \bar{m}}$ and $\mathbf{e} \in (-\frac{\bar{q}}{2\bar{p}}, \frac{\bar{q}}{2\bar{p}}]^{\bar{N} \cdot \bar{m}}$. Then, we compute

$$\hat{\mathbf{c}}^\top = \mathbf{c} \cdot \begin{bmatrix} -\mathbf{T} \\ \mathbf{I}_{\bar{d} \times \bar{d}} \end{bmatrix} = s \cdot \mathbf{g}^\top + \hat{\mathbf{e}}^\top = s \cdot \mathbf{g}^\top + \mathbf{e}^\top \cdot \begin{bmatrix} -\mathbf{T} \\ \mathbf{I}_{\bar{d} \times \bar{d}} \end{bmatrix}. \quad (4)$$

For simplicity, we denote $\mathbf{T}' = \begin{bmatrix} -\mathbf{T} \\ \mathbf{I}_{\bar{d} \times \bar{d}} \end{bmatrix}$. And it holds $s_1(\mathbf{T}') = \sqrt{s_1(\mathbf{T})^2 + 1}$.

Thus we have $\|\hat{\mathbf{e}}\| \leq s_1(\mathbf{T}') \cdot \frac{\bar{q}}{2\bar{p}} \cdot \sqrt{\bar{N}} \cdot \bar{m}$. According to the property of primitive vector $\mathbf{g}^\top$, we know that (4) will be successfully inverted if $\hat{\mathbf{e}} \in \mathcal{P}_{1/2}(\bar{q} \cdot \mathbf{B}^{-\top})$, where $\mathbf{B}$ is the basis for the lattice $\Lambda_{\bar{q}}^\perp(\mathbf{g}^\top)$, satisfying $\|\mathbf{B}\| \leq \sqrt{5}$. This equivalently implies that $\|\hat{\mathbf{e}}\| \leq \frac{\bar{q}}{2\sqrt{5}}$. Thus, it suffices to set $s_1(\mathbf{T}') \cdot \frac{\bar{q}}{2\bar{p}} \cdot \sqrt{\bar{N}} \cdot \bar{m} \leq \frac{\bar{q}}{2\sqrt{5}}$. Combining $s_1(\mathbf{T}) \leq (\sqrt{2\bar{N}} + \sqrt{\bar{m} \cdot \bar{N}})$ by Lemma A.1, it is sufficient to set $\bar{p} \geq 3 \cdot \sqrt{\bar{m}\bar{N}} \cdot (\sqrt{2\bar{N}} + \sqrt{\bar{d}\bar{N}})$. $\square$

Lemma B.3 (Restatement of Lemma 4.6) For the adversary $\mathcal{A}$ without the trapdoor $\mathbf{T}$ of the vector $\mathbf{a}$, if the $\text{RLWE}_{\bar{q}, 1, 1, S_1}$ assumption holds, then Constructions 4.1 and 4.5 are computational indistinguishability, even $\mathcal{A}$ queries the functions in a superposition for any polynomial times.

Proof. Notice that for any $i \in [\bar{d}]$, we know $a_{i+2} = (a_1, a_2) \cdot \begin{pmatrix} t_{1,i} \\ t_{2,i} \end{pmatrix} + 2^i \pmod{\bar{q}}$. Furthermore, due to the $\text{RLWE}_{\bar{q}, 1, 1, S_1}$ assumption, we know that for uniform and public ring elements a_1, a_2 , $(a_1, a_2) \cdot \begin{pmatrix} t_{1,i} \\ t_{2,i} \end{pmatrix}$ is computationally indistinguishable

from uniform over $\bar{R}_{\bar{q}}$. As a result, Constructions 4.1 and 4.5 are computational indistinguishability. $\square$

Theorem B.4 (Restatement of Theorem 4.7) *For some $\mathbf{a} \in R_{\bar{q}}^{\bar{m}}$ and integers $\bar{p}, \bar{q}, \bar{d}, \bar{N}, \bar{m}$ such that $\bar{q} \geq \bar{p} \cdot \bar{\ell} \cdot (\bar{r} \cdot \sqrt{2(\bar{N} + \bar{\ell})} \cdot \omega(\sqrt{\log(\bar{N} + \bar{\ell})}))^{\bar{\ell}} \cdot \bar{N}^{\omega(1)} \geq (\bar{r} \cdot \sqrt{2\bar{N}})^{\bar{\ell}}$, $\bar{d} = \lceil \log \bar{q} \rceil$, and $\bar{m} = \bar{d} + 2$ and $\bar{p} \geq 3 \cdot \sqrt{\bar{m}\bar{N}} \cdot (\sqrt{2\bar{N}} + \sqrt{\bar{d}\bar{N}})$, suppose the oracle $O_{\text{RLWRI}^{-1}}$ in Algorithm 1 correctly invert $\lfloor \mathbf{a}^\top \cdot s \rfloor_{\bar{p}}$ for any $s \in \bar{R}_{\bar{q}}$. Then, for any invertible ring element $s_i \in \bar{R}_{\bar{q}}$, Algorithm 1 correctly inverts $\text{Inj-QPRF}_{\mathbf{a}, \{s_i\}} = \left\lfloor \mathbf{a}^\top \cdot \prod_{i=1}^{\bar{\ell}} s_i^{x_i} \right\rfloor_{\bar{p}}$, assuming the $\text{DSPR}_{\bar{q}, \bar{R}, \chi}$ assumption.*

Proof. From the oracle $O_{\text{RLWRI}^{-1}}$, we can get the correct ring element $\hat{s} = \prod_{i=1}^{\bar{\ell}} s_i^{x_i} \bmod \bar{q}$ from the above Step 1, due to our parameter settings on $\bar{m}, \bar{N}, \bar{d}, \bar{\ell}, \bar{q}$ and $\bar{p}$.

Then, in order to show the correctness of the following Steps 3 and 4, Particularly, as each $s_i \leftarrow D_{\bar{R}, \bar{r}}$ is invertible over $\bar{R}_{\bar{q}}$ with overwhelming probability, if the matrix $s'_{i-1} = s_i \cdot s_{i+1}^{x_{i+1}} \cdot \dots \cdot s_{\bar{\ell}}^{x_{\bar{\ell}}}$, then it is clearly that the norm of $s'_i = s_i^{-1} \cdot s'_{i-1} = s_{i+1}^{x_{i+1}} \cdot \dots \cdot s_{\bar{\ell}}^{x_{\bar{\ell}}}$ will be smaller than $(\bar{r}\sqrt{2\bar{N}})^{\bar{\ell}-i}$ with overwhelming probability, according to Lemma A.3.

On the other hand, if the matrix s'_{i-1} does not consist of the i -th small ring element s_i , i.e., $s'_{i-1} = s_{i+1}^{x_{i+1}} \cdot \dots \cdot s_{\bar{\ell}}^{x_{\bar{\ell}}}$, then $s'_i = s_i^{-1} \cdot s'_{i-1} = s_i^{-1} \cdot s_{i+1}^{x_{i+1}} \cdot \dots \cdot s_{\bar{\ell}}^{x_{\bar{\ell}}}$. Without loss of generality, we assume $x_{i+1} = 1$. In this case, we know

$$s'_i = \frac{s_{i+1}}{s_i} \cdot \dots \cdot s_{\bar{\ell}}^{x_{\bar{\ell}}}.$$

According to the DSPR assumption, we know that $\frac{s_{i+1}}{s_i}$ is computationally indistinguishable from uniform over R_q . And thus, s'_i is computationally indistinguishable from uniform, which implies that $\|s'_i\| > (\bar{r}\sqrt{2\bar{N}})^{\bar{\ell}-i}$ with overwhelming probability, according to our parameter setting.

Summing up all above analyzes, we conclude that Algorithm 1 correctly inverts $\text{Inj-QPRF}_{\mathbf{a}, \{s_i\}} = \left\lfloor \mathbf{a}^\top \cdot \prod_{i=1}^{\bar{\ell}} s_i^{x_i} \right\rfloor_{\bar{p}}$. $\square$

B.2 Detailed Proof for Theorems in Section 4.2

Theorem B.5 (Restatement of Theorem 4.8) *Let $\text{QPRF} : \mathcal{K} \times (\mathcal{X}_1, \mathcal{X}_2) \rightarrow \mathcal{Y}$ be a quantum secure pseudorandom function for certain sets $\mathcal{K}, \mathcal{X}_1, \mathcal{X}_2, \mathcal{Y}$. For a random key $k \xleftarrow{\$} \mathcal{K}$, consider the adaptive reprogramming games in Figure 5. Let $\mathcal{A}$ be any efficient adversary conducting q_1 times quantum queries to QPRF and q_2 times reprogramming queries as defined in Figure 5. Then the distinguishing advantage of $\mathcal{A}$ is bounded by*

$$\left| \Pr(\text{Repro}_0^{\mathcal{A}} = 1) - \Pr(\text{Repro}_1^{\mathcal{A}} = 1) \right| \leq \frac{3q_2}{2} \sqrt{q_1/|\mathcal{X}_1|} + 2\varepsilon_{\text{QPRF}}, \quad (5)$$

where $\varepsilon_{\text{QPRF}}$ is the probability for the efficient quantum adversary to distinguish QPRF and random function.

Proof. For sets $\mathcal{X}_1, \mathcal{X}_2, \mathcal{Y}$, we first define the following adaptive reprogramming game for truly random function.

GAME RFRepro_b	$\text{RFReprogram}(x_2)$
1: $\mathcal{O}_0 \xleftarrow{\$} \mathcal{Y}^{\mathcal{X}_1 \times \mathcal{X}_2}$	1: $(x_1, y) \xleftarrow{\$} \mathcal{X}_1 \times \mathcal{Y}$
2: $\mathcal{O}_1 := \mathcal{O}_0$	2: $\mathcal{O}_1 := \mathcal{O}_1^{(x_1 \ x_2) \mapsto y}$
3: $b' \leftarrow A^{ \mathcal{O}_b , \text{Reprogram}}$	3: return x_1
4: return b'	

Fig. 17. Adaptive reprogramming games RFRepro_b for random function

Then, for $\mathcal{A}$ be any efficient adversary conducting q_1 times quantum queries to oracle $\mathcal{O}_b$ and q_2 times reprogramming queries as defined in the above Figure 17, we define two probabilities: $\hat{P}_{\mathcal{A}}^1 := \Pr(\text{RFRepro}_0^{\mathcal{A}} = 1)$ and $\hat{P}_{\mathcal{A}}^2 := \Pr(\text{RFRepro}_1^{\mathcal{A}} = 1)$. According to Proposition 1 in [27], it holds

$$\left| \hat{P}_{\mathcal{A}}^1 - \hat{P}_{\mathcal{A}}^2 \right| \leq \frac{3q_2}{2} \sqrt{\frac{q_1}{|\mathcal{X}_1|}}.$$

Thus, in order to prove (5), it suffices to prove

$$\left| P_{\mathcal{A}}^1 - \hat{P}_{\mathcal{A}}^1 \right| \leq \varepsilon_{\text{QPRF}} \text{ and } \left| P_{\mathcal{A}}^2 - \hat{P}_{\mathcal{A}}^2 \right| \leq \varepsilon_{\text{QPRF}}. \quad (6)$$

Furthermore, we just need to focus on the left-hand side of (6), as the right-hand side of (6) will be set up for the similar argument.

Particularly, we could establish the following reduction: suppose there is an efficient quantum adversary $\mathcal{D}$ distinguishing $P_{\mathcal{A}}^1$ and $\hat{P}_{\mathcal{A}}^1$ with probability ε , then we can construct another quantum adversary $\mathcal{B}$ breaking the stronger security of QPRF with probability ε . More precisely, according to Definition 3.7, suppose there is an oracle H^* , the goal of $\mathcal{B}$ is to distinguish $H^* = \text{QPRF}_k(\cdot)$ or $H^* \xleftarrow{\$} (\mathcal{X} \rightarrow \mathcal{Y})$. Now, $\mathcal{B}$ just needs to answer all $\mathcal{D}$'s queries through further querying H^* , and return the answer of $\mathcal{D}$ as his answer. Clearly, if $H^* = \text{QPRF}_k(\cdot)$, then $\mathcal{D}$ is interacting with the case of $P_{\mathcal{A}}^1$; Otherwise, $\mathcal{D}$ is interacting with the case of $\hat{P}_{\mathcal{A}}^1$.

Furthermore, combining with the stronger security of QPRF in Definition 3.7 and Lemma 3.8, we know that $\varepsilon \leq \varepsilon_{\text{QPRF}}$ for any efficient quantum algorithm $\mathcal{B}$, and thus the left-hand side of (6) holds. So, the right-hand side of (6) holds too. Summing up all above analysis, for any efficient adversary $\mathcal{A}$, (5) holds. $\square$

B.3 Detailed Proof for Theorem in Section 4.3

Theorem B.6 (Restatement of Theorem 4.10) *Assume QPRF exists, the above simulation for QROM in Figure 6 satisfies the separable property.*

Proof. Below, we first present an ideal simulation in the following Figure 18, and then prove it perfectly satisfies the separable property in Definition 4.9. Furthermore, we show that for any polynomial-time quantum adversary, the simulations

1. For any input $x \in \mathcal{X}$, sample one random bit string $(ra_1, ra_2) \xleftarrow{\$} \{0, 1\}^{l_{ra_1}} \times \{0, 1\}^{l_{ra_2}}$.
2. If the number of 1 in ra_1 is more than num , then sample $y_1 \leftarrow Y_1$ through using ra_2 as running randomness, return y_1 as $H(x)$. Here, num is set to make $\Pr[\|ra_1\|_1 > num] = p$.
3. Otherwise, compute $y_2 \leftarrow Y_2$ through using ra_2 as the running randomness, return y_2 as $H(x)$.

Fig. 18. Simulation of QROM $H : \mathcal{X} \rightarrow \mathcal{Y}$ with the separable property through using QPRF

in Figures 6 and 18 are indistinguishable. As a result, we can complete the proof of this theorem.

In Figure 18, as ra_1 is the truly random bit sting sampled from $\{0, 1\}^{l_{ra_1}}$, it holds $\Pr[\|ra_1\|_1 > num] = \frac{l_{ra_1} - num - 1}{l_{ra_1} + 1}$. Thus, it is easy for us to set the corresponding parameters num and p , such that $\Pr[\|ra_1\|_1 > num] = p$. Moreover, as ra_2 is the truly random bit sting sampled from $\{0, 1\}^{l_{ra_2}}$, the resulting y_1 and y_2 in Steps 3 and 4 of Figure 6 follow the desired distributions Y_1 and Y_2 , respectively. Clearly, such ideal simulation in Figure 18 satisfies the separable property in Definition 4.9.

Then, notice that the difference between Figure 6 and Figure 18 is that (ra_1, ra_2) is truly random or output by QPRF. Since Y_1, Y_2 are public sampling distributions, we can establish an efficient reduction from the indistinguishability of Figures 6 and 18 to the pseudorandomness of QPRF. As a result, we conclude that the simulation in Figure 6 satisfies the separable property, for any efficient quantum adversary. $\square$

C Supplementary for QDS₂ in Section 5

Due to the space limitation in the main body, we present many more supplementary for QDS₂ in Section 5

Theorem C.1 (Restatement of Theorem 5.1) *For public parameters as in Table 2, two-round threshold n -out-of- n signature $\text{QDS}_2 = (\text{Setup}, (\text{Gen}_u)_{u \in [n]}, (\text{Sign}_u)_{u \in [n]}, \text{Ver})$ in Figures 7, 8, 9 satisfies the correctness. In other words, suppose the underlying Dilithium scheme is correct, and the trapdoor commitment scheme Eqv-TCOM and the trapdoor-inversion function Inv-H are correct and additively homomorphic, then a valid generated signatures will be accepted by the verification algorithm, except with a negligible probability.*

Proof. Notice that the algorithm Ver in Figure 9 needs to conduct four checks. Thus, below we will discuss them one by one.

1. Due to the collision resistance of random oracle, it will output different values for different inputs, except with a negligible probability. Thus, for $c_{i,j} \leftarrow H_0(i, j, \mu, \text{pk}, \text{ck}, \text{ck}')$, all $c_{i,1}, \dots, c_{i,m}$ are pairwise distinct except with

the probability $m \cdot \text{negl}(\lambda)$. Clearly, this is still negligible in λ , when m is a polynomial.

2. For $\mathbf{y}_i^{(n)} \leftarrow D_{\sigma\sqrt{n}}^{\ell+k}$, $\mathbf{z}_{i,j}^{(n)} = c_{i,j}\mathbf{s}_n + \mathbf{y}_i^{(n)}$ and $\text{Rej}(\mathbf{z}_{i,j}^{(n)}, c_{i,j}\mathbf{s}_n, \sigma) \rightarrow 1$, we know that the distribution of $\mathbf{z}_{i,J_i}^{(n)}$ is statistically close to $D_{\sigma\sqrt{n}}^{(\ell+k)}$, according to Lemma A.8. Thus, we have $\|\mathbf{z}_{i,j}^{(n)}\| \leq \sigma\sqrt{2 \cdot (\ell+k) \cdot N} = B$ with overwhelming probability, according to Lemma A.3. Furthermore, according to Lemma A.4, we know that the distribution of $\mathbf{z}_i := \sum_{u \in [n]} \mathbf{z}_{i,J_i}^{(u)}$ is statistically close to $D_{\sigma\sqrt{n}}^{(\ell+k)}$. And thus, it holds $\|\mathbf{z}_i\| \leq \sigma\sqrt{2 \cdot n \cdot (\ell+k) \cdot N} = B_n$, except with a negligible probability.
3. Due to the correctness of the underlying Dilithium scheme, we know that $\overline{\mathbf{A}}\mathbf{z}_{i,J_i}^{(u)} = \mathbf{w}_i^{(u)} + c_{i,J_i}\mathbf{t}_u$ for each participant P_u with $u \in [n]$. Then, it holds $\overline{\mathbf{A}}\mathbf{z}_{i,J_i} = \mathbf{w}_i + c_{i,J_i}\mathbf{t}$, according to $\mathbf{w}_i = \sum \mathbf{w}_i^{(u)}$, $\mathbf{z}_i = \sum \mathbf{z}_i^{(u)}$, and $\mathbf{t} = \sum \mathbf{t}_u$ from the Sign and Gen protocols in Figures 8 and 7. Then, Verifier can reconstruct $\mathbf{w}_i := \overline{\mathbf{A}}\mathbf{z}_{i,J_i} - c_{i,J_i}\mathbf{t}$. And according to the homomorphic property and correctness of the used Eqv-TCOM, it holds $\text{Eqv-Open}_{\text{ck}}(\text{com}_i, r_i, \overline{\mathbf{A}}\mathbf{z}_i - c_{i,J_i}\mathbf{t}) = 1$, except with a negligible probability.
4. According to the homomorphic property and correctness of the used Inv-H, for all honestly generated signatures, it holds $\widetilde{\text{com}}_{i,J_i} = \mathbf{A}_{\text{Inv}} \cdot \mathbf{z}_i + \mathbf{r}'_{i,\cdot}$, except with a negligible probability.

Summing up all above analysis, the honestly generated signatures should be accepted, except with at most a negligible probability. $\square$

Theorem C.2 (Restatement of Theorem 5.2) *Suppose that the trapdoor commitment scheme Eqv-TCOM is hiding, binding, additively homomorphic, and has a secure equivocation trapdoor, uniform keys and uniform commitment. Particularly, the output of $\text{Eqv-TCommit}_{\text{tck}}(\text{td})$ has sufficient bit-length ϑ . And suppose there exists QPRF that can be programmable and invertible simultaneously under $\text{DSPR}_{\bar{q}, \bar{R}, \chi}$ assumption. For any quantum polynomial-time adversary $\mathcal{A}$ that initiates a single key generation protocol by querying $\mathcal{O}_{\text{Gen}}(\cdot, \cdot)$, initiates Q_s signature generation protocols by querying $\mathcal{O}_{\text{Sign}_1}(\cdot, \cdot), \mathcal{O}_{\text{Sign}_2}(\cdot, \cdot)$ with sid , and makes Q_h quantum superpositions queries to random oracle $H_0, H_1, H'_1, H_2, H_3, H_4, H_5$, the protocol QDS₂ of Figures 7, 8, 9 is QDS-UF-CMA secure under $\text{MSIS}_{q,k,k+\ell+1,\beta}$ and $\text{MLWE}_{q,\ell,k,\eta}$ assumptions in the QROM, where $\beta = 2\sqrt{B_n^2 + \kappa}$. Concretely, using other parameters specified in Table 2, the advantage of $\mathcal{A}$ is bounded as follows.*

$$\begin{aligned}
\text{Adv}_{\text{QDS}_2}^{\text{UF-CMA}}(\mathcal{A}) &\leq 2\varepsilon_{\text{Inj-QPRF}} + 5\varepsilon_{\text{QPRF}} + e(Q_h + Q_s + 1) \left[(Q_h + Q_s) \cdot t \cdot \varepsilon_{\text{td}} \right. \\
&\quad + 2(Q_h + Q_s) \cdot \varepsilon_{\text{QPRF}} + \frac{3Q_s}{2} \sqrt{Q_s + Q_h} 2^{-\frac{t \cdot \vartheta}{2}} + 2\varepsilon_{\text{QPRF}} + t \cdot Q_s \cdot (m-1) \\
&\quad \cdot \text{Adv}_{\text{MLWE}_{q,(\ell+k),m',\chi}} + t \cdot Q_s \cdot \varepsilon_{\text{Rej}} + \frac{3}{2} \sqrt{Q_h} (2^{-\frac{l_1^*}{2}} + 2^{-\frac{kN \log q}{2}}) \\
&\quad + 4(\varepsilon_{\text{QPRF}} + \varepsilon_{\text{Inj-QPRF}}) + \text{Adv}_{\text{MLWE}_{q,\ell,k,\eta}} + 2(Q_h + 1) 2^{-(t \log m)/2} + \varepsilon_{\text{bind}} \\
&\quad \left. + \text{Adv}_{\text{MSIS}_{q,k,k+\ell+1,\beta}} \right]
\end{aligned}$$

Below, we first sketch the proof idea, before presenting the detailed proof. According to Definition A.16, we need to prove that for any efficient adversary $\mathcal{A}$ against QDS_2 , its advantage $\text{Adv}_{\text{QDS}_2}^{\text{UF-CMA}}(\mathcal{A})$ is negligible. In order to do this, we conduct the following two steps:

- We first show that the party P_n in the experiment $\text{Adv}_{\text{QDS}_2}^{\text{UF-CMA}}(\mathcal{A})$ can be simulated by a simulator $\mathcal{B}$ defined in Figure 19, together with its subroutines Figures 21, 22, and 23. We show that $\mathcal{B}$ do not have any secret key, through using a sequence of hybrid experiments. Particularly, we present more details on such simulator $\mathcal{B}$ as follows:
 1. For $\text{QDS}_2.\text{SimGen}_n$, we use the invertible and adaptively programmable properties of QROM H_1, H'_1 and H_2 .
 2. For $\text{QDS}_2.\text{SimSign}_n(\text{sid}, m)$, we use the separable property of H_3 , the adaptively programmable property of H_5 , and the equivocal property of trapdoor commitment scheme Eqv-TCOM .
 3. For random oracle simulation, we use the QPRF to simulate the quantum random oracle, which satisfy the requirements of inversion and adaptive programming. Moreover, for the separable property of H_3 , we use the QPRF as an intermediate tool, i.e., use the output of QPRF as randomness to invoke the corresponding key generation algorithm CGen or TCGen .
- Then, we show that in such a simulated experiment, the signature is unforgeability, through establishing a reduction to MSIS and the binding property of Eqv-TCOM , following from the similar proof idea of [42]. Particularly, we first show that there is an efficient extractor Ext in Figure 20, such that given a valid forgery message-signature pair (μ^*, Sig^*) for $\mu^* \notin \text{Mset}$, Ext can output a solution for MSIS problem, if the used Eqv-TCOM satisfies the binding property. And then, we bound the probability of generating a valid forgery message-signature pair (μ^*, Sig^*) for $\mu^* \notin \text{Mset}$ by the union bound of two events happen: Ext succeeds and Ext fails.

Proof. We first begin with the real experiment denoted as $\mathbf{G}_0$.

$\mathbf{G}_0$ This is the real experiment just as defined in Figure 13. Here $\mathcal{B}$ holds the real random oracles $H_0, H_1, H'_1, H_2, H_3, H_4, H_5$, and allows $\mathcal{A}$ to query all H_i and H'_1 in superpositions. Besides, with the honestly generated secret key share s_n , $\mathcal{B}$ answers $\mathcal{A}$'s key generation and signature generation queries, just as in Figures 14, which invokes Figures 7 and Figures 8. Let $\text{Pr}[\mathbf{G}_i]$ denote a probability that $\mathcal{A}$ wins the experiment $\mathbf{G}_i$, i.e., outputs a valid forgery, at the game $\mathbf{G}_i$.

Below, we explicitly describe the Forgery phase in the experiment as follows, as we will need to modify its certain steps in the following hybrid experiments.

Forgery. When $\mathcal{A}$ outputs a forgery $\widehat{\text{Sig}}^* = (\{\text{com}^*\}_{i \in [t]}, \{\widetilde{\text{com}}^*\}_{i \in [t], j \in [m]}, \{\mathbf{z}_i^*\}_{i \in [t]}, \{\mathbf{r}_i^*\}_{i \in [t]}, \{\mathbf{r}'_i^*\}_{i \in [t]})$ for μ^* at the end of experiment, $\mathcal{B}$ proceeds as follows.

1. If $\mu^* \in \text{Mset}$ then $\mathcal{B}$ halts with output $\perp$.

Algorithm $\mathcal{B}(\mathbf{A}, \mathbf{t})$ The algorithm is initialized with a set $\text{Mset} = \emptyset$ and a flag $\text{BAD}_4 = \text{false}$. Here, Mset is used to store the queried messages.

Honest party oracle simulation. Upon receiving a query of the form (sid, m) from $\mathcal{A}$, reply the query as described in $\text{QDS}_2.\text{SimGen}_n$ (Fig.21) and $\text{QDS}_2.\text{SimSign}_n(\text{sid}, m)$ (Fig.23). If $\text{QDS}_2.\text{SimGen}_n$ or $\text{SimO}^{\text{QDS}_2}_n(\text{sid}, m)$ halts with output $\perp$ then $\mathcal{B}$ also halts with output $\perp$.

Random oracle simulation. Upon receiving a query to the random oracles from $\mathcal{A}$, reply the query as described in Fig.22.

Forgery. Upon receiving a forgery $(\mu^*, \widehat{\text{Sig}}^*) = (\mu^*, \{\text{com}_i^*\}_{i \in [t]}, \{\widehat{\text{com}}_{i,j}^*\}_{i \in [t], j \in [m]}, \{\mathbf{z}_i^*\}_{i \in [t]}, \{r_i^*\}_{i \in [t]}, \{r_i'^*\}_{i \in [t]})$ from $\mathcal{A}$, it conducts:

1. If $\mu^* \in \text{Mset}$ then $\mathcal{B}$ halts with output $\perp$.
2. Make queries $\text{ck}^* \leftarrow H_3(\mu^*, \text{pk})$, $\mathbf{A}_{\text{Inv}}^* \leftarrow H_4(\mu^*, \text{pk})$, $c_{i,j}^* \leftarrow H_0(i, j, \mu^*, \text{pk}, \text{ck}^*, \mathbf{A}_{\text{Inv}}^*)$ where $i \in [t]$, $j \in [m]$ and $J_1 || \dots || J_t \leftarrow H_5(\text{pk}, \{\text{com}_i^*\}_{i \in [t]}, \{c_{i,j}^*\}_{i \in [t], j \in [m]}, \{\widehat{\text{com}}_{i,j}^*\}_{i \in [t]})$.
3. If $\|\mathbf{z}_i^*\| > B_n$ or $\text{Eqv-Open}_{\text{ck}^*}(\text{com}_i^*, r_i^*, \overline{\mathbf{A}}\mathbf{z}_i^* - c_{i,J_i}^* \mathbf{t}) \neq 1$ or $\widehat{\text{com}}_{i,J_i}^* \neq \text{Inv-H}(\mathbf{z}_i^*) = \mathbf{A}_{\text{Inv}}^* \cdot \mathbf{z}_i^* + r_i'^*$, then $\mathcal{B}$ halts with output $\perp$. Otherwise, $\mathcal{B}$ computes $H_3'(\mu^*, \text{pk})$. If the output of $H_3'(\mu^*, \text{pk})$ is not $\perp$, (i.e., Eqv-TCGen was called), then set flag $\text{BAD}_4 = 1$ and $\mathcal{B}$ halts with output $\perp$.
4. $\mathcal{B}$ halts with output $(\mu^*, \{\text{com}_i^*\}_{i \in [t]}, \{\widehat{\text{com}}_{i,j}^*\}_{i \in [t], j \in [m]}, \{\mathbf{z}_i^*\}_{i \in [t]}, \{r_i^*\}_{i \in [t]}, \{r_i'^*\}_{i \in [t]})$.

Fig. 19. The algorithms simulating the view of $\mathcal{A}$ in $\text{Exp}_{\text{QDS}_2}^{\text{UF-CMA}}(\mathcal{A})$ experiment

Input : $H_0, H_3, H_4, H_4', H_5, \text{pk}, \mu, \widehat{\text{Sig}} = (\{\text{com}_i\}_{i \in [t]}, \{\widehat{\text{com}}_{i,j}\}_{i \in [t], j \in [m]}, \{\mathbf{z}_i\}_{i \in [t]}, \{r_i\}_{i \in [t]}, \{r_i'\}_{i \in [t]})$, compute $\text{ck} \leftarrow H_3(\mu, \text{pk})$, $\mathbf{A}_{\text{Inv}} \leftarrow H_4(\mu, \text{pk})$, $c_{i,j} \leftarrow H_0(i, j, \mu, \text{pk}, \text{ck}, \text{ck}')$ for all $i \in [t]$, $j \in [m]$, and $J_1 || \dots || J_t \leftarrow H_5(\text{pk}, \mu, \{\text{com}_i\}_{i \in [t]}, \{c_{i,j}\}_{i \in [t], j \in [m]}, \{\widehat{\text{com}}_{i,j}\}_{i \in [t], j \in [m]})$. Furthermore, conduct the followings, for $i = 1$ to t do
for $j = 1$ to m except J_i do
for each $\mathbf{z}_{i,j}' \leftarrow \text{Inv}(\widehat{\text{com}}_{i,j}, \mathbf{A}_{\text{Inv}})$ do
if $\|\mathbf{z}_{i,j}'\| \leq B \wedge \text{Eqv-Open}_{\text{ck}}(\text{com}_i, r_i, \mathbf{w}_i := \overline{\mathbf{A}}\mathbf{z}_{i,j}' - c_{i,j} \mathbf{t})$
return $(\mathbf{z}_i - \mathbf{z}_{i,j}', c_{i,J_i} - c_{i,j})$

Fig. 20. Extractor for $\mathbf{G}_9$

Protocol $\text{QDS}_2.\text{SimGen}_n(\text{pp}, \mathbf{A}, \mathbf{t})$. The simulator is parameterized by public parameters described in Table 2 and relies on the random oracles: $H_1 : \{0, 1\}^{t_1} \rightarrow \{0, 1\}^{l_1}$, $H_1' : \{0, 1\}^{t_1} \rightarrow \{0, 1\}^{l_1'}$, $H_2 : \{0, 1\}^{l_2} \rightarrow \{0, 1\}^{l_2}$. The variables $\text{BAD}_1, \text{BAD}_2$ is initially set to false

Matrix Generation

1. Sample a random seed $s_n \in \{0, 1\}^{1 - \log n}$, generate and output a random oracle commitment $g_n \leftarrow H_1(s_n, n)$.
2. Upon receiving g_u for all $u \in [n-1]$,
 - (a) Invoke Algorithm 1 on input $(g_1, \dots, g_{n-1})$ to obtain $((s_1, 1), \dots, (s_{n-1}, n-1))$.
 - (b) Compute $\mathbf{A}_u = H_1'(s_u, u)$ for all $u \in [n-1]$.
 - (c) Compute $\mathbf{A}_n := \mathbf{A} - \sum_{u=1}^{n-1} \mathbf{A}_u$.
 - (d) Reprogram the random oracle $H_1'(s_n, n) := \mathbf{A}_n$ and set public random matrix $\overline{\mathbf{A}} := [\mathbf{A} | \mathbf{I}] \in R_q^{k \times (\ell+k)}$, where $\mathbf{A} := \sum_{u \in [n]} \mathbf{A}_u$.
 - (e) Send out the seed s_n .
3. Upon receiving s_u for all $u \in [n-1]$, if $H_1(s_u, u) \neq g_u$ for some u , then send out $\perp$.

Key Pair Generation

1. Sample $g_n' \xleftarrow{\$} \{0, 1\}^{l_2}$ and send out g_n' .
 2. Upon receiving g_u' for all $u \in [n-1]$ proceed as follows.
 - (a) Invoke Algorithm 1 on input $(g_1', \dots, g_{n-1}')$ to obtain $((t_1, 1), \dots, (t_{n-1}, n-1))$.
 - (b) Compute $\mathbf{t}_n := \mathbf{t} - \sum_{u=1}^{n-1} \mathbf{t}_u$.
 - (c) Reprogram the random oracle $H_2(\mathbf{t}_n, n) := g_n'$ and then send out $\mathbf{t}_n$.
 3. Upon receiving $\mathbf{t}_u$ for all $u \in [n-1]$, if $H_2(\mathbf{t}_u, u) \neq g_u'$ for some u then send out $\perp$.
- If neither the protocol does not output $\perp$, the simulator obtains public key share $\mathbf{t}_n$ and $\text{pk} = (\mathbf{A}, \mathbf{t})$ as local output.

Fig. 21. Key generation simulator for QDS_2

Algorithm Random Oracle Simulation

$H_0(x)$

1. Simulate H_0 as $\text{QPRF}_{k_0} : \{0, 1\}^{l_0^*} \rightarrow \{0, 1\}^{l_0}$, where $k_0 \xleftarrow{\$} \mathcal{K}$
2. Return $H_0(x)$

$H_1(x)$

1. Simulate H_1 as $\text{Inj-QPRF}_{k_1} : \{0, 1\}^{l_1^*} \rightarrow \{0, 1\}^{l_1}$, where $k_1 \xleftarrow{\$} \mathcal{K}$
2. Return $H_1(x)$

$H'_1(x)$

1. Simulate H'_1 as $\text{QPRF}_{k'_1} : \{0, 1\}^{l_1^*} \rightarrow \{0, 1\}^{l'_1}$, where $k'_1 \xleftarrow{\$} \mathcal{K}$
2. Return $H'_1(x)$

$H_2(x)$

1. Simulate H_2 as $\text{Inj-QPRF}_{k_2} : \{0, 1\}^{l_2^*} \rightarrow \{0, 1\}^{l_2}$, where $k_2 \xleftarrow{\$} \mathcal{K}$
2. Return $H_2(t_u, u)$

$H_3(x), H'_3(x)$

1. Parse x as (μ, pk)
2. Invoke $\text{QPRF}_{k_3}(\mu, \text{pk}) : \{0, 1\}^{l_3^*} \rightarrow (\{0, 1\}^{l_{ra1}} \times \{0, 1\}^{l_{ra2}})$, where $k_3 \xleftarrow{\$} \mathcal{K}$ and l_{ra1}, l_{ra2} are the bit-lengths of ra_1, ra_2 , respectively.
3. Compute $(ra_1, ra_2) = \text{QPRF}_{k_3}(\mu, \text{pk})$
4. If the number of 1 in ra_1 is more than num , then compute $(\text{tck}, \text{td}) \leftarrow \text{Eqv-TCGen}(\text{cpp}_{\text{Eqv}}, ra_2)$, return tck and td as $H_3(\mu, \text{pk})$ and $H'_3(\mu, \text{pk})$, respectively. Here, num is set to make $\Pr[\|ra_1\|_1 > num] = \varpi$
5. Otherwise, compute $\text{ck} \leftarrow \text{Eqv-CGen}(\text{cpp}_{\text{Eqv}}, r_2)$, return ck and $\perp$ as $H_3(\mu, \text{pk})$ and $H'_3(\mu, \text{pk})$, respectively.

$H_4(x), H'_4(x)$

1. Parse x as (μ, pk)
2. Invoke $\text{QPRF}_{k_4}(\mu, \text{pk}) : \{0, 1\}^{l_4^*} \rightarrow \{0, 1\}^{l_r}$, where $k_4 \xleftarrow{\$} \mathcal{K}$ and l_r is the length of r .
3. Compute $r = \text{QPRF}_{k_4}(\mu, \text{pk})$
4. Compute $(\mathbf{A}_{\text{Inv}}, \mathbf{T}_{\mathbf{A}_{\text{Inv}}}) \leftarrow \text{TrapGen}(1^n, q, m)$ with r as the running randomness, and return $\mathbf{A}_{\text{Inv}}$ and $\mathbf{T}_{\mathbf{A}_{\text{Inv}}}$ as $H_4(x)$ and $H'_4(x)$, respectively.

$H_5(x)$

1. Simulate H_5 as $\text{QPRF}_{k_5} : \{0, 1\}^{l_5^*} \rightarrow \{0, 1\}^{l_5}$, where $k_5 \xleftarrow{\$} \mathcal{K}$
2. Return $H_5(x)$

Fig. 22. Quantum random oracle simulator and the related functions for QDS_2

2. Compute $\text{ck}^* \leftarrow H_3(\mu^*, \text{pk})$, $\mathbf{A}_{\text{Inv}}^* \leftarrow H_4(\mu^*, \text{pk})$, $c_{i,j}^* \leftarrow H_0(i, j, \mu^*, \text{pk}, \text{ck}^*, \mathbf{A}_{\text{Inv}}^*)$ where $i \in [t], j \in [m]$ and $J_1^* || \dots || J_t^* \leftarrow H_5(\text{pk}, \mu^*, \{\text{com}_i^*\}_{i \in [t]})$, $\{\widetilde{c}_{i,j}^*\}_{i \in [t], j \in [m]}, \{\widetilde{\text{com}}_{i,j}^*\}_{i \in [t]}$.
3. If $\|z_i^*\| > B_n$ or $\text{Eqv-Open}_{\text{ck}^*}(\text{com}_i^*, r_i^*, \overline{\mathbf{A}} z_i^* - c_{i,J_i^*}^* t) \neq 1$ or $\widetilde{\text{com}}_{i,J_i^*}^* \neq \text{Inv-H}(z_i^*) = \mathbf{A}_{\text{Inv}} \cdot z_i^* + \mathbf{r}_i'^*$ then $\mathcal{B}$ halts with output $\perp$.
4. $\mathcal{B}$ halts with output $(\mu^*, \text{Sig}^*) = (\mu^*, \{\text{com}_i^*\}_{i \in [t]}, \{\widetilde{\text{com}}_{i,j}^*\}_{i \in [t], j \in [m]}, \{z_i^*\}_{i \in [t]}, \{r_i^*\}_{i \in [t]}, \{\mathbf{r}_i'^*\}_{i \in [t]})$.

Thus, we have

$$\Pr[\mathbf{G}_0] = \text{Adv}_{\text{QDS}_2}^{\text{UF-CMA}}(\mathcal{A}).$$

$\mathbf{G}_1$ This experiment is identical to $\mathbf{G}_0$, except that the random oracles H_0, H_1, H'_1, H_2, H_5 are simulated by QPRFs. Among them, $H_0 : \{0, 1\}^{l_0^*} \rightarrow C, H'_1 : \{0, 1\}^{l_1^*} \rightarrow \{0, 1\}^{l'_1}, H_5 : \{0, 1\}^{l_5^*} \rightarrow \{0, 1\}^{l_5}$ are simulated as QPRFs in Construct 4.1. According to Theorem 4.3, QPRF and quantum random oracle are computationally indistinguishable except with a negligible probability $\varepsilon_{\text{QPRF}}$, for any efficient quantum adversary. $H_1 : \{0, 1\}^{l_1^*} \rightarrow \{0, 1\}^{l_1}, H_2 : \{0, 1\}^{l_2^*} \rightarrow \{0, 1\}^{l_2}$ are simulated as Inj-QPRFs in Construct 4.5. According to Lemma 4.6, Construct 4.1 and Construct 4.5 are computational indistinguishability except with a negligible probability $\varepsilon_{\text{Inj-QPRF}}$, for any efficient

Protocol QDS₂.SimSign_n(sid, $\mathbf{t}_n$, pk, μ) The simulator is parameterized by public parameters described in Table 2 and relies on the random oracles $H_0 : \{0, 1\}^{l_0^*} \rightarrow C$, $H_3 : \{0, 1\}^{l_3^*} \rightarrow \text{Eqv-}S_{\text{ck}}$, $H_4 : \{0, 1\}^{l_4^*} \rightarrow \text{Inv-}S_{\text{ck}}$ and $H_5 : \{0, 1\}^{l_5^*} \rightarrow \{0, 1\}^{l_5}$. The simulator assumes that QDS₂.SimGen_n(pp) has been previously invokes. If a party halts with $\perp$ at any point, then all SimSign_n(sid, $\mathbf{t}_n$, pk, μ) executions are aborted. The variable BAD₃ is initially set to false. Inputs 1. The simulator receives a unique sessions ID sid, $\mathbf{t}_n$, pk = ($\mathbf{A}$, $\mathbf{t}$) and message $\mu \in M$ as input. 2. The simulator verifies that sid has not been used before (if it has been, the protocol is not executed). 3. The simulator locally computes a per-message commitment key by computing $\mathbf{A}_{\text{Inv}} \leftarrow H_4(\mu, \text{pk})$. 4. The simulator locally computes a per-message commitment key by computing $\text{tck} \leftarrow H_3(\mu, \text{pk})$. Moreover, compute $\text{QPRF}_{k_3}(\mu, \text{pk}) = (ra_1, ra_2)$. If the number of 1 in ra_1 is less than num, then set BAD₃ = 1 and simulation fails with output $\perp$. Otherwise obtain $(\text{tck}, \text{td}) \leftarrow \text{Eqv-TCGen}(\text{cpp}_{\text{Eqv}})$ with ra_2 as the running randomness. Signature Generation P_n works as follows: 1. $\mathcal{O}_{\text{Sign}_1}(\text{sid}, \mu)$: Up receiving message μ, conduct the first round interaction as follows. (a) Sample $(J_1 \ \dots \ J_t) \xleftarrow{\\$} \mathbb{Z}_m^t$, where $J_i \in \mathbb{Z}_m$. (b) for $i = 1$ to t; conduct as follows: i. for $j = J_i$, conduct as follows: A. Derive challenges $c_{i,J_i} \leftarrow H_0(i, J_i, \mu, \text{pk}, \text{tck}, \mathbf{A}_{\text{Inv}})$. B. Sample $\mathbf{z}_{i,J_i}^{(n)} \xleftarrow{\\$} D_s^{\ell+k}$. C. Output $\mathbf{z}_{i,J_i}^{(n)}$ with probability $1/M$. ii. If the above $\mathbf{z}_{i,J_i}^{(n)}$ does not output, then go to the Step i. (c) For all $j \in [t] \setminus \{J_i\}$, sample $\widehat{\text{com}}_{i,J_i}^{(n)} \xleftarrow{\\$} R_q^{m'}$. And compute $\widehat{\text{com}}_{i,J_i}^{(n)} \leftarrow \text{Inv-H}(\mathbf{z}_{i,J_i}^{(n)}) = \mathbf{A}_{\text{Inv}} \cdot \mathbf{z}_{i,J_i}^{(n)} + \mathbf{r}_i'^{(n)}$, where $\mathbf{r}_i'^{(n)} \leftarrow D_{\sigma'}^{m'}$. (d) Compute $\text{com}_i^{(n)} \leftarrow \text{Eqv-TCommit}_{\text{tck}}(\text{td})$, and then derive randomness $r_i^{(n)} \leftarrow \text{Eqv}_{\text{tck}}(\text{td}, \text{com}_i^{(n)}, \mathbf{w}_i^{(n)})$. (e) Set $\text{st}_n := (\{\mathbf{r}_i'^{(n)}, r_i^{(n)}\}_{i \in [t], j \in [m]}, \mathbf{z}_{i,J_i}^{(n)})$. (f) Send out $\text{msg}_n := (\{\text{com}_i^{(n)}\}_{i \in [t]}, \{\widehat{\text{com}}_{i,j}^{(n)}\}_{i \in [t], j \in [m]})$. 2. $\mathcal{O}_{\text{Sign}_2}(\text{sid}, \{\text{msg}_u\}_{u \in [n-1]})$: Upon receiving $\text{msg}_u := (\{\text{com}_i^{(u)}\}_{i \in [t]}, \{\widehat{\text{com}}_{i,j}^{(u)}\}_{i \in [t], j \in [m]})$ for all $u \in [n-1]$, compute the signature shares as follows. (a) Set $\text{com}_i := \sum_{u \in [n]} \text{com}_i^{(u)}$ and $\widehat{\text{com}}_{i,j} := \sum_{u \in [n]} \widehat{\text{com}}_{i,j}^{(u)}$ for all $i \in [t], j \in [m]$. (b) Reprogram H_5 as $H_5(\text{pk}, \mu, \{\text{com}_i\}_{i \in [t]}, \{c_{i,j}\}_{i \in [t], j \in [m]}, \{\widehat{\text{com}}_{i,j}\}_{i \in [t], j \in [m]}) := (J_1 \ \dots \ J_t)$. (c) Derive randomness $r_i^{(n)} \leftarrow \text{Eqv}_{\text{tck}}(\text{td}, \text{com}_i^{(n)}, \mathbf{w}_i^{(n)} = \overline{\mathbf{A}} \mathbf{z}_{i,J_i}^{(n)} - c_{i,J_i} \mathbf{t}_n)$ for $i \in [t]$. (d) Send out $\text{Sig}_n := (\{\text{com}_i^{(n)}\}_{i \in [t]}, \{\widehat{\text{com}}_{i,j}^{(n)}\}_{i \in [t], j \in [m]}, \{\mathbf{z}_{i,J_i}^{(n)}\}_{i \in [t]}, \{r_i^{(n)}\}_{i \in [t]}, \{\mathbf{r}_i'^{(n)}\}_{i \in [t]})$. 3. $\text{SAgg}(\text{sid}, \{\text{Sig}_1, \dots, \text{Sig}_n\})$: Upon receiving $\text{Sig}_u := (\{\mathbf{z}_{i,J_i}^{(u)}\}_{i \in [t]}, \{r_i^{(u)}\}_{i \in [t]}, \{\mathbf{r}_i'^{(u)}\}_{i \in [t]})$ for all $u \in [n]$ compute the combined signature as follows. (a) Set $\text{com}_i := \sum_{u \in [n]} \text{com}_i^{(u)}$ and $\widehat{\text{com}}_{i,j} := \sum_{u \in [n]} \widehat{\text{com}}_{i,j}^{(u)}$ for all $i \in [t], j \in [m]$. (b) Compute $\mathbf{z}_i := \sum_{u \in [n]} \mathbf{z}_{i,J_i}^{(u)}$, $r_i := \sum_{u \in [n]} r_i^{(u)}$ and $\mathbf{r}_i' := \sum_{u \in [n]} \mathbf{r}_i'^{(u)}$ for all $i \in [t]$. (c) If the protocol does not abort, P_n obtains a signature $\widehat{\text{Sig}} := (\{\text{com}_i\}_{i \in [t]}, \{\widehat{\text{com}}_{i,j}\}_{i \in [t], j \in [m]}, \{\mathbf{z}_i\}_{i \in [t]}, \{r_i\}_{i \in [t]}, \{\mathbf{r}_i'\}_{i \in [t]})$ as local output.
--

Fig. 23. Signature generation simulator for QDS₂

quantum adversary. Thus, we have

$$|\Pr[\mathbf{G}_1] - \Pr[\mathbf{G}_0]| \leq 5 \cdot \varepsilon_{\text{QPRF}} + 2 \cdot \varepsilon_{\text{Inj-QPRF}}.$$

G₂ This experiment is identical to **G₁**, except with the simulation of H_3 , H_4 and the related several differences in QDS.Sign_n.

For the simulation of H_4 . When receiving a query (μ, pk) , $\mathcal{B}$ runs $\mathbf{A}_{\text{Inv}} \leftarrow H_4(\mu, \text{pk})$ as in Figure 22. Particularly, $\mathcal{B}$ first computes $r \leftarrow \text{QPRF}_{k_4}(\mu, pk)$,

where QPRF_{k_4} is a quantum secure pseudorandom function as Construct 4.1. Then, $\mathcal{B}$ invokes $(\mathbf{A}_{\text{Inv}}, \mathbf{T}_{\mathbf{A}_{\text{Inv}}}) \leftarrow \text{TrapGen}(1^{k+\ell}, q, m')$, through using r as the running randomness. Finally, $\mathcal{B}$ returns $\mathbf{A}_{\text{Inv}}$ as the output of $H_4(\mu, \text{pk})$.

For the simulation of H_3 . When receiving a query (μ, pk) , $\mathcal{B}$ runs $\text{tck}/\text{ck} \leftarrow H_3(\mu, \text{pk})$ as in Figure 22. Particularly, $\mathcal{B}$ first computes $(ra_1, ra_2) \leftarrow \text{QPRF}_{k_3}(\mu, \text{pk})$, where QPRF_{k_3} is a quantum secure pseudorandom function as Construct 4.1, then

- If the number of 1 in ra_1 is more than num , then $\mathcal{B}$ invokes Eqv-TCGen with cpp_{Eqv} and ra_2 as public parameter and randomness respectively, to obtain (tck, td) . Finally, $\mathcal{B}$ returns tck as the output of $H_3(\mu, \text{pk})$.
- Otherwise, $\mathcal{B}$ invokes Eqv-CGen with cpp_{Eqv} and ra_2 as public parameter and randomness respectively, to obtain ck . Finally, $\mathcal{B}$ returns ck as the output of $H_3(\mu, \text{pk})$.

In the above process, $\mathcal{B}$ also runs the H'_3 as in Figure 22 to get $\text{td} = H'_3(\mu, \text{pk})$. Here, we set the value num such that the probability that the number of 1 in ra_1 is more than num is ϖ .

Based on the above simulation for H_3 , H_4 , $\mathbf{G}_2$ has the following concrete differences with QDS.Sign_n in $\mathbf{G}_1$.

- With respect to **Inputs 3**: Given (μ, pk) , compute $(ra_1, ra_2) \leftarrow \text{QPRF}_{k_3}(\mu, \text{pk})$. If the number of 1 in ra_1 is less than num (i.e., Eqv-TCGen was not called), then set the flag $\text{BAD}_3 = 1$ and halts with output $\perp$. Otherwise obtain the trapdoor $(\text{tck}, \text{td}) \leftarrow \text{Eqv-TCGen}(\text{cpp}_{\text{Eqv}})$, through using ra_2 as the running randomness.
- With respect to **Signature Generation 1.(c)**: For $i \in [t]$, generate $\text{com}_i^{(n)} \leftarrow \text{Eqv-TCommit}_{\text{tck}}(\text{td})$, and then derive randomness $r_i^{(n)} \leftarrow \text{Eqv}_{\text{tck}}(\text{td}, \text{com}_i^{(n)}, w_i^{(n)})$, instead of computing $\text{com}_i^{(n)} \leftarrow \text{Eqv-Commit}_{\text{ck}}(w_i^{(n)}, r_i^{(n)})$ with $r_i^{(n)} \xleftarrow{\$} \text{Eqv-S}_r$.

Moreover, $\mathbf{G}_2$ has the following concrete differences with respect to the **Forgery** phase in $\mathbf{G}_1$. Particularly, when $\mathcal{A}$ outputs a successful forgery $(\mu^*, \{\text{com}_i^*\}_{i \in [t]}, \{\widetilde{\text{com}}_{i,j}^*\}_{i \in [t], j \in [m]}, \{z_i^*\}_{i \in [t]}, \{r_i^*\}_{i \in [t]}, \{r'_{i,J_i}\}_{i \in [t]})$ at the end of the experiment, we modify the step 3 of $\mathbf{G}_2$ as follows.

Forgery 3. If $\|z_i^*\| > B_n$ or $\text{Eqv-Open}_{\text{ck}^*}(\text{com}_i^*, r_i^*, \overline{\mathbf{A}}z_i^* - c_{i,J_i}^* t) \neq 1$ or $\widetilde{\text{com}}_{i,J_i}^* \neq \text{Inv-H}(z_i^*) = \mathbf{A}_{\text{Inv}} \cdot z_i^* + r_i'^*$ then $\mathcal{B}$ halts with output $\perp$. Furthermore, $\mathcal{B}$ computes $H'_3(\mu^*, \text{pk})$. If the output of $H'_3(\mu^*, \text{pk})$ is not $\perp$, (i.e., Eqv-TCGen was called), then set flag $\text{BAD}_4 = 1$ and $\mathcal{B}$ halts with output $\perp$.

Note that due to the way H_3 is simulated, if $\mathcal{B}$ does not output $\perp$, it is now guaranteed that ck^* is generated by Eqv-CGen instead of Eqv-TCGen . Furthermore, according to the security of Eqv-TCOM , we have

$$\Pr[\mathbf{G}_2] \geq \varpi^{Q_h + Q_s} \cdot (1 - \varpi) \cdot \Pr[\mathbf{G}_1] - (Q_h + Q_s) \cdot t \cdot \varepsilon_{\text{td}} - 2(Q_h + Q_s) \cdot \varepsilon_{\text{QPRF}},$$

where ε_{td} is the statistical distance of true commitment and trapdoor commitment for Eqv-TCOM .

In other word, it is only successful neither BAD_3 nor BAD_4 is set above. Note that by setting $\varpi = (Q_h + Q_s)/(Q_h + Q_s + 1)$ since $(1/(1 + 1/(Q_h + Q_s)))^{(Q_h + Q_s)} \geq 1/e$ for $Q_h + Q_s \geq 0$ we obtain

$$\Pr[\mathbf{G}_2] \geq \frac{\Pr[\mathbf{G}_1]}{e(Q_h + Q_s + 1)} - (Q_h + Q_s) \cdot t \cdot \varepsilon_{\text{td}} - 2(Q_h + Q_s) \cdot \varepsilon_{\text{QPRF}}.$$

$\mathbf{G}_3$ This game is identical to $\mathbf{G}_2$ except at the choice of the challenge $J_1 || \dots || J_t$. Particularly, instead of getting the challenge $J_1 || \dots || J_t \leftarrow \mathbf{H}_5(\cdot)$, $\mathcal{B}$ firstly chooses $J_1 || \dots || J_t$ at random at the beginning of the Signature Generation algorithm. Then, at the step of **Signature Generation 2.(b)** of Figure 8, $\mathcal{B}$ reprograms the random oracle $\mathbf{H}_5$ at the particular input $x := (\text{pk}, \mu, \{\text{com}_i\}_{i \in [t]}, \{c_{i,j}\}_{i \in [t], j \in [m]}, \{\widetilde{\text{com}}_{i,j}\}_{i \in [t], j \in [m]})$, such that $\mathbf{H}_5(x) := J_1 || \dots || J_t$.

Notice that, the above reprogramming is regarded to $\mathbf{H}_5$, which will not affect the rejection sampling steps in the step of **Signature Generation 1.(a)** of Figure 8. Moreover, according to the properties of Eqv-TCommit , it holds $\text{com}_i^{(n)} \leftarrow \text{Eqv-TCommit}_{\text{td}}(\text{td})$ has sufficient min-entropy ϑ , so does $\text{com}_i := \sum_{u \in [n]} \text{com}_i^{(u)}$. This further implies the programmed point x has sufficient min-entropy. Thus, according to the adaptive reprogramming property in Theorem 4.8, it holds

$$|\Pr[\mathbf{G}_3] - \Pr[\mathbf{G}_2]| \leq \frac{3Q_s}{2} \sqrt{Q_h + Q_s} 2^{\frac{-t \cdot \vartheta}{2}} + 2\varepsilon_{\text{QPRF}}.$$

$\mathbf{G}_4$ This game is identical to $\mathbf{G}_3$ except at the following steps. Particularly, at **Signature Generation 1.(b)** of Figure 8, for $j \in [m] \setminus J_i$, $\mathcal{B}$ directly samples $\widetilde{\text{com}}_{i,j}^{(n)} \xleftarrow{\$} R_q^{m'}$. For $j = J_i$, $\mathcal{B}$ still computes $\widetilde{\text{com}}_{i,J_i}^{(n)}$ in the original way. Thus, according to the fact that the pseudorandomness of Inv-H is followed from the $\text{MLWE}_{q,(\ell+k),m',\chi}$ with $\chi = D_\sigma$, it holds

$$|\Pr[\mathbf{G}_4] - \Pr[\mathbf{G}_3]| \leq Q_s \cdot t \cdot (m - 1) \cdot \text{Adv}_{\text{MLWE}_{q,(\ell+k),m',\chi}}.$$

$\mathbf{G}_5$ This game is identical to $\mathbf{G}_4$ except at the following steps. Particularly, at **Signature Generation 1.(a).ii** of Figure 8, for $j \in [m] \setminus J_i$, $\mathcal{B}$ directly omit the computations of $\mathbf{z}_{i,j}^{(n)}$. For $j = J_i$, $\mathcal{B}$ still computes $\mathbf{z}_{i,J_i}^{(n)}$ in the original way. As the $\{\mathbf{z}_{i,j}^{(n)}\}_{j \in [m] \setminus J_i}$ have never been used in the following steps, it holds

$$\Pr[\mathbf{G}_5] = \Pr[\mathbf{G}_4].$$

$\mathbf{G}_6$ This game is identical to $\mathbf{G}_5$ except at the following points.

Honest party oracle simulatuon. The $\mathcal{B}$ doesn't honestly generate transcript $((\text{com}_i^{(n)}, \mathbf{w}_i^{(n)}, r_i^{(n)}), c_{i,J_i}, \mathbf{z}_{i,J_i}^{(n)})_{i \in [t]}$ through using the secret key share sk_n anymore, but instead simulating it as follows.

- **Signature Generation 1.(a).ii.** B. Sample $\mathbf{z}_{i,J_i}^{(n)} \leftarrow D_\sigma^{\ell+k}$, output it with probability $1/M$.
- **Signature Generation 1.(c).** Derive randomness $r_i^{(n)} \leftarrow \text{Eqv}_{\text{td}}(\text{td}, \text{com}_i^{(n)})$, $\mathbf{w}_i^{(n)} = \bar{\mathbf{A}} \mathbf{z}_{i,J_i}^{(n)} - c_{i,J_i} \mathbf{t}_n$.

Clearly, the above approach is different from the usual simulation of Fiat-Shamir-with-abort signature, as $H_0(i, j, \mu, \text{pk}, \text{ck}, \mathbf{A}_{\text{Inv}})$ can not be reprogrammed. However, we can still prove the statistical distance between $\mathbf{G}_5$ and $\mathbf{G}_6$ are negligible. In order to simplify the presentation, it is reasonable for us to temporarily ignore the components of $\{\widehat{\text{com}}_{i,j}^{(n)}\}_{i \in [t], j \in [m] \setminus J_i}$ in the view of the adversary, as all of them are independently sampled in a public way. Particularly, we just need to prove

$$\begin{aligned} \text{SD}((\text{com}_i^{(n)}, \widehat{\text{com}}_{i,J_i}^{(n)}, \mathbf{z}_{i,J_i}^{(n)}, r_i^{(n)}, \mathbf{r}'_{i,J_i}{}^{(n)})_{i \in [t];} \\ ((\widehat{\text{com}}_i^{(n)}, \widehat{\text{com}}_{i,J_i}^{(n)}, \widehat{\mathbf{z}}_{i,J_i}^{(n)}, \widehat{r}_i^{(n)}, \widehat{\mathbf{r}}'_{i,J_i}{}^{(n)})_{i \in [t]}) \leq \text{negl}(\lambda), \end{aligned} \quad (7)$$

where the left-hand side and the right-hand side of the above (7) represent the transcripts of $\mathbf{G}_5$ and $\mathbf{G}_6$, respectively.

Notice that for different $i \in [t]$, the signature generation algorithm in Figure 8 just conducts independent operations. Thus, for (7), it suffices to show

$$\begin{aligned} \text{SD}(\text{com}_i^{(n)}, \widehat{\text{com}}_{i,J_i}^{(n)}, \mathbf{z}_{i,J_i}^{(n)}, r_i^{(n)}, \mathbf{r}'_{i,J_i}{}^{(n)}; \\ \widehat{\text{com}}_i^{(n)}, \widehat{\text{com}}_{i,J_i}^{(n)}, \widehat{\mathbf{z}}_{i,J_i}^{(n)}, \widehat{r}_i^{(n)}, \widehat{\mathbf{r}}'_{i,J_i}{}^{(n)}) \leq \text{negl}(\lambda), \end{aligned} \quad (8)$$

for certain $i \in [t]$. This is because we can get (7) from (8), through using simple hybrid arguments.

Furthermore, according to the following facts:

1. $\mathbf{r}'_{i,J_i}{}^{(n)}$ and $\widehat{\mathbf{r}}'_{i,J_i}{}^{(n)}$ are independently sampled from the identical distribution,
 2. $\widehat{\text{com}}_{i,j}^{(n)} \leftarrow \text{Inv-H}(\mathbf{z}_{i,j}^{(n)}) = \mathbf{A}_{\text{Inv}} \cdot \mathbf{z}_{i,j}^{(n)} + \mathbf{r}'_{i,j}{}^{(n)}$,
 3. $\widehat{\widehat{\text{com}}}_{i,j}^{(n)} \leftarrow \text{Inv-H}(\widehat{\mathbf{z}}_{i,j}^{(n)}) = \mathbf{A}_{\text{Inv}} \cdot \widehat{\mathbf{z}}_{i,j}^{(n)} + \widehat{\mathbf{r}}'_{i,j}{}^{(n)}$,
- it suffices to show

$$\text{SD}(\text{com}_i^{(n)}, \mathbf{z}_{i,J_i}^{(n)}, r_i^{(n)}; \widehat{\text{com}}_i^{(n)}, \widehat{\mathbf{z}}_{i,J_i}^{(n)}, \widehat{r}_i^{(n)}) \leq \text{negl}(\lambda). \quad (9)$$

This is because there exists efficient reduction from (8) to (9). Particularly, given one instance in (9), we can simulate the corresponding instance in (8), through using the above mentioned facts.

Furthermore, according to the facts

1. $c_{i,J_i} = \widehat{c}_{i,J_i} \leftarrow H_0(i, J_i, \mu, \text{pk}, \text{tck}, \mathbf{A}_{\text{Inv}})$,
2. $\mathbf{w}_i^{(n)} = \overline{\mathbf{A}} \mathbf{y}_i^{(n)} = \overline{\mathbf{A}} \mathbf{z}_{i,J_i}^{(n)} - c_{i,J_i} \mathbf{t}_n$,
3. $\widehat{\mathbf{w}}_i^{(n)} = \overline{\mathbf{A}} \widehat{\mathbf{z}}_{i,J_i}^{(n)} - \widehat{c}_{i,J_i} \mathbf{t}_n$,
4. $\text{com}_i^{(n)} \leftarrow \text{Eqv-TCommit}_{\text{tck}}(\text{td})$, and $r_i^{(n)} \leftarrow \text{Eqv}_{\text{tck}}(\text{td}, \text{com}_i^{(n)}, \widehat{\mathbf{w}}_i^{(n)})$,
5. $\widehat{\text{com}}_i^{(n)} \leftarrow \text{Eqv-TCommit}_{\text{tck}}(\text{td})$, and $\widehat{r}_i^{(n)} \leftarrow \text{Eqv}_{\text{tck}}(\text{td}, \widehat{\text{com}}_i^{(n)}, \mathbf{w}_i^{(n)})$,

it suffices to show

$$\text{SD}(\mathbf{z}_{i,J_i}^{(n)}; \widehat{\mathbf{z}}_{i,J_i}^{(n)}) \leq \text{negl}(\lambda). \quad (10)$$

This is because there exists efficient reduction from (9) to (10). Particularly, given one instance in (10), we can simulate the corresponding instance in (9), through using the above mentioned facts.

Furthermore, according to the property of rejection sampling in Lemma A.8, $\mathbf{z}_{i,J_i}^{(n)}$ and $\widehat{\mathbf{z}}_{i,J_i}^{(n)}$ are statistically close, i.e., it holds

$$\text{SD}(\mathbf{z}_{i,J_i}^{(n)}; \widehat{\mathbf{z}}_{i,J_i}^{(n)}) \leq \varepsilon_{\text{Rej}}. \quad (11)$$

Summing up all above analysis, we know (7) holds. And thus, we have

$$|\Pr[\mathbf{G}_6] - \Pr[\mathbf{G}_5]| \leq t \cdot Q_s \cdot \varepsilon_{\text{Rej}}.$$

Notice that up until now, i.e., in $\mathbf{G}_6$, the signing queries are answered through using the simulated algorithm SimSign_n in Figure 23, and it doesn't rely on the actual secret key $\mathbf{s}_n$ anymore.

G₇ This experiment is identical to $\mathbf{G}_6$, except with the generation of $\mathbf{A}_n$. Rather than directly sampling $s_n \xleftarrow{\$} \{0,1\}^{l_2^*}$ and computing $\mathbf{A}_n \leftarrow \mathbf{H}'_1(s_n)$, $\mathcal{B}$ first picks the random matrix $\mathbf{A} \in R_q^{k \times \ell}$ and a random seed $s_n \xleftarrow{\$} \{0,1\}^{l_2^*}$, and send out a random oracle commitment $g_n \xleftarrow{\$} \mathbf{H}_1(s_n, n)$. Then, after receiving all other random oracle commitments $g_u \in \{0,1\}^{l_1}$, $\mathcal{B}$ can extract the adversary's corresponding committed seeds $s_1, \dots, s_{n-1} \in R_q^{k \times \ell}$, and compute $\mathbf{A}_u = \mathbf{H}'_1(s_u)$ for all $u \in [n-1]$. As $\mathbf{H}_1$ has been simulated by Inj-QPRF_{k_1} in Construction 4.5, according to Theorem 4.7, this extraction can be efficiently done through using Algorithm 1. Furthermore, $\mathcal{B}$ computes $\mathbf{A}_n = \mathbf{A} - \sum_{i=1}^{n-1} \mathbf{A}_i$. And for the consistency of the following queries by $\mathcal{A}$, we need to reprogram $\text{QPRF}_{k'_1}(\cdot)$ at (s_n, n) such that $\text{QPRF}_{k'_1}(s_n, n) := \mathbf{A}_n$ (i.e., $\mathbf{H}'_1(s_n, n) := \mathbf{A}_n$). Note that the distribution of $\mathbf{A}_n$ are uniform, which follows that of $\mathbf{A}$. The formal simulation strategy is described in **Matrix Generation** part of Figure 21. According to Theorem 4.8, $\mathcal{B}$ reprograms the random oracle $\mathbf{H}'_1$ to make $\mathbf{A}_n \leftarrow \mathbf{H}'_1(s_n, n)$ will not be noticed by $\mathcal{A}$. Because the distribution of s_n are uniform, we have

$$|\Pr[\mathbf{G}_7] - \Pr[\mathbf{G}_6]| \leq \frac{3}{2} \sqrt{Q_h} 2^{-\frac{l_1^*}{2}} + 2(\varepsilon_{\text{QPRF}} + \varepsilon_{\text{Inj-QPRF}}).$$

G₈ This experiment is identical to $\mathbf{G}_7$ except that $\mathcal{B}$ simply picks the random public key share $\mathbf{t}_n \xleftarrow{\$} R_q^k$ during the key generation phase, rather than computing $\mathbf{t}_n = \overline{\mathbf{A}}\mathbf{s}_n$ with $\mathbf{s}_n \xleftarrow{\$} S_\eta^{\ell+k}$. As $\mathbf{A}$ follows the uniform distribution over $R_q^{k \times \ell}$, if the adversary $\mathcal{A}$ can distinguish $\mathbf{G}_8$ and $\mathbf{G}_7$ then we can use $\mathcal{A}$ as a distinguisher that breaks $\text{MLWE}_{q,\ell,k,\eta}$ assumption; hence we have

$$|\Pr[\mathbf{G}_8] - \Pr[\mathbf{G}_7]| \leq \text{Adv}_{\text{MLWE}_{q,\ell,k,\eta}}.$$

G₉ This experiment is identical to **G₈**, except with the generation of $\mathbf{t}_n$. Rather than sampling $\mathbf{t}_n \xleftarrow{\$} R_q^k$, $\mathcal{B}$ first choose $\mathbf{t} \xleftarrow{\$} R_q^k$, and send out a random $g'_n \xleftarrow{\$} \{0, 1\}^{l_2}$. Then, after receiving all others random oracle commitments $g'_u \in \{0, 1\}^{l_2}$, $\mathcal{B}$ can extract the adversary's corresponding committed shares $\mathbf{t}_1, \dots, \mathbf{t}_{n-1} \in R_q^k$. As H_2 has been simulated by Inj-QPRF_{k_2} in Construction 4.5, according to Theorem 4.7, this extraction can be efficiently done through using Algorithm 1. Furthermore, $\mathcal{B}$ computes $\mathbf{t}_n = \mathbf{t} - \sum_{i=1}^{n-1} \mathbf{t}_i$. And for the consistency of the following queries by $\mathcal{A}$, we need to reprogram $\text{Inj-QPRF}_{k_2}(H_2)$ at $(\mathbf{t}_n, n)$ such that $\text{Inj-QPRF}_{k_2}(\mathbf{t}_n, n) := g'_n$ (i.e., $H_2(\mathbf{t}_n, n) := g'_n$). Note that the distribution of $\mathbf{t}_n$ are uniform, which follows that of $\mathbf{t}$. The formal simulation strategy is described in **Key Pair Generation** part of Figure 21.

According to Theorem 4.8, $\mathcal{B}$ reprograms the random oracle H_2 to make $g'_n \leftarrow H_2(\mathbf{t}_n, n)$ will not be noticed by $\mathcal{A}$. Because the distribution of $\mathbf{t}_n$ are uniform, we have

$$|\Pr[\mathbf{G}_9] - \Pr[\mathbf{G}_8]| \leq \frac{3}{2} \sqrt{Q_h} 2^{-\frac{kN \log q}{2}} + 2(\varepsilon_{\text{QPRF}} + \varepsilon_{\text{Inj-QPRF}}).$$

Up until now, notice that the key generation query is simulated according to SimGen_n in Figure 21. This implies that $\mathcal{B}$ can be fully simulated without using any secret key.

Based on this, our next goal is to show that in **G₉**, the probability of $\mathcal{A}$ forging a valid message-signature pair $(\mu^*, \widehat{\text{Sig}}^*)$ for $\mu^* \notin \text{Mset}$ is negligible in λ . In order to do this, we need to establish an efficient reduction: if $\mathcal{A}$ outputs a valid forgery $(\mu^*, \widehat{\text{Sig}}^*)$ for $\mu^* \notin \text{Mset}$, then $\mathcal{B}$ can solve some underlying hard problems. Particularly, we need to embed an instance of $\text{MSIS}_{q,k,k+\ell+1,\beta}$, which is denoted as $[\mathbf{A}'|\mathbf{I}]$ with $\mathbf{A}' \xleftarrow{\$} R_q^{k \times (\ell+1)}$, into the public key. As in **G₉** the combined public key $(\mathbf{A}, \mathbf{t})$ is uniformly distributed in $R_q^{k \times \ell} \times R_q^k$, replacing it with $\text{MSIS}_{q,k,k+\ell+1,\beta}$ instance doesn't change the view of the adversary at all. Particularly, we can set $\mathbf{A}' := [\mathbf{A}|\mathbf{t}]$. Moreover, according to the simulation of H_3 , it is guaranteed that the challenge commitment key satisfies $\text{ck} \leftarrow \text{Eqv-CGen}(\text{cpp}_{\text{Eqv}})$.

Below, we follow the similar proof idea of [42] for proving the unforgeability in **G₉**, i.e., $\Pr[\mathbf{G}_9] \leq \text{negl}(\lambda)$. Particularly, we first show that there is an efficient extractor Ext in Figure 20, such that given a valid message-signature pair $(\mu^*, \widehat{\text{Sig}}^*)$ for $\mu^* \notin \text{Mset}$ in **G₉**, $\text{Ext}(\text{pp}, \text{pk}, \mu^*, \widehat{\text{Sig}}^*)$ can output a solution for MSIS problem with overwhelming probability, just as formalized in the following Lemma C.3. And then, we bound the probability $\Pr[\mathbf{G}_9]$ by the union bound of two events happen: Ext succeeds and Ext fails.

Lemma C.3 *There exists an extractor Ext presented in Figure 20, such that if $\mathcal{A}$ could output a valid forgery $(\mu^*, \widehat{\text{Sig}}^*)$ for $\mu^* \notin \text{Mset}$ in **G₉**, then $\text{Ext}(\text{pp}, \text{pk}, \mu^*, \widehat{\text{Sig}}^*)$ will output a solution for $\text{MSIS}_{q,k,k+\ell+1,\beta}$ problem except with probability $(2(Q_h + 1)2^{-(t \log m)/2} + \varepsilon_{\text{bind}})$, where $\varepsilon_{\text{bind}}$ is the advantages for any polynomial time quantum adversary to break the binding of Eqv-TCOM .*

Proof. According to the basic structure of valid forgery signature $\widehat{\text{Sig}}^*$, for any $i \in [t]$, if there exists one different index $j \neq J_i^*$ such that $\mathbf{z}_{i,j}$ satisfies the following two conditions:

1. $\|\mathbf{z}_{i,j}\| \leq B_n$;
2. $\text{Eqv-Open}_{\text{ck}^*}(\text{com}_i^*, r_i^*, \mathbf{w}_i := \overline{\mathbf{A}}\mathbf{z}_{i,j} - c_{i,j}^* \mathbf{t}) = 1$,

then we know

$$\begin{aligned} & \text{Eqv-Open}_{\text{ck}^*}(\text{com}_i^*, r_i^*, \mathbf{w}_i^* := \overline{\mathbf{A}}\mathbf{z}_i^* - c_{i,J_i^*}^* \mathbf{t}) \\ &= \text{Eqv-Open}_{\text{ck}^*}(\text{com}_i^*, r_i^*, \mathbf{w}_i^* := \overline{\mathbf{A}}\mathbf{z}_{i,j} - c_{i,j}^* \mathbf{t}) = 1, \end{aligned}$$

where $\text{ck}^* \leftarrow \text{H}_3(\mu^*, \text{pk})$, $\mathbf{A}_{\text{Inv}}^* \leftarrow \text{H}_4(\mu^*, \text{pk})$, $c_{i,j}^* \leftarrow \text{H}_0(i, j, \mu^*, \text{pk}, \text{ck}^*, \mathbf{A}_{\text{Inv}}^*)$ for all $i \in [t], j \in [m]$, $J_1^* || \dots || J_t^* \leftarrow \text{H}_5(\text{pk}, \mu^*, \{\text{com}_i^*\}_{i \in [t]}, \{c_{i,j}^*\}_{i \in [t], j \in [m]}, \{\widehat{\text{com}}_{i,j}^*\}_{i \in [t], j \in [m]})$, and $\mathbf{z}_{i,j} = \text{Inv}_{\mathbf{A}_{\text{Inv}}^*}(\widehat{\text{com}}_{i,j}^*, \mathbf{T}_{\mathbf{A}_{\text{Inv}}^*})$ with $(\mathbf{A}_{\text{Inv}}^*, \mathbf{T}_{\mathbf{A}_{\text{Inv}}^*}) \leftarrow \text{TrapGen}(1^{k+\ell}, q, m')$, through using $r = \text{QPRF}_{k_4}(\mu^*, \text{pk})$ as the running randomness.

Then, according to the binding property of Eqv-TCOM, we have $\overline{\mathbf{A}}\mathbf{z}_i^* - c_{i,J_i^*}^* \mathbf{t} = \overline{\mathbf{A}}\mathbf{z}_{i,j} - c_{i,j}^* \mathbf{t}$, from which we get

$$(\mathbf{A}|\mathbf{I}|\mathbf{t}) \begin{pmatrix} \mathbf{z}_i^* - \mathbf{z}_{i,j} \\ c_{i,j}^* - c_{i,J_i^*}^* \end{pmatrix} = 0.$$

Recalling that $(\mathbf{A}'|\mathbf{I}) = (\mathbf{A}|\mathbf{t}|\mathbf{I})$ is an instance of $\text{MSIS}_{q,k,k+\ell+1,\beta}$ problem, we have found a valid solution with $\beta = \sqrt{(2B_n)^2 + 4\kappa}$, since $\|\mathbf{z}_i^* - \mathbf{z}_{i,j}\| \leq 2B_n$ and $0 < \|c_{i,J_i^*}^* - c_{i,j}^*\| \leq \sqrt{4\kappa}$.

Thus, with the valid forgery $\widehat{\text{Sig}}^*$ satisfying the above two conditions for certain $i \in [t], j \neq J_i^*$, the extractor $\text{Ext}(\text{pp}, \text{pk}, \mu^*, \widehat{\text{Sig}}^*)$ indeed successfully extracts the solution for the underlying MSIS problem, assuming the binding property of Eqv-TCOM.

Below, it suffices to show that for a valid forgery $\widehat{\text{Sig}}^*$, such extractor Ext always wins, except with negligible probability. In fact, we just need to analyze the probability of its complementary event: for a valid forgery Sig^* , there is exactly only one index $J_i^* \in [m]$ satisfying the above two conditions, which clearly derives Ext fails.

More formally, we define the following event $\mathbf{E}$.

- $\mathbf{E}$: The valid forgery signature in $\mathbf{G}_9$ can be only verified successfully for $\mathbf{z}_{i,j} = \text{Inv}_{\mathbf{A}_{\text{Inv}}^*}(\widehat{\text{com}}_{i,j}^*, \mathbf{T}_{\mathbf{A}_{\text{Inv}}^*})$, with $j = J_i^*$, where $\mathbf{T}_{\mathbf{A}_{\text{Inv}}^*} \leftarrow \text{H}'_4(\mu^*, \text{pk})$. According to the binding property of Eqv-TCOM, this means the following two conditions happen simultaneously:

$$\text{Eqv-Open}_{\text{ck}^*}(\text{com}_i^*, r_i^*, \mathbf{w}_i^* := \overline{\mathbf{A}}\mathbf{z}_i^* - c_{i,J_i^*}^* \mathbf{t}) = 1,$$

$$\text{Eqv-Open}_{\text{ck}^*}(\text{com}_i^*, r_i^*, \mathbf{w}_i := \overline{\mathbf{A}}\mathbf{z}_{i,j} - c_{i,j}^* \mathbf{t}) \neq 1, \text{ for } j \neq J_i^*.$$

Clearly, based on the binding property of Eqv-TCOM, if the above defined events E do not happen, then the above extraction by Ext should be successful. Particularly, it holds

$$\Pr[\text{Ext succeeds}] \geq 1 - \Pr[E]$$

Thus, it suffices to show the upper bounds of $\Pr[E]$ is negligible in λ , i.e., $\Pr[E] \leq 2(Q_h + 1) \cdot 2^{-(t \cdot \log m)/2}$, in the following Lemma C.4. $\square$

Lemma C.4 (E) *Suppose after making Q_s times signature queries for μ_i in $\mathbf{G}_9$ and Q_h times queries to H_5 , $\mathcal{A}$ gives a forgery $\widehat{\text{Sig}}^*$ such that $\text{QDS}_2.\text{Ver}(\text{pk}, \mu^*, \widehat{\text{Sig}}^*) = 1$, where $\widehat{\text{Sig}}^* = (\{\text{com}_i^*\}_{i \in [t]}, \{\widehat{\text{com}}_{i,j}^*\}_{i \in [t], j \in [m]}, \{r_i^*\}_{i \in [t]}, \{r_i'^*\}_{i \in [t]}, \{z_i^*\}_{i \in [t]})$. Then*

$$\Pr[E] \leq 2(Q_h + 1)2^{-(t \cdot \log m)/2}.$$

Proof (Sketch). This proof is almost identical to that of Lemma 17 of [42]. Particularly, we define a valid forgery $\widehat{\text{Sig}}^*$ just as in the above event E to be an H_5 solution. Thus, it suffices to prove the adversary making at most Q_s times signature queries and Q_h queries to H_5 outputs an H_5 solution with probability at most $\Pr[E] \leq 2(Q_h + 1)2^{-(t \cdot \log m)/2}$. If such H_5 is a random function, this result can be easily proven. However, during the signature simulation process in Figure 23, H_5 has been reprogrammed for Q_s times, and thus it can not be treated as a random function. In order to conquer this dilemma, we can define the equivalent H_5' solution, where H_5' is another carefully defined random function. Here, we just defer more details to the proof of Lemma 17 in [42].

Notice also that compared with the above mentioned proof in [42], there is still one obvious difference: in Figure 22, H_5 is simulated as a QPRF which is not truly random. But, we notice that this is not the essential obstacle. This is because for an efficient quantum adversary, we can securely switch between QPRF and truly random function, through using additional hybrid argument step. For simplicity, we omit the related details here. $\square$

According to Lemma C.3, if the extraction is successful, $\mathcal{B}$ can solve the $\text{MSIS}_{q,k,\ell+1,\beta}$ problem with $\beta = \sqrt{(2B_n)^2 + 4\kappa}$.

Thus, we get

$$\Pr[\text{ExSuccess}] \leq \text{Adv}_{\text{MSIS}_{q,k,\ell+1,\beta}}.$$

Moreover, according to the above analysis, we know for the valid forgery $\widehat{\text{Sig}}^*$, there are two cases deriving Ext fails: (1) the event E happens; (2) the binding property of Eqv-TCOM is broken. Thus, we have the following equation:

$$\Pr[\text{ExFail}] \leq 2(Q_h + 1)2^{-(t \cdot \log m)/2} + \varepsilon_{\text{bind}}.$$

Finally, we know

$$\begin{aligned} \Pr[\mathbf{G}_9] &= \Pr[\mathbf{G}_9 | \text{ExFail}] \Pr[\text{ExFail}] + \Pr[\mathbf{G}_9 | \text{ExSuccess}] \Pr[\text{ExSuccess}] \\ &\leq \Pr[\text{ExFail}] + \Pr[\text{ExSuccess}] \\ &\leq 2(Q_h + 1)2^{-(t \cdot \log m)/2} + \text{Adv}_{\text{MSIS}_{q,k,\ell+1,\beta}} + \varepsilon_{\text{bind}}. \end{aligned}$$

Summing up all above analysis, we conclude that the statement of theorem holds.

□

D Supplementary for QMS₂ in Section 6

D.1 Non-interactive Zero-knowledge Proof of Knowledge

Let's recall the notion of non-interactive zero-knowledge proof of knowledge (NIZKPoK) system.

Definition D.1 ([17]) *Let $\mathcal{R}$ be a relation (and $\mathcal{L}_{\mathcal{R}}$ is the related language). A non-interactive proof system Π for $\mathcal{R}$ (or $\mathcal{L}_{\mathcal{R}}$) is a tuple of PPT algorithms (Setup, Prove, Verify) having the following interfaces (where 1^λ are implicit inputs to Prove, Verify):*

- Setup(1^λ): given a security parameter λ , outputs a string CRS.
- Prove(CRS, x, w): given a string CRS and a statement-witness pair $(x, w) \in \mathcal{R}$ (or w is the witness for $x \in \mathcal{L}_{\mathcal{R}}$), outputs a proof π .
- Verify(CRS, x, π): given a string CRS, a statement x , and a proof π , either accepts or rejects.

A secure NIZKPoK should have four properties: Completeness, Soundness, and Zero-knowledge, Straight-line extractability.

- Completeness: for every $(x, w) \in \mathcal{R}$ and every λ , Verify(CRS, x, π) accepts with probability 1, over the choice of $\text{CRS} \leftarrow \text{Setup}(1^\lambda)$ and $\pi \leftarrow \text{Prove}(\text{CRS}, x, w)$.
- Soundness: let $\mathcal{L}_{\mathcal{R}}$ be the language defined by relation $\mathcal{R}$. For any PPT adversary $\mathcal{A}$,

$$\Pr_{\text{CRS} \leftarrow \text{Setup}(1^\lambda)} [\exists x \text{ s.t. } \pi^* \leftarrow \mathcal{A}(\text{CRS}, x) : \text{Verify}(\text{CRS}, x, \pi^*) \text{ accepts} \wedge x \notin \mathcal{L}_{\mathcal{R}}] \leq \text{negl}(\lambda).$$

- Zero-Knowledge: There exists two PPT algorithms (SimSetup, SimProve), such that, for any PPT adversary $\mathcal{A}$ we have $|\Pr[\mathcal{A} \text{ wins}] - \frac{1}{2}| \leq \text{negl}(\lambda)$ in the following game:
 1. The challenger samples $(\widehat{\text{CRS}}, \text{tk}) \leftarrow \text{SimSetup}(1^\lambda)$ such that $\widehat{\text{CRS}}$ is indistinguishable from CRS output by Setup, and gives the simulated $\widehat{\text{CRS}}$ to $\mathcal{A}$.
 2. The adversary $\mathcal{A}$ chooses $(x, w) \in \mathcal{R}$ and gives these to the challenger.
 3. The challenger samples $\pi_0 \leftarrow \text{Prove}(\text{CRS}, x, w)$, $\pi_1 \leftarrow \text{SimProve}(\widehat{\text{CRS}}, x, \text{tk})$, $b \leftarrow \{0, 1\}$ and gives π_b to $\mathcal{A}$.
 4. The adversary $\mathcal{A}$ outputs a bit b' and wins if $b' = b$.

Notice that in the above zero-knowledge game, if we allow the adversary $\mathcal{A}$ to choose any polynomial numbers of $(x_i, w_i) \in \mathcal{R}$, and all the resulting $\{\pi_{i,0}\}$ and $\{\pi_{i,1}\}$ are still indistinguishable, we say that Π is a multi-proof NIZKPoK system.

Moreover, we consider straight-line extractability. Here, the adversary can pick the statements adaptively. In order to perform extraction in this stronger setting, the common reference string is simulated and the corresponding trapdoor is provided to the extractor.

Definition D.2 (Multi-Proof Straight-Line Extractability [?]) An NIZKPoK system is multi-proof straight-line extractable, if there exists a PPT oracle simulator SimSetup and a PPT extractor Ext with the following properties:

CRS indistinguishability. For any PPT adversary $\mathcal{A}$, we have

$$\begin{aligned} \text{Adv}(\mathcal{A}) : &= \left| \Pr[\text{CRS} \leftarrow \text{Setup}(1^\lambda) : \mathcal{A}(\text{CRS}) = 1] \right. \\ &\quad \left. - \Pr[(\widehat{\text{CRS}}, \text{tk}) \leftarrow \text{SimSetup}(1^\lambda) : \mathcal{A}(\widehat{\text{CRS}}) = 1] \right| \leq \text{negl}(\lambda). \end{aligned}$$

Straight-Line Extractability. There exists constants c, e_1, e_2 and polynomial $p(\lambda)$ such that for any $Q_H = \text{poly}(\lambda)$, $Q_s = \text{poly}(\lambda)$ and PPT adversary $\mathcal{A}$ that makes at most Q_H random oracle queries, and generates at most Q_s statement proof pairs with

$$\begin{aligned} \Pr \left[(\widehat{\text{CRS}}, \text{tk}) \leftarrow \text{SimSetup}(1^\lambda), \{(x_i, \pi_i)\}_{i \in [Q_s]} \leftarrow \mathcal{A}^{H(\cdot)}(\widehat{\text{CRS}}) : \right. \\ \left. \forall i \in [Q_s], \text{Verify}^{H(\cdot)}(\widehat{\text{CRS}}, x_i, \pi_i) = 1 \right] \geq \mu(\lambda), \end{aligned}$$

we have

$$\begin{aligned} \Pr \left[(\widehat{\text{CRS}}, \text{tk}) \leftarrow \text{SimSetup}(1^\lambda), \{(x_i, \pi_i)\}_{i \in [Q_s]} \leftarrow \mathcal{A}^{H(\cdot)}(\widehat{\text{CRS}}), \right. \\ \left. \{w_i \leftarrow \text{Ext}(1^\lambda, Q_H, Q_s, 1/\mu, \text{tk}, x_i, \pi_i)\}_{i \in [Q_s]} : \right. \\ \left. \forall i \in [Q_s], (x_i, w_i) \in \mathfrak{R} \wedge \text{Verify}^{H(\cdot)}(\widehat{\text{CRS}}, x_i, \pi_i) = 1 \right] \\ \geq \frac{1}{2} \cdot \mu(\lambda) - \text{negl}(\lambda). \end{aligned}$$

Moreover, the running time of Ext is upper bounded by $Q_H^{e_1} \cdot Q_s^{e_2} \cdot \frac{1}{\mu^c} \cdot p(\lambda)$.

Below, we recall the instantiation of NIZKPoK for MSIS relation. Particularly, for the following language

$$\mathcal{L}_{B,q} = \left\{ (\bar{\mathbf{A}}, \mathbf{u}) \in R_q^{k \times (\ell+k)} \times R_q^k : \exists \mathbf{x} \in R^{\ell+k} \text{ such that } 0 < \|\mathbf{x}\| \leq B \text{ and } \bar{\mathbf{A}} \cdot \mathbf{x} = \mathbf{u} \right\},$$

there are practical multi-proof straight-line extractable NIZKPoK systems for $\mathcal{L}_{B,q}$, according to [17].

D.2 Security of QMS₂

Theorem D.3 (Restatement of Theorem 6.1) Suppose the trapdoor commitment schemes Inv-TCOM and Eqv-TCOM are secure, additively homomorphic, have uniform keys and uniform commitment. Particularly, the output of $\text{Eqv-TCommit}_{\text{tk}}(\text{td})$ has sufficient min-entropy ϑ . Suppose $\Pi = (\Pi.\text{Setup}, \Pi.\text{Prove}, \Pi.\text{Verify})$ is a multi-proof straight-line extractable NIZKPoK system for $\mathcal{L}_{B,q}$, just as defined in Definition D.1. And suppose there exists QPRF that can be programmable and invertible simultaneously. For any quantum polynomial-time adversary $\mathcal{A}$ that initiates a single key generation protocol by querying $\mathcal{O}_n^{\text{QMS}_2}$

with $sid = 0$, initiates Q_s signature generation protocols by querying $\mathcal{O}_n^{\text{QMS}_2}$ with $sid \neq 0$, and makes Q_h quantum superpositions queries to random oracle H_0, H_3, H_4, H_5 , the protocol QMS_2 of Figures 10, 12, 11 is QMS-SUF-CMA secure under $\text{MSIS}_{q,k,k+\ell+1,\beta}$ and $\text{MLWE}_{q,\ell,k,\eta}$ assumptions in the QROM, where $\beta = \sqrt{(2B_n)^2 + 4\kappa^2 + \eta^2(4\kappa^2 \cdot (\ell + k))}$. Concretely, using other parameters specified in Table 2, the advantage of $\mathcal{A}$ is bounded as follows.

$$\begin{aligned} \text{Adv}_{\text{QMS}_2}^{\text{UF-CMA}}(\mathcal{A}) &\leq 2\varepsilon_{\text{Inj-QPRF}} + 5\varepsilon_{\text{QPRF}} + e(Q_h + Q_s + 1) \left[(Q_h + Q_s) \cdot t \cdot \varepsilon_{\text{td}} \right. \\ &\quad + 2(Q_h + Q_s) \cdot \varepsilon_{\text{QPRF}} + \frac{3Q_s}{2} \sqrt{Q_s + Q_h} 2^{-\frac{t \cdot \vartheta}{2}} + 2\varepsilon_{\text{QPRF}} + t \cdot Q_s \cdot (m - 1) \\ &\quad \cdot \text{Adv}_{\text{MLWE}_{q,(\ell+k),m',\chi}} + t \cdot Q_s \cdot (\varepsilon_{\text{Rej}} + \varepsilon_{\text{td}}) + \frac{3}{2} \sqrt{Q_h} (2^{-\frac{t_1^*}{2}} + 2^{-\frac{kN \log q}{2}}) \\ &\quad + 4(\varepsilon_{\text{QPRF}} + \varepsilon_{\text{Inj-QPRF}}) + \text{Adv}_{\text{MLWE}_{q,\ell,k,\eta}} + 2(Q_h + 1) 2^{-(t \log m)/2} \\ &\quad + Q_s \cdot t \cdot \text{Adv}_{\text{MSIS}_{q,m',(\ell+k+m'),\sigma \cdot \sqrt{2N}}} + \frac{Q_s(Q_s + 1)}{2} \cdot 2^{-t \cdot \vartheta} + \varepsilon_{\text{bind}} \\ &\quad \left. + \text{Adv}_{\text{MSIS}_{q,k,k+\ell+1,\beta}} \right] \end{aligned}$$

Proof (sketch). As this proof is almost same as the proof for QDS_2 in Theorem 5.2, for simplicity of presentation, we just highlight the difference: how to use $\text{Ext}(\text{pp}, \mu^*, \text{Sig}^*)$ in Figure 24 to output a solution for $\text{MSIS}_{q,k,\ell+1,\beta}$ problem for a different β , with the help of multi-proof NIZKPoK system Π .

According to the basic structure of valid forge signature Sig^* , for any $i \in [t]$, if there exists one different index $j \neq J_i^*$ such that $\mathbf{z}_{i,j}$ satisfies: (1) $\|\mathbf{z}_{i,j}\| \leq B_n$; (2) $\text{Eqv-Open}_{\text{ck}^*}(\text{com}_i^*, r_i^*, \mathbf{w}_i := \overline{\mathbf{A}}\mathbf{z}_{i,j} - \sum_{u \in [n^*-1]} a_u c_{i,j}^* \mathbf{t}_u - a_n c_{i,j}^* \mathbf{t}_n) = 1$, then we know

$$\begin{aligned} &\text{Eqv-Open}_{\text{ck}^*}(\text{com}_i^*, r_i^*, \mathbf{w}_i^* := \overline{\mathbf{A}}\mathbf{z}_{i,J_i^*}^* - \sum_{u \in [n^*-1]} a_u c_{i,J_i^*}^* \mathbf{t}_u - a_n c_{i,J_i^*}^* \mathbf{t}_n) \\ &= \text{Eqv-Open}_{\text{ck}^*}(\text{com}_i^*, r_i^*, \mathbf{w}_i := \overline{\mathbf{A}}\mathbf{z}_{i,j} - \sum_{u \in [n^*-1]} a_u c_{i,j}^* \mathbf{t}_u - a_n c_{i,j}^* \mathbf{t}_n), \end{aligned}$$

where $\text{ck}^* \leftarrow H_3(\mu^*, \mathbf{L}^*)$, $\text{ck}'^* \leftarrow H_4(\mu^*, \mathbf{L}^*)$, $c_{i,j}^* \leftarrow H_0(i, j, \mu^*, \text{ck}^*, \text{ck}'^*, \mathbf{L}^*)$ for all $i \in [t], j \in [m]$, $J_1^* || \dots || J_t^* \leftarrow H_5(\mu^*, \{\text{com}_i^*\}_{i \in [t]}, \{c_{i,j}^*\}_{i \in [t], j \in [m]}, \{\widehat{\text{com}}_{i,j}^*\}_{i \in [t], j \in [m]}, \mathbf{L}^*)$, and $\mathbf{z}_{i,j} = \text{Inv}_{\text{ck}'^*}(\widehat{\text{com}}_{i,j}^*, \text{td}')$ with $\text{td}' \leftarrow \text{Inv-TCGen}(\text{cpp}_{\text{Inv}}, r)$, $r = \text{QPRF}_{k_4}(\mu^*, \mathbf{L}^*)$.

We know that if the above equality holds, then we have

$$\overline{\mathbf{A}}\mathbf{z}_{i,J_i^*}^* - \sum_{u \in [n^*-1]} a_u c_{i,J_i^*}^* \mathbf{t}_u - a_n c_{i,J_i^*}^* \mathbf{t}_n = \overline{\mathbf{A}}\mathbf{z}_{i,j} - \sum_{u \in [n^*-1]} a_u c_{i,j}^* \mathbf{t}_u - a_n c_{i,j}^* \mathbf{t}_n. \quad (12)$$

Furthermore, for any $u \in [n^* - 1]$, we can extract $\mathbf{sk}_u = \mathbf{s}_u$, i.e., run $\Pi.\text{Ext}(\text{CRS}, \text{tk}, \overline{\mathbf{A}}, \mathbf{t}_u, \pi_u)$ to get $\mathbf{s}_u$, such that $\overline{\mathbf{A}} \cdot \mathbf{s}_u = \mathbf{t}_u$. In this case, (12) can be rewritten as

$$\overline{\mathbf{A}} \mathbf{z}_{i,J_i^*}^* - \sum_{u \in [n^* - 1]} (a_u c_{i,J_i^*}^* \overline{\mathbf{A}} \mathbf{s}_u) - a_n c_{i,J_i^*}^* \mathbf{t}_n = \overline{\mathbf{A}} \mathbf{z}_{i,j} - \sum_{u \in [n^* - 1]} (a_u c_{i,j}^* \overline{\mathbf{A}} \mathbf{s}_u) - a_n c_{i,j}^* \mathbf{t}_n. \quad (13)$$

Furthermore, from (13), we have

$$\overline{\mathbf{A}} \left(\mathbf{z}_{i,J_i^*}^* - \sum_{u \in [n^* - 1]} a_u c_{i,J_i^*}^* \mathbf{s}_u \right) - a_n c_{i,J_i^*}^* \mathbf{t}_n = \overline{\mathbf{A}} \left(\mathbf{z}_{i,j} - \sum_{u \in [n^* - 1]} a_u c_{i,j}^* \mathbf{s}_u \right) - a_n c_{i,j}^* \mathbf{t}_n. \quad (14)$$

From (14), we get

$$(\mathbf{A} | \mathbf{I} | \mathbf{t}_n) \begin{pmatrix} \mathbf{z}_{i,J_i^*}^* - \mathbf{z}_{i,j} + \sum_{u \in [n^* - 1]} a_u (c_{i,j^*}^* - c_{i,J_i^*}^*) \mathbf{s}_u \\ a_n (c_{i,J_i^*}^* - c_{i,j}^*) \end{pmatrix} = 0.$$

Recalling that $(\mathbf{A}' | \mathbf{I}) = (\mathbf{A} | \mathbf{t}_n | \mathbf{I})$ is an instance of $\text{MSIS}_{q,k,\ell+1,\beta}$ problem, we have found a valid solution if $\beta = \sqrt{(2B_n)^2 + 4\kappa^2 + \eta^2(4\kappa^2 \cdot (\ell + k))}$, since $\|\mathbf{z}_i^* - \mathbf{z}_{i,j}\| \leq 2B_n$, $0 < \|c_{i,J_i^*}^{(n)*} - c_{i,j}^{(n)*}\| \leq \sqrt{4\kappa}$, $\|a_u\|, \|a_n\| \leq \sqrt{\kappa}$ and $\|\mathbf{s}_u\| = \eta\sqrt{\ell + k}$.

Input : $\text{H}_0, \text{H}_3, \text{H}_4, \text{H}_5, \text{H}_{\text{agg}}, \text{Sig} = (\{\text{com}_i\}_{i \in [t]}, \{\widetilde{\text{com}}_{i,j}\}_{i \in [t], j \in [m]}, \{\mathbf{z}_i\}_{i \in [t]}, \{r_i\}_{i \in [t]}, \{\mathbf{r}'_i\}_{i \in [t]}, \mu, \mathbf{L})$,
conduct the following steps:

1. Compute $\text{ck} \leftarrow \text{H}_3(\mu, \mathbf{L})$, and $\mathbf{A}_{\text{Inv}} \leftarrow \text{H}_4(\mu, \mathbf{L}), \mathbf{T}_{\mathbf{A}_{\text{Inv}}} \leftarrow \text{H}'_4(\mu, \mathbf{L})$,
2. Compute $\tilde{\mathbf{t}} = \text{QMS}_2.\text{KAgg}(\text{pp}, \mathbf{L})$;
3. Derive challenges $c_{i,j} \leftarrow \text{H}_0(i, j, \mu, \text{ck}, \text{ck}', \mathbf{L})$ for all $i \in [t], j \in [m]$;
4. Compute $J_1 || \dots || J_t \leftarrow \text{H}_5(\{\text{com}_i\}_{i \in [t]}, \{c_{i,j}\}_{i \in [t], j \in [m]}, \{\widetilde{\text{com}}_{i,j}\}_{i \in [t], j \in [m]}, \mathbf{L})$.
5. For $j \in [n' - 1]$, compute $a_i := \text{H}_{\text{agg}}(\mathbf{L}, \mathbf{t}_j)$, and compute $a_n := \text{H}_{\text{agg}}(\mathbf{L}, \mathbf{t}_n)$;
6. For any $u \in [n^* - 1]$, we can extract $\mathbf{sk}_u = \mathbf{s}_u$, i.e., run $\Pi.\text{Ext}(\text{CRS}, \text{tk}, \overline{\mathbf{A}}, \mathbf{t}_u, \pi_u)$ to get $\mathbf{s}_u$.
7. for $i = 1$ to t do
 for $j = 1$ to m except J_i do
 for each $\mathbf{z}' \leftarrow \text{Inv}(\widetilde{\text{com}}_{i,j}, \mathbf{T}_{\mathbf{A}_{\text{Inv}}})$ do
 if $(\|\mathbf{z}'\| \leq B) \wedge (\text{Eqv-Open}_{\text{ck}}(\text{com}_i, r_i, \mathbf{w}_i := \overline{\mathbf{A}} \mathbf{z}' - c_{i,j} \tilde{\mathbf{t}} = \overline{\mathbf{A}} \mathbf{z}' - \sum_{u \in [n^* - 1]} a_u c_{i,j} \mathbf{t}_u - a_n c_{i,j} \mathbf{t}_n), \text{ where } n^* = |L|.)$
 return $\begin{pmatrix} \mathbf{z}_{i,J_i^*}^* - \mathbf{z}_{i,j} + \sum_{u \in [n^* - 1]} a_u (c_{i,j^*}^* - c_{i,J_i^*}^*) \mathbf{s}_u \\ a_n (c_{i,J_i^*}^* - c_{i,j}^*) \end{pmatrix}$.

Fig. 24. Extractor for QMS_2

□